GUIA DE SQL

DDL (DATA DEFINITION LANGUAGE)

DML (DATA MANIPULATION LANGUAGE)

Agosto de 2008

AUTORES:

Ernesto Chinkes
Diego Alarcon Regolini
Lucas Coronel
Diego Hernan Contreras Ocampo
Ernesto Goldman

GUIA DE SQL

CASO PRÁCTICO: JURASSIC PARK 3
CLASE 1: INTRODUCCION AL SQL Y CREACION DE TABLAS. 5
INTRODUCCION 5
VISIÓN GENERAL DEL SQL 5
HISTORIA. 6
COMO SON LAS SENTENCIAS 7
EJERCICIOS CLASE 1 17
CLASE 2: CREACION DE INDICES y ASIGNACION DE PERMISOS. 18
INDICES 18
ASINGACION DE PERMISOS 20
EJERCICIOS CLASE 2 31
CLASE 3: ESTRUCTURAS BÁSICAS DE DML 32
INSERTAR - INSERT INTO 32
MODIFICACION - UPDATE 33
ELIMINAR - DELETE 33
CONSULTAS – ESTRUTURA BÁSICA – SELECT 34
EJERCICIOS CLASE 3 49
CLASE 4: CONSULTAS MULTITABLA. 51
INTRODUCCIÓN 51
LA UNIÓN DE TABLAS 51
LA REUNIÓN DE TABLAS 53
EJERCICIOS CLASE 4 59
CLASE 5: CONSULTAS ANIDADAS - SUBCONSULTAS 61
SUBCONSULTAS 61
Referencias externas 62
SUBCONSULTA EN LA LISTA DE SELECCIÓN 63
SUBCONSULTA EN LA CLÁUSULA FROM 64
SUBCONSULTA EN LAS CLÁUSULAS WHERE Y HAVING 64
CONDICIONES DE SELECCIÓN CON SUBCONSULTAS 65
Subconsultas en UPDATE 69
Subconsultas en DELETE 70
EJERCICIOS CLASE 5 70
CLASE 6: CREACIÓN DE PROCEDIMIENTOS ALMACENADOS Y TRIGGERS. 72
PROCEDIMIENTOS ALMACENADOS 72
TRIGGERS 74
EJERCICIOS CLASE 6 80
ANEXO I TIPOS DE DATOS DE SQL 82
ANEXO II - EJERCICIOS RESUELTOS 90
CLASE 1 90
CLASE 2 93
CLASE 3 94
CLASE 4 100
CLASE 5 106
CLASE 6 111

CASO PRÁCTICO: JURASSIC PARK

Se presentan a continuación los requerimientos de un caso de estudio, con su respectivo modelo de datos. En este caso se basarán todos los ejercicios que se proponen en esta guía.

A continuación se describe el caso:

“La empresa Jurasic Park S.A. es la propietaria de un parque de recreación donde el público visita animales prehistóricos. Dicha empresa tiene un área (VISITAS PREHISTÓRICAS) que se encarga de ofrecer visitas guiadas para escuelas.

El área Visitas Prehistóricas vende a las escuelas visitas guiadas al parque para sus alumnos. Las escuelas realizan reservas telefónicas de las visitas que van a realizar, y se les cobra el día que realizan la visita, en el momento de ingresar (según los tipos de visitas que realicen).

Las escuelas en el momento de realizar la reserva informan, día, hora, tipos de visitas, cantidad de alumnos por visita. Pueden en una misma reserva contratar distintos tipos de visitas, para distintos grados de la escuela. En caso de que la escuela sea la primera vez que realiza una reserva se le piden su domicilio y teléfono(s).

Jurasic Park S.A. desea desarrollar un sistema de informático que le ayude en un eficiente desempeño y control del área mencionada.

Necesita obtener del mismo, todos los días, para la boletería del parque (en el momento en que ésta lo solicite) un listado con las visitas que haya reservadas para el día siguiente, de manera de que la misma sepa a quienes debe dejar entrar y cuanto cobrarles. El listado debe contener (para cada reserva del día pedido): Número de reserva de visita, Hora, Código de la escuela, Nombre de la escuela, y por cada tipo de visita de la reserva: Código de tipo de visita, Descripción de tipo de visita, Arancel por alumno del tipo de visita, Cantidad de alumnos y grado(s).

Al final del día boletería debe informar la cantidad real de ingresantes que fueron por cada reserva.

También se debe entregar todos los días un listado con las visitas reservadas para el día siguiente para el Supervisor de Guías, cuando el mismo lo solicite. El supervisor debe, en base al listado, asignar las distintas visitas a su personal a cargo. El listado debe contener (para cada visita del día pedido): Número de reserva de visita, Hora, Código de la escuela, Nombre de la escuela, y por cada tipo de visita de la reserva: Código de tipo de visita, Descripción de tipo de visita, grado(s).

Al final del día el supervisor informa los guías que se encargaron de cada tipo de visita en cada reserva.

El gerente del área desea poder pedir, en cualquier momento, un listado con la cantidad de alumnos (reales) que ingresaron. Dicha información debe salir discriminada día por día para el periodo de tiempo que él determine, y por cada día discriminado por tipo de visita. Y otro listado donde pueda observar que cantidad de cada uno de los grados reservaron, entre dos fechas a

determinar, cada una de los tipos de visitas que existen.

El sistema debe permitir, además, permitir que el responsable de Recepción de reservas, consulte quienes fueron los guías (Código de guía, y Nombre de guía), que participaron en una reserva especifica, teniendo identificado cual fue el guía responsable por cada tipo de visita de la misma, ya que puede existir una queja de alguna escuela.

Además se desea poder realizar un mailing, con Nombre, Domicilio y Teléfono(s), de todas las escuelas que alguna vez realizaron una reserva".

MODELO DE DATOS

ESCUELA
Código_Escuela
Nombre_Escuela
Domicilio_Escuela

TELEFONO_ESCUELA
Código_Escuela
Teléfono_Escuela

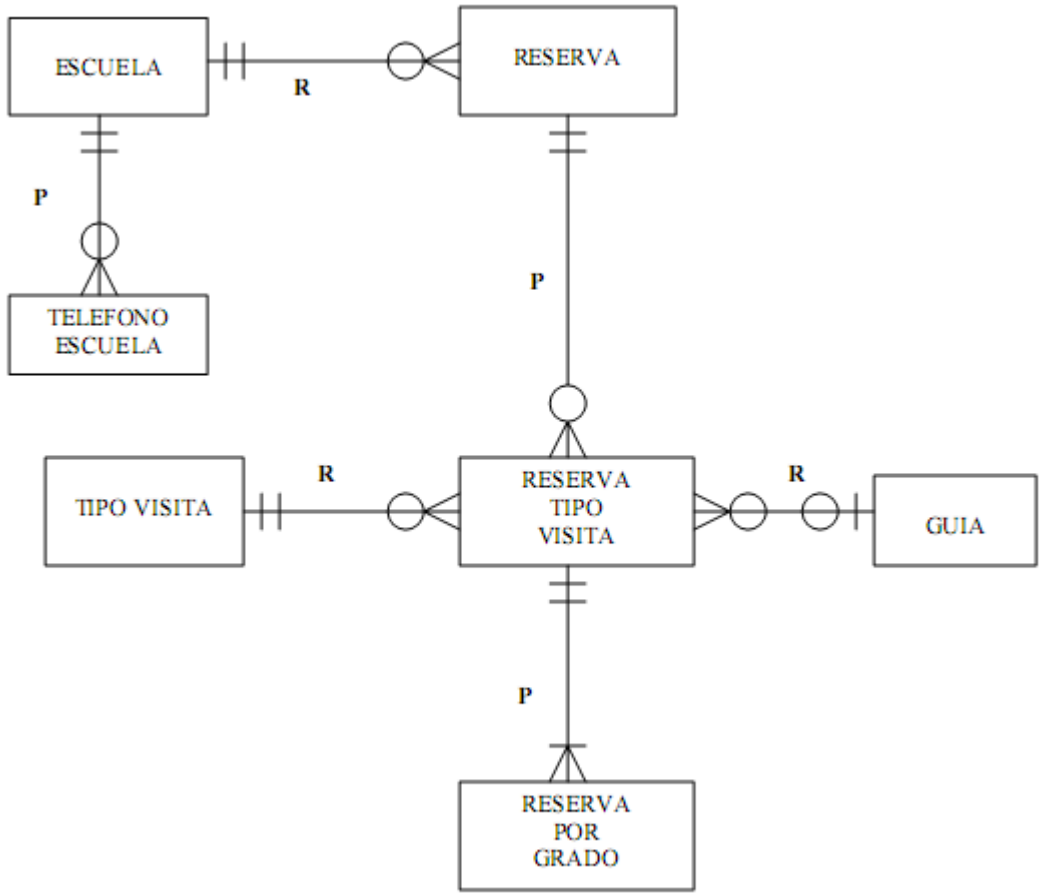
RESERVA
Número_Reserva
Fecha_Visita_Reservada
Hora_Visita_Reservada
Código_Escuela

TIPO_VISITA
Código_Tipo_Visita
Descripción_Tipo_Visita
Arancel_por_Alumno

RESERVA_TIPO_VISITA
Número_Reserva
Código_Tipo_Visita
Cantidad_Alumnos_Reservados
Cantidad_Alumnos_Reales
Código_Guía

GUIA
Código_Guía
Nombre_Guía
Apellido_Guía

RESERVA_POR_GRADO
Número_Reserva
Código_Tipo_Visita
Grado



CLASE 1: INTRODUCCION AL SQL Y CREACION DE TABLAS.

INTRODUCCION

VISIÓN GENERAL DEL SQL

El SQL (Structured Query Language - lenguaje de consulta estructurado), es un lenguaje de computación que permite interactuar con bases de datos, y que desde hace ya varios años es el estándar en los Software de administración de bases de datos relacionales.

Como su nombre indica, el SQL permite realizar consultas a la base de datos, pero se podría decir que su nombre no es representativo de todo su potencial, ya que también permite realizar las funciones de definición de la base de datos, asi como de actualización y gestión.

Las sentencias SQL se clasifican según su finalidad dando origen a dos ‘lenguajes’ o mejor dicho sublenguajes:

El DDL (Data Description Language)
El DML (Data Manipulation Language),

El Lenguaje de definición de datos (DDL), incluye órdenes para definir, modificar o borrar las tablas en las que se almacenan los datos, las relaciones entre estas, definir restricciones de integridad, de acceso, de índices, etc. Es decir, todo lo que se refiera a la definición de la base de datos.

Un ejemplo de este tipo de sentencia sería:

CREATE TABLE Reserva (codigo_reserva CHAR (10) PRIMARY KEY, fecha_visita_reserva DATETIME);

Esta sentencia permitiría crear una tabla denominada Reserva, y que tiene dos columnas codigo_reserva y fecha_visita_reservada, donde la primera es la clave primaria.

El lenguaje de manipulación de datos (DML), incluye ordenes para consultar los datos almacenados en la base de datos y también incluye órdenes para permitir actualizar la base de datos añadiendo nuevos datos, suprimiendo o modificando datos existentes previamente almacenados.

Un ejemplo de este tipo de sentencias sería:
SELECT codigo_reserva, fecha_visita_reservada FROM reserva WHERE codigo_reserva = '500'

Esta sentencia mostraría el código y fecha de las visita reservada que se encuentran en la tabla reserva y que tiene el código igual a 500.

INSERT INTO reserva (codigo_reserva, fecha_visita_reservada) VALUES ('501', '01/03/2004');

Esta sentencia insertaría una nueva fila en la tabla reserva, incoporando los valores correspondientes a las columnas codigo_reserva y fecha_visita_reservada

UPDATE reserva SET fecha_visita_reservada = '15/03/2004' WHERE codigo_reserva = '501'

Esta sentencia modificaría el valor de la fecha de visita de la tabla reserva, para la fila donde el codigo_reserva es igual a 501.

HISTORIA.

La historia de SQL empieza en 1974 con la definición, por parte de Donald Chamberlin y de otras personas que trabajaban en los laboratorios de investigación de IBM, de un lenguaje para la especificación de las características de las bases de datos que adoptaban el modelo

relacional. Este lenguaje se llamaba SEQUEL (Structured English Query Language) y se implementó en un prototipo llamado SEQUEL-XRM entre 1974 y 1975. Las experimentaciones con ese prototipo condujeron, entre 1976 y 1977, a una revisión del lenguaje (SEQUEL/2), que a partir de ese momento cambió de nombre por motivos legales, convirtiéndose en SQL. El prototipo (System R), basado en este lenguaje, se adoptó y utilizó internamente en IBM y lo adoptaron algunos de sus clientes elegidos. Gracias al éxito de este sistema, que no estaba todavía comercializado, también otras compañías empezaron a desarrollar sus productos relacionales basados en SQL. A partir de 1981, IBM comenzó a entregar sus productos relacionales y en 1983 empezó a vender DB2. En el curso de los años ochenta, numerosas compañías (por ejemplo Oracle y Sybase, sólo por citar algunos) comercializaron productos basados en SQL, que se convierte en el estándar industrial de hecho por lo que respecta a las bases de datos relacionales. En 1986, el ANSI (American National Standards Institute) adoptó SQL como estándar para los lenguajes relacionales y en 1987 se transformó en estándar ISO (International Standards Organization). Esta versión del estándar va con el nombre de SQL/86. En los años siguientes, éste ha sufrido diversas revisiones que han conducido primero a la versión SQL/89, posteriormente la versión SQL/92, y la SQL/1999, y la actual SQL/2003.

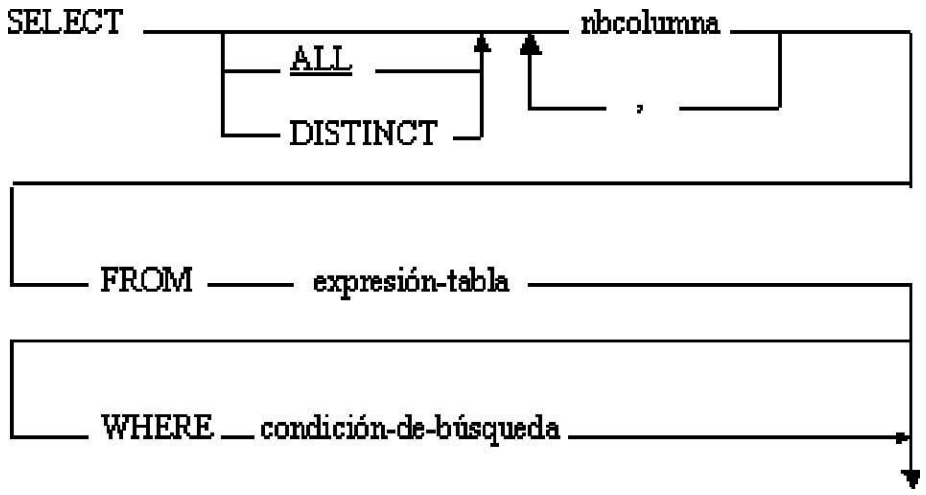
El hecho de tener un estándar definido por un lenguaje para bases de datos relacionales abre potencialmente el camino a la intercomunicabilidad entre todos los productos que se basan en él. Desde el punto de vista práctico, por desgracia las cosas fueron de otro modo. Efectivamente, en general cada productor adopta e implementa en la propia base de datos sólo el corazón del lenguaje SQL (el así llamado Entry level o al máximo el Intermediate level), extendiéndolo de manera individual según la propia visión que cada cual tenga del mundo de las bases de datos.

Actualmente, se está trabajando en la versión que hasta el momento se conoce como SQL4.

COMO SON LAS SENTENCIAS

Una sentencia SQL es como una frase (escrita en inglés) con la que decimos lo que queremos obtener y de donde obtenerlo.

Todas las sentencias empiezan con un verbo (palabra reservada que indica la acción a realizar), seguido del resto de cláusulas, algunas obligatorias y otras opcionales que completan la frase. Todas las sentencias siguen una sintaxis para que se puedan ejecutar correctamente, para describir esa sintaxis utilizaremos un diagrama sintáctico como el que se muestra a continuación.



Las palabras que aparecen en mayúsculas son palabras reservadas se tienen que poner tal cual y no se pueden utilizar para otro fin, por ejemplo, en el diagrama de la figura tenemos las palabras reservadas SELECT, ALL, DISTINCT, FROM, WHERE.
Las palabras en minúsculas son variables que el usuario deberá sustituir por un dato concreto. En el diagrama tenemos nbcolumna, expresion-tabla y condicion-de-busqueda.

Una sentencia válida se construye siguiendo la línea a través del diagrama hasta el punto que marca el final. Las líneas se siguen de izquierda a derecha y de arriba abajo. Cuando se quiere alterar el orden normal se indica con una flecha.

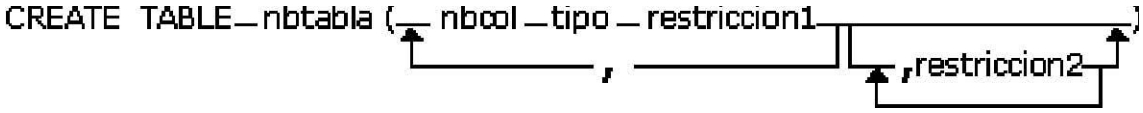
El diagrama sintáctico de la figura se interpretaría como sigue:

Hay que empezar por la palabra SELECT, después puedes poner ALL o bien DISTINCT o nada, a continuación un nombre de columna, o varios separados por comas, a continuación la palabra FROM y una expresión-tabla, y por último de forma opcional puedes incluir la cláusula WHERE con una condición-de-búsqueda.

CREATE TABLE (Creación de Tablas)

La sentencia CREATE TABLE sirve para crear la estructura de una tabla. Permite definir las columnas que tiene y ciertas restricciones que deben cumplir esas columnas.

La sintaxis es la siguiente:



nbtabla: nombre de la tabla que estamos definiendo.

nbcol: nombre de la columna que estamos definiendo.

tipo: tipo de dato de la columna, todos los datos almacenados en la columna deberán ser del tipo definido. (ver Anexo I - Tipos de Datos de SQL).

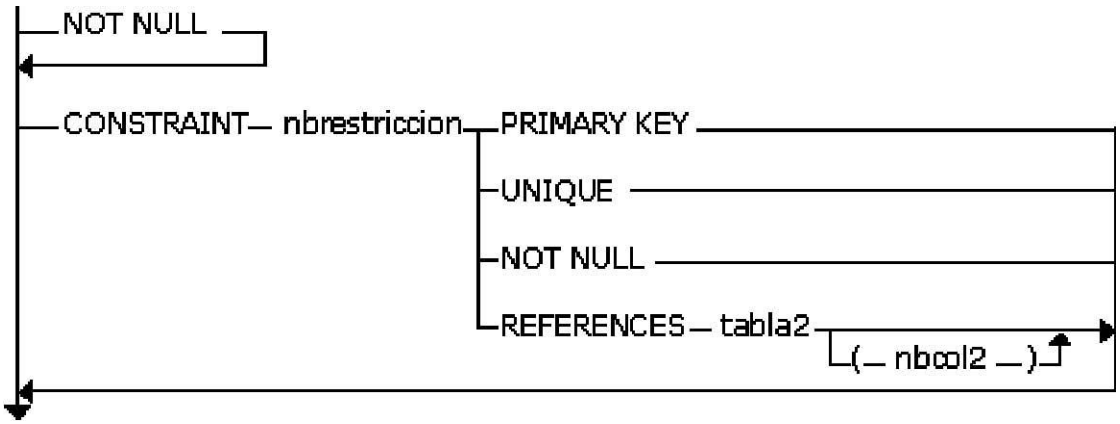
Una restricción consiste en la definición de una característica adicional que tiene una columna o una combinación de columnas, suelen ser características como valores no nulos (campo requerido), definición de índice sin duplicados, definición de clave principal y definición de clave foránea (clave ajena o externa, campo que sirve para relacionar dos tablas entre sí).

restricción1: una restricción de tipo 1 es una restricción que aparece dentro de la definición de la columna después del tipo de dato y afecta a una columna, la que se está definiendo.

restricción2: una restricción de tipo 2 es una restricción que se define después de definir todas las columnas de la tabla y afecta a una columna o a una combinación de columnas.

Para escribir una sentencia CREATE TABLE se empieza por indicar el nombre de la tabla que queremos crear y a continuación entre paréntesis indicamos separadas por comas las definiciones de cada columna de la tabla, la definición de una columna consta de su nombre, el tipo de dato que tiene y podemos añadir si queremos una serie de especificaciones que deberán cumplir los datos almacenados en la columna, después de definir cada una de las columnas que compone la tabla se pueden añadir una serie de restricciones, esas restricciones son las mismas que se pueden indicar para cada columna pero ahora pueden afectar a más de una columna por eso tienen una sintaxis ligeramente diferente.

Una restricción de tipo 1 se utiliza para indicar una característica de la columna que estamos definiendo, tiene la siguiente sintaxis:



La cláusula NOT NULL indica que la columna no podrá contener un valor nulo, es decir que se deberá rellenar obligatoriamente y con un valor válido (equivale a la propiedad requerido Sí de las propiedades del campo).

La cláusula CONSTRAINT sirve para definir una restricción que se podrá eliminar cuando queramos sin tener que borrar la columna. A cada restricción se le asigna un nombre que se utiliza para identificarla y para poder eliminarla cuando se quiera.

Como restricciones tenemos la de clave primaria (clave principal), la de índice único (sin duplicados), la de valor no nulo, y la de clave foránea.

La cláusula PRIMARY KEY se utiliza para definir la columna como clave principal de la tabla. Esto supone que la columna no puede contener valores nulos ni pueden haber valores duplicados en esa columna, es decir que dos filas no pueden tener el mismo valor en esa columna.

En una tabla no pueden existir varias claves principales, por lo que no podemos incluir la cláusula PRIMARY KEY más de una vez, en caso contrario la sentencia da un error. No hay que confundir la definición de varias claves principales con la definición de una clave principal compuesta por varias columnas, esto último sí está permitido y se define con una restricción de tipo 2.

La cláusula UNIQUE sirve para definir un índice único sobre la columna. Un índice único es un índice que no permite valores duplicados, es decir que si una columna tiene definida un restricción de UNIQUE no podrán haber dos filas con el mismo valor en esa columna. Se suele emplear para que el sistema compruebe el mismo que no se añaden valores que ya existen, por ejemplo si en una tabla de clientes queremos asegurarnos que dos clientes no puedan tener el mismo DNI. y la tabla tiene como clave principal un código de cliente, definiremos la columna dni con la restricción de UNIQUE.

La cláusula NOT NULL indica que la columna no puede contener valores nulos, cuando queremos indicar que una columna no puede contener el valor nulo lo podemos hacer sin poner la cláusula CONSTRAINT, o utilizando una cláusula CONSTRAINT.

La última restricción que podemos definir sobre una columna es la de clave foránea, una clave foránea es una columna o conjunto de columnas que contiene un valor que hace referencia a una fila de otra tabla, en una restricción de tipo 1 se puede definir con la cláusula

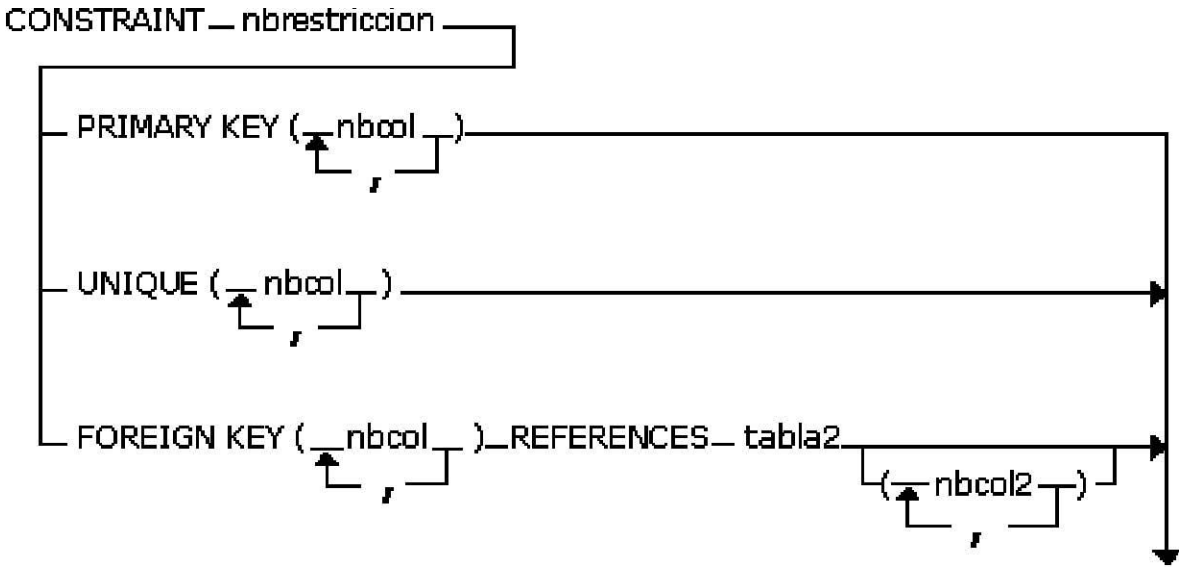
REFERENCES, después de la palabra reservada indicamos a qué tabla hace referencia, opcionalmente podemos indicar entre paréntesis el nombre de la columna donde tiene que buscar el valor de referencia, por defecto toma la clave principal de la tabla2, si el valor que tiene que buscar se encuentra en otra columna de tabla2, entonces debemos inidicar el nombre de esta columna entre paréntesis, además sólo podemos utilizar una columna que esté definida con una restricción de UNIQUE, si la columna2 que indicamos no está definida sin duplicados, la sentencia CREATE nos dará un error.

Ejemplo:

```
CREATE TABLE tab1 ( col1 INTEGER CONSTRAINT pk PRIMARY KEY, col2 CHAR(25)
NOT NULL, col3 CHAR(10) CONSTRAINT uni1 UNIQUE, col4 INTEGER, col5 INT
CONSTRAINT fk5 REFERENCES tab2 );
```

Con este ejemplo estamos creando la tabla tab1 compuesta por: una columna llamada col1 de tipo entero definida como clave principal, una columna col2 que puede almacenar hasta 25 caracteres alfanuméricos y no puede contener valores nulos, una columna col3 de hasta 10 caracteres que no podrá contener valores repetidos, una columna col4 de tipo entero sin ninguna restricción, y una columna col5 de tipo entero clave foránea que hace referencia a valores de la clave principal de la tabla tab2.

Una restricción de tipo 2 se utiliza para definir una característica que afecta a una columna o a una combinación de columnas de la tabla que estamos definiendo, se escribe después de haber definido todas las columnas de la tabla.
Tiene la siguiente sintaxis:



La sintaxis de una restricción de tipo 2 es muy similar a la CONSTRAINT de una restricción 1 la diferencia es que ahora tenemos que indicar sobre qué columnas queremos definir la restricción. Se utilizan obligatoriamente las restricciones de tipo 2 cuando la restricción afecta a un grupo de columnas o cuando queremos definir más de una CONSTRAINT para una columna (sólo se puede definir una restricción1 en cada columna).

La cláusula PRIMARY KEY se utiliza para definir la clave principal de la tabla. Después de las palabras PRIMARY KEY se indica entre paréntesis el nombre de la columna o las columnas que forman la clave principal. Las columnas que forman la clave principal no pueden contener valores nulos ni pueden haber valores duplicados de la combinación de columnas.

En una tabla no pueden haber varias claves principales, por lo que no podemos indicar la cláusula PRIMARY KEY más de una vez, en caso contrario la sentencia da un error.

La cláusula UNIQUE sirve para definir un índice único sobre una columna o sobre una combinación de columnas. Un índice único es un índice que no permite valores duplicados. Si el índice es sobre varias columnas no se puede repetir la misma combinación de valores en dos o más filas. Se suele emplear para que el sistema compruebe el mismo que no se añaden valores que ya existen.

La cláusula FOREIGN KEY sirve para definir una clave foránea sobre una columna o una combinación de columnas. Una clave foránea es una columna o conjunto de columnas que contiene un valor que hace referencia a una fila de otra tabla, en una restricción 1 se puede definir con la cláusula REFERENCES. Para definir una clave foránea en una restricción de tipo 2 debemos empezar por las palabras FOREIGN KEY después indicamos entre paréntesis la/s columna/s que es clave foránea, a continuación la palabra reservada REFERENCES seguida del nombre de la tabla a la que hace referencia, opcionalmente podemos indicar entre paréntesis el nombre de la/s columna/s donde tiene que buscar el valor de referencia, por defecto toma la clave principal de la tabla2, si el valor que tiene que buscar se encuentra en otra/s columna/s de tabla2, entonces debemos escribir el nombre de esta/s columna/s entre paréntesis, además sólo podemos utilizar una columna (o combinación de columnas) que esté definida con una restricción de UNIQUE, de lo contrario la sentencia CREATE TABLE nos dará un error.

Ejemplo:

```
CREATE TABLE tab1 (col1 INTEGER, col2 CHAR(25) NOT NULL, col3 CHAR(10), col4
INTEGER, col5 INT, CONSTRAINT pk PRIMARY KEY (col1), CONSTRAINT uni1 UNIQUE
(col3), CONSTRAINT fk5 FOREIGN KEY (col5) REFERENCES tab2 );
```

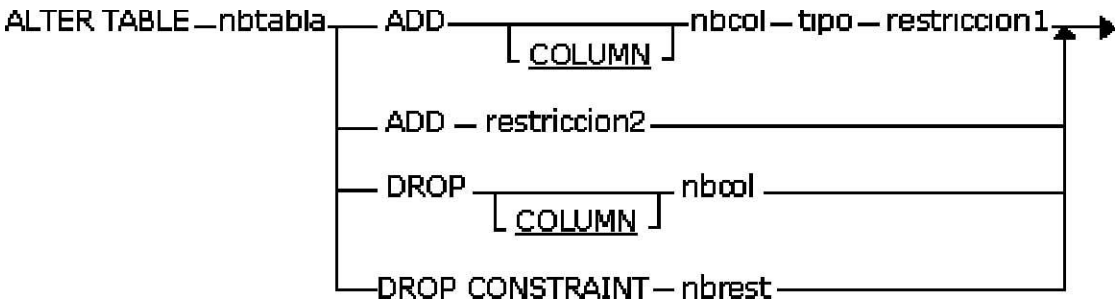
Con este ejemplo estamos creando la misma tabla tab1 del ejemplo de la página anterior pero ahora hemos definido las restricciones utilizando restricciones de tipo 2.

ALTER TABLE (Modificación de la Estructura de una Tabla)

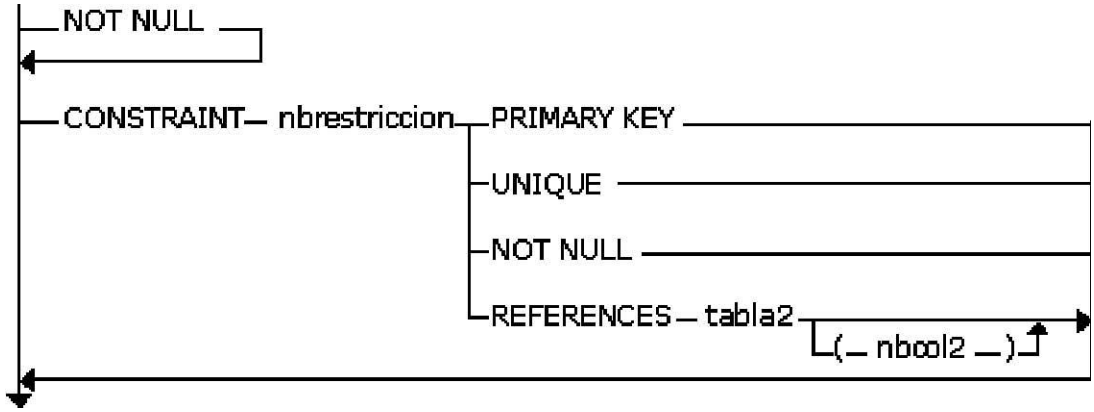
La sentencia ALTER TABLE sirve para modificar la estructura de una tabla que ya existe. Mediante esta instrucción podemos añadir columnas nuevas, eliminar columnas. Es importante considerar que cuando eliminamos una columna se pierden todos los datos almacenados en ella.

También nos permite crear nuevas restricciones o borrar algunas existentes. La sintaxis puede parecer algo complicada pero sabiendo el significado de las palabras reservadas la sentencia se aclara bastante; ADD (añade), ALTER (modifica), DROP (elimina), COLUMN (columna), CONSTRAINT (restricción).

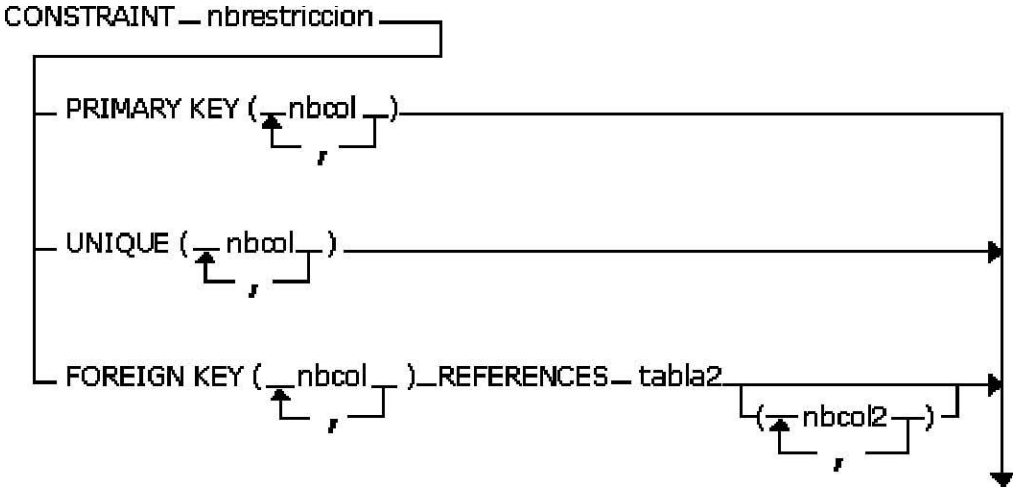
La sintaxis es la siguiente:



La sintaxis de restriccion1 es idéntica a la restricción1 de la sentencia CREATE TABLE, se describe a continuación.



La sintaxis de restriccion2 es idéntica a la restricción2 de la sentencia CREATE TABLE, se describe a continuación.



La cláusula ADD COLUMN (la palabra COLUMN es opcional) permite añadir una columna nueva a la tabla. Como en la creación de tabla, hay que definir la columna indicando su nombre, tipo de datos que puede contener, y si lo queremos alguna restricción de valor no nulo, clave primaria, clave foránea, e índice único, restriccion1 es opcional e indica una restricción de tipo 1 que afecta a la columna que estamos definiendo.

Ejemplo:

ALTER TABLE tab1 ADD COLUMN col3 integer NOT NULL CONSTRAINT c1 UNIQUE

Con este ejemplo estamos añadiendo a la tabla tab1 una columna llamada col3 de tipo entero, requerida (no admite nulos) y con un índice sin duplicados llamado c1.

Cuando añadimos una columna lo mínimo que se puede poner sería:

ALTER TABLE tab1 ADD col3 integer

En este caso la nueva columna admite valores nulos y duplicados.

Para añadir una nueva restricción en la tabla podemos utilizar la cláusula ADD restriccion2 (ADD CONSTRAINT...).

Ejemplo:

ALTER TABLE tab1 ADD CONSTRAINT c1 UNIQUE (col3)

Con este ejemplo estamos añadiendo a la tabla tab1 un índice único (sin duplicados) llamado c1 sobre la columna col3.

Para borrar una columna basta con utilizar la cláusula DROP COLUMN (COLUMN es opcional) y el nombre de la columna que queremos borrar, se perderán todos los datos almacenados en la columna.

Ejemplo:

ALTER TABLE tab1 DROP COLUMN col3

También podemos escribir:

ALTER TABLE tab1 DROP col3

El resultado es el mismo, la columna col3 desaparece de la tabla tab1.

Para borrar una restricción basta con utilizar la cláusula DROP CONSTRAINT y el nombre de la restricción que queremos borrar, en este caso sólo se elimina la definición de la restricción pero los datos almacenados no se modifican ni se pierden.

Ejemplo:

ALTER TABLE tab1 DROP CONSTRAINT c1

Con esta sentencia borramos el índice c1 creado anteriormente pero los datos de la columna col3 no se ven afectados por el cambio.

Por otra parte, se puede modificar los parámetros definidos para las columnas de la tabla a través de la instrucción ALTER TABLE junto con la cláusula ALTER COLUMN.

ALTER TABLE nombreTabla ALTER COLUMN nombreCampo tipoDato [(tamaño)]

Si solamente desea establecer o modificar el valor por defecto, puede utilizar, la siguiente sintaxis:

ALTER TABLE nombreTabla ALTER COLUMN nombreCampo SET DEFAULT valorDefecto

No utilice la palabra reservada SET cuando modifique a la misma vez el tipo de dato y el valor por defecto del campo, ya que obtendrá un error de sintaxis.
La palabra SET únicamente se utiliza cuando sólo se desea modificar o establecer el valor por defecto de un campo, en cuyo caso es completamente necesario incluirla, tal y como se muestra en el siguiente ejemplo:

ALTER TABLE Alumnos ALTER COLUMN Localidad SET DEFAULT ""Jaén""

El valor por defecto debe de especificarse encerrando el valor entre pares de comillas dobles, si el mismo es un valor alfanumérico, y deberá de corresponderse con el tipo de valor declarado para el campo. En todo caso, si el valor por defecto incluye espacios en blancos, necesariamente tendrá que encerrar el valor entre pares de comillas dobles, dado que no sirve en este caso encerrar las palabras entre corchetes [ ].
Por último, si desea eliminar el valor por defecto de un campo, no incluya ninguna palabra o número tras la palabra reservada DEFAULT:

ALTER TABLE Alumnos ALTER COLUMN Localidad SET DEFAULT

A continuación modificaremos el tipo de dato de un campo, de tal forma que su valor incial de texto pasará ahora a ser del tipo numérico entero largo:

ALTER TABLE Alumnos ALTER COLUMN CodPostal INTEGER DEFAULT

Si los valores existentes en el campo no se pueden convertir al nuevo tipo de dato definido, recibirá un mensaje de error indicando que “no coinciden los tipos de datos en la expresión de criterios”.

En el ejemplo anterior, si algún registro de la tabla tiene un valor alfanumérico en el campo cuyo tipo de dato queremos modificar, no podremos llevar a cabo dicha operación. Sin embargo, existen casos en donde se permite el cambio del tipo de dato, pero los valores alfanuméricos serán reemplazados por valor NULL.

DROP TABLE (Eliminar una Tabla)

La sentencia DROP TABLE sirve para eliminar una tabla. No se puede eliminar una tabla si está abierta, tampoco la podemos eliminar si el borrado infringe las reglas de integridad referencial (si interviene como tabla padre en una relación y tiene registros relacionados).

La sintaxis es la siguiente:

DROP TABLE – nbtabla

Ejemplo:

DROP TABLE tab1

Elimina de la base de datos la tabla tab1.

CREATE DATABASE (Creación de una Base de Datos)

Esta es una sentencia SQL que varia considerablemente entre los diferentes motores, a continuación se verán la sentencia correspondiente al motor SQL 2000.

Si bien esta sentencia es suficiente para crear una base de datos ya que el mismo motor se encarga de poner los atributos de la base por default, es conveniente añadirle los atributos según nuestra conveniencia. Para ello se utiliza la siguiente sentencia:

```
CREATE DATABASE SampleDatabase
ON
( NAME = MyDatabase,
FILENAME = ‘C:\program files\MyData\mydatabasefile.mdf’,
SIZE = 10MB,
MAXSIZE = UNLIMITED,
FILEGROWTH = 1MB)
LOG ON
( NAME = MyDatabase_LOG,
FILENAME = ‘C:\program files\MyData\mydatabaselog.ldf’,
SIZE = 5MB,
MAXSIZE = 25MB,
FILEGROWTH = 5MB )
GO
```

Donde NAME es el nombre lógico que se le da a la base de datos, FILENAME es el nombre físico y la ubicación del archivo, SIZE es el tamaño que ser reserva para ese archivo, MAXSIZE es el máximo que podrá ocupar el archivo y FILEGROWTH es la forma en que irá creciendo el mismo. Estas características del archivo se definen tanto para los archivos de datos como para los archivos de log.

Cabe mencionar que por defecto el SQL toma los valores descriptos, con lo cual se puede crear una base de datos directamente con la siguiente sentencia:

CREATE DATABASE NombredelaBD

EJERCICIOS CLASE 1

1. Crear una base de datos de nombre Jurassic_Park sin especificar los valores de la base.

2. Crear la tabla Escuela y definir su clave principal en la misma instrucción de creación. Continuar con tablas Guia, Reserva y Tipo_Visita.

3. Crear la tabla Telefono_Escuela con su clave principal. (hacer restricción en caso de ser una CP compuesta).

4. Crear la tabla Reserva_Por_Grado con su clave principal. Hacer las correspondientes restricciones.

5. Crear la tabla Reserva_Tipo_Visita con sus campos propios y los referenciados. Sin generar claves.

6. Completar el ejercicio anterior, con la creación de las claves correspondientes.

7. Añadir a la tabla de Guía la columna sueldo_hora.

   Nota: En SQL Server no es necesario agregar COLUMN.

8. Hacer que no puedan haber dos escuelas con el mismo nombre.

CLASE 1 - Adicionales

1. Crear la tabla Distrito_Escolar con su correspondiente CP.

2. Agregar clave foránea codigo_distrito_escolar a la tabla Escuela

3. Eliminar la columna de domicilios de la tabla Escuela.

4. Agregar columnas calle_escuela y altura_escuela a la tabla Escuela.

5. Agregar domicilio_guia en tabla Guia.

6. Eliminar tabla Telefono_Escuela

7. Agregar tabla Email_Escuela (sin Clave primaria)

8. Establecer clave primaria para Email_Escuela

9. Establecer que los nombres y apellidos de los guias no tengan valores nulos. nulos.

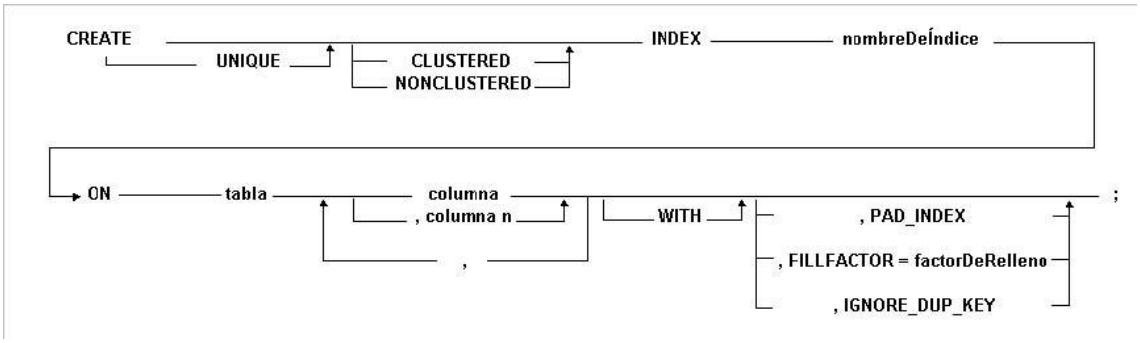
10. Establecer que no se repita la calle y la altura de las escuelas.

CLASE 2: CREACION DE INDICES y ASIGNACION DE PERMISOS.

INDICES

CREATE INDEX (Creación de índices)
Sintaxis (SQL 2000)

La instrucción CREATE INDEX se utiliza para crear índices sobre una o más columnas de una tabla.



Si no se especifica UNIQUE en la instrucción CREATE INDEX, el índice admitirá duplicados. Si no se especifica CLUSTERED en la instrucción CREATE INDEX, se creará un índice no agrupado.

Ejemplo
CREATE CLUSTERED INDEX Ind_apellido ON miembro (apellido)

En este ejemplo se crea un índice agrupado sobre la columna apellidos de la tabla miembro.

Un índice único asegura que ninguno de los datos de una columna indizada contendrá valores duplicados. Si la tabla tiene una restricción PRIMARY KEY o UNIQUE.

La opción UNIQUE crea un índice único (es decir, que no permite que dos filas tengan el mismo valor de índice) en una tabla o vista.

SQL Server comprueba si hay valores duplicados cuando se crea el índice (si ya existen datos) y realiza la comprobación cada vez que se agregan datos con una instrucción INSERT o UPDATE. Si existen valores de clave duplicados, se cancela la instrucción CREATE INDEX y se devuelve un mensaje de error con el primer duplicado. Varios valores NULL se consideran como duplicados al crear un índice UNIQUE.

La opción FILLFACTOR permite optimizar el rendimiento de las instrucciones INSERT y UPDATE en las tablas que contienen índices agrupados o no agrupados.

Cuando se llena una página de índice, el servidor de base de datos necesita un tiempo para dividirla y hacer sitio para las nuevas filas. La opción FILLFACTOR especifica, en forma de porcentaje, cuánto se llenarán las páginas del nivel de hojas. Al dejar espacio libre disminuye la división de páginas ya que las páginas no están llenas al principio, pero aumenta el tamaño del índice. Las páginas del nivel de hojas son las páginas de datos de un índice agrupado. Para los índices agrupados, el FILLFACTOR afecta al nivel en el que se llenan las páginas de datos. A su vez, el nivel afecta al número de filas que se almacenan en una página, lo que determina el tamaño de la tabla.

El valor especificado para el FILLFACTOR tiene los efectos siguientes:
Un FILLFACTOR bajo aumenta el rendimiento de UPDATE e INSERT debido a la menor división de páginas. Un FILLFACTOR bajo es adecuado para los entornos de proceso de transacciones en línea (OLTP, Online Transaction Processing).
Un FILLFACTOR alto aumenta el rendimiento de las consultas o las lecturas porque se leen filas de menos páginas. Un FILLFACTOR alto es adecuado para entornos de servicios de apoyo a la toma de decisiones (DSS, Decision Support Services).

La opción CLUSTERED crea un objeto en el que el orden físico de las filas es el mismo que el orden indizado de las filas y el nivel inferior (hojas) del índice agrupado contiene las filas de datos reales. Una tabla o vista permite un índice agrupado al mismo tiempo.

Una vista con un índice agrupado se denomina vista indizada. Es necesario crear un índice agrupado único en una vista antes de poder definir otros índices en la misma vista.

La opción NONCLUSTERED crea un objeto que especifica la ordenación lógica de una tabla. Con un índice no agrupado, el orden físico de las filas es independiente del orden indizado. El nivel hoja de un índice no agrupado contiene las filas del índice. Cada fila del índice contiene el valor de clave no agrupada, y uno o varios localizadores de fila que apuntan a la fila que contiene dicho valor. Si la tabla no tiene un índice agrupado, el localizador de fila es la dirección de disco de la fila. Si la tabla tiene un índice agrupado, el localizador de fila es la clave del índice agrupado de la fila.
Cada tabla puede tener hasta 249 índices no agrupados (sin importar cómo se hayan creado: implícitamente con las restricciones PRIMARY KEY y UNIQUE, o explícitamente con CREATE INDEX). Cada índice puede proporcionar acceso a los datos en un orden distinto.

Para las vistas indizadas, sólo se pueden crear índices no agrupados en una vista que ya tenga definido un índice agrupado. Por lo tanto, el localizador de fila de un índice no agrupado de una vista indizada siempre es la clave agrupada de la fila.

Crear índices compuestos

Un índice creado sobre más de una columna de una tabla se denomina índice compuesto. Puede crear índices compuestos:
Cuando la búsqueda en dos o más columnas es más fácil como una clave.
Si las consultas sólo hacen referencia a las columnas del índice. Esto se denomina consulta cubierta. Cuando una consulta está cubierta, no es necesario recuperar las filas de datos, ya que los valores de columna se leen directamente de las claves del índice.
Por ejemplo, una guía telefónica es un buen ejemplo de la utilidad de un índice compuesto. La guía está organizada por apellidos. Dentro de los apellidos, está organizada por nombre, ya que normalmente existen entradas con los mismos apellidos. Aunque las entradas están indexadas por apellidos y nombre, no se puede buscar una entrada de la guía telefónica sólo

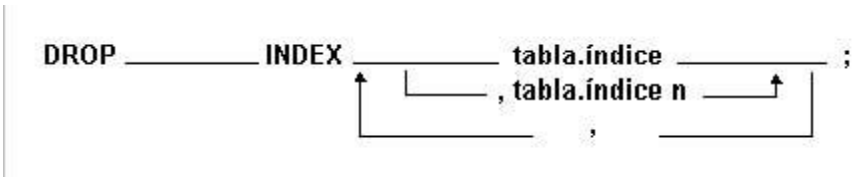
por el nombre.

Ejemplo
CREATE INDEX ind_Ape_Nom ON miembro (apellido, nombre)

Las columnas apellido y nombre son los valores de clave compuestos. Observe que la columna apellido aparece en primer lugar porque es más selectiva que nombre.

DROP INDEX (Eliminación de un índice)
Sintaxis (SQL 2000)

La instrucción DROP INDEX se utiliza para eliminar un índice de una tabla.



Ejemplo
 DROP INDEX miembro.cl_apellido

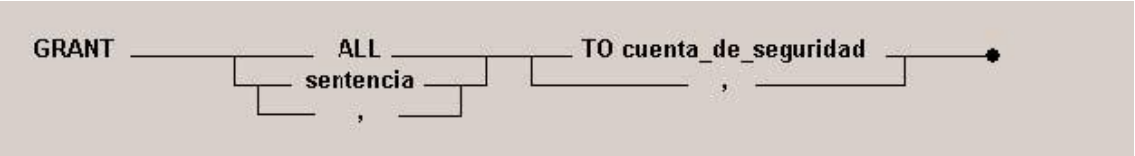
En este ejemplo se elimina el índice cl_lastname de la tabla de miembro.

ASINGACION DE PERMISOS

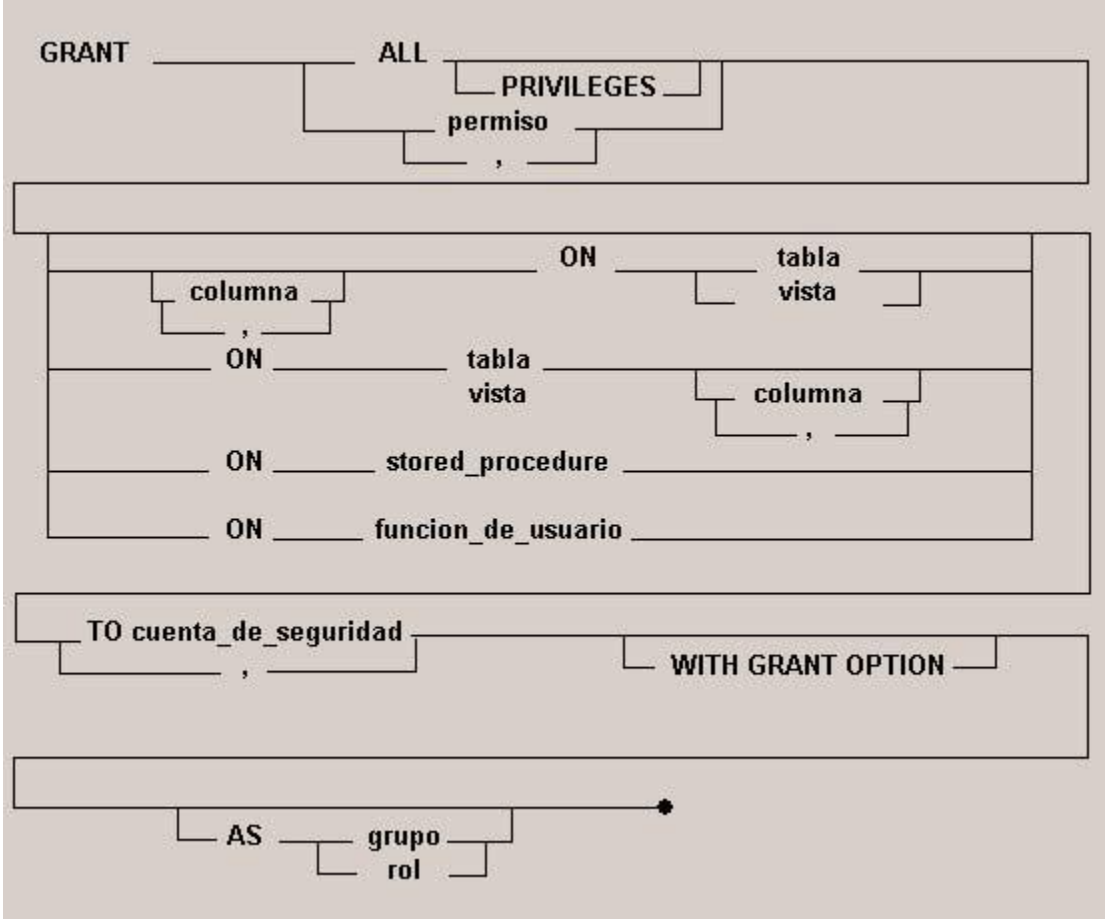
GRANT (Asignar permisos)

Crea una entrada en el sistema de seguridad que permite a un usuario de la base de datos actual trabajar con datos de la base de datos.

Sintaxis
Permisos de la instrucción:



Permisos del objeto:



Argumentos

ALL
Especifica que se conceden todos los permisos aplicables.

sentencia
Es la instrucción para la que se concede el permiso. La lista de instrucciones puede contener:
CREATE DATABASE
CREATE DEFAULT
CREATE FUNCTION
CREATE PROCEDURE
CREATE RULE
CREATE TABLE
CREATE VIEW
BACKUP DATABASE

BACKUP LOG

n
Marcador de posición que indica que el elemento se puede repetir en una lista separada por comas.

TO
Especifica la lista de cuentas de seguridad.

cuenta_de_seguridad
Es la cuenta de seguridad de la base de datos actual cuyos permisos se quitan. La cuenta de seguridad puede ser de:
Un Usuario
Una Función.
Un Usuario del SO.
UnGrupo del SO.

PRIVILEGES
Es una palabra clave opcional que se puede incluir para cumplir con SQL-92.

permiso
Se trata de un permiso de objeto que se concede. Cuando se conceden permisos sobre una tabla, una función de valores de tabla o una vista, la lista de permisos puede incluir uno o más de los siguientes permisos: SELECT, INSERT, DELETE, REFERENCES o UPDATE. Es posible suministrar una lista de columnas junto con los permisos SELECT y UPDATE. Si no se suministra una lista de columnas con los permisos SELECT y UPDATE, los permisos se aplican a todas las columnas de la tabla, vista o función de valores de tabla.
Los permisos que se conceden a objetos en un procedimiento almacenado sólo pueden incluir EXECUTE. Los permisos que se conceden a objetos en una función de valores escalares pueden incluir EXECUTE y REFERENCES.
Para tener acceso a una columna en una instrucción SELECT es necesario disponer de permiso para utilizar SELECT en esa columna. Para actualizar una columna mediante una instrucción UPDATE es necesario disponer de permiso para utilizar UPDATE en esa columna.
Para crear una restricción FOREIGN KEY que haga referencia a una tabla es necesario disponer de permiso para utilizar REFERENCES en esa tabla.
Para crear una FUNCTION o VIEW con la cláusula WITH SCHEMABINDING que haga referencia a un objeto es necesario disponer de permiso para utilizar REFERENCES en ese objeto.

columna
Es el nombre de la columna de la base de datos actual sobre la que se conceden los permisos.

tabla
Es el nombre de la tabla de la base de datos actual sobre la que se conceden los permisos.

vista
Es el nombre de la vista de la base de datos actual sobre la que se conceden los permisos.

stored_procedure
Es el nombre del procedimiento almacenado de la base de datos actual sobre el que se conceden los permisos.

funcion_de_usuario
Es el nombre de la función definida por el usuario sobre la que se conceden los permisos.

WITH GRANT OPTION
Especifica que se concede a cuenta_de_seguridad la capacidad de conceder el permiso de objeto especificado a otras cuentas de seguridad. La cláusula WITH GRANT OPTION sólo es válida con los permisos de objeto.

AS {grupo | rol}
Especifica el nombre opcional de la cuenta de seguridad de la base de datos actual que tiene los permisos necesarios para ejecutar la instrucción GRANT. AS se utiliza cuando se conceden permisos sobre un objeto a un grupo o función, y es necesario que los permisos de objetos se concedan además a otros usuarios que no son miembros del grupo o función. Debido a que sólo un usuario, y no un grupo o función, puede ejecutar una instrucción GRANT, un miembro específico del grupo o función concederá los permisos del objeto bajo la autoridad del grupo o función.

Ejemplos

A. Conceder permisos sobre instrucciones

En este ejemplo se conceden diversos permisos sobre instrucciones a los usuarios Maria y Jose, y al grupo de Windows NT Corporate\GrupoJ.

```
GRANT CREATE DATABASE, CREATE TABLE
TO Maria, Jose, [Corporate\ GrupoJ]
```

B. Conceder permisos de objeto dentro de la jerarquía de permisos

En este ejemplo se muestra el orden preferente de los permisos. En primer lugar, se conceden permisos SELECT a la función public. A continuación se conceden permisos específicos a los usuarios Maria, Jose y Tomas. De este modo, todos estos usuarios tienen permisos sobre la tabla autores.

```
USE pubs
GO

GRANT SELECT
ON autores
TO public
GO

GRANT INSERT, UPDATE, DELETE
ON autores
TO Maria, Jose, Tomas
GO
```

C. Conceder permisos a una función de SQL Server

En este ejemplo se conceden permisos CREATE TABLE a todos los miembros de la función

Contar.

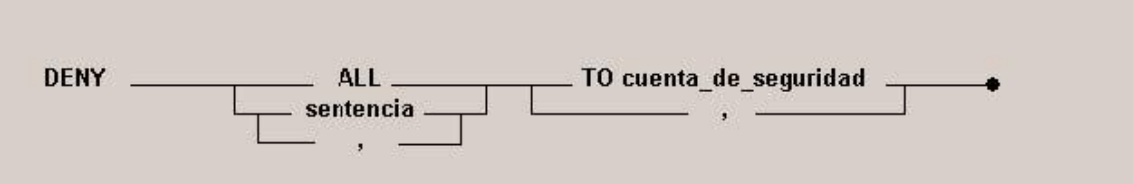
GRANT CREATE TABLE TO Contar

DENY (Denegar permisos)

Crea una entrada en el sistema de seguridad que deniega un permiso de una cuenta de seguridad en la base de datos.

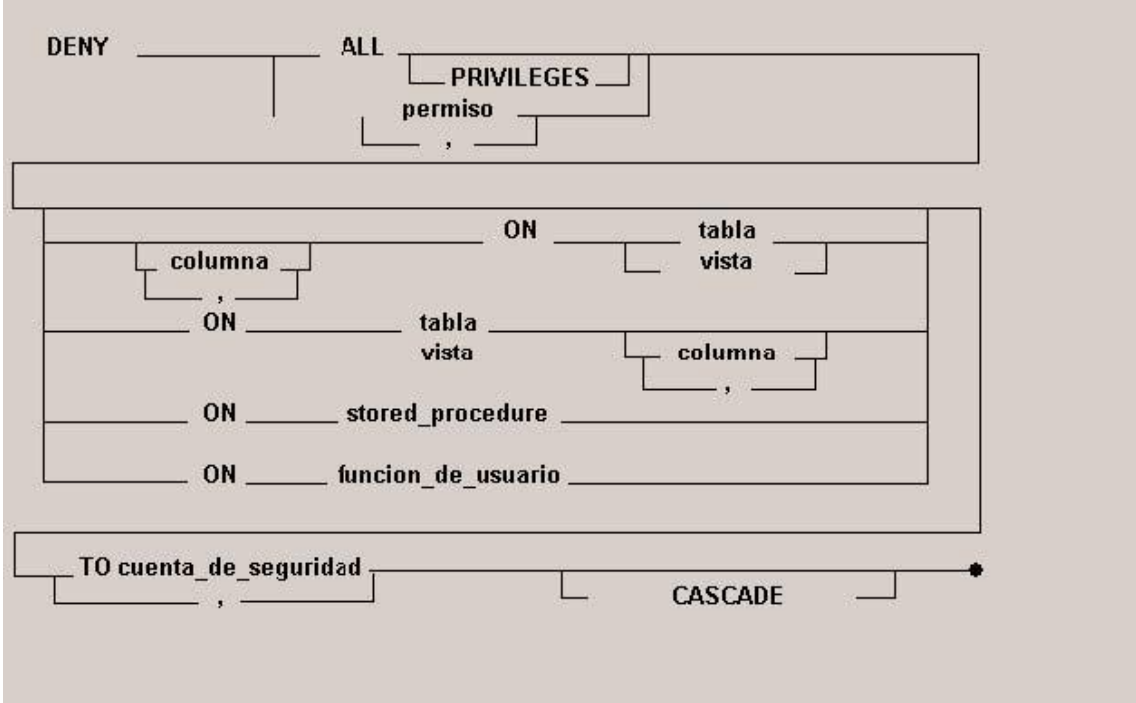
Sintaxis

Permisos de la instrucción:



Permisos del objeto:

Argumentos



ALL
Especifica que se deniegan todos los permisos aplicables.

sentencia
Es la instrucción para la que se deniega el permiso. La lista de instrucciones puede contener:
CREATE DATABASE

CREATE DEFAULT
CREATE FUNCTION
CREATE PROCEDURE
CREATE RULE
CREATE TABLE
CREATE VIEW
BACKUP DATABASE
BACKUP LOG

n
Es un marcador de posición que indica que el elemento se puede repetir en una lista separada por comas.

TO
Especifica la lista de cuentas de seguridad.

cuenta_de_seguridad
Es la cuenta de seguridad de la base de datos actual cuyos permisos se quitan. La cuenta de seguridad puede ser de:
Un Usuario
Una Función.
Un Usuario del SO.
UnGrupo del SO.

PRIVILEGES
Es una palabra clave opcional que se puede incluir para cumplir con SQL-92.

permiso
Se trata de un permiso de objeto denegado. Cuando se deniegan permisos sobre una tabla o una vista, la lista de permisos puede incluir una o más de las siguientes instrucciones: SELECT, INSERT, DELETE o UPDATE.
Los permisos de objeto que se deniegan sobre una tabla también pueden incluir REFERENCES y los permisos de objeto que se deniegan sobre un procedimiento almacenado o procedimiento almacenado extendido pueden incluir EXECUTE. Cuando se deniegan permisos sobre columnas, la lista de permisos puede incluir SELECT o UPDATE.

columna
Es el nombre de la columna de la base de datos actual para la que se deniegan los permisos.

tabla
Es el nombre de la tabla de la base de datos actual para la que se deniegan los permisos.

vista
Es el nombre de la vista de la base de datos actual para la que se deniegan los permisos.

stored_procedure
Es el nombre del procedimiento almacenado de la base de datos actual para el que se deniegan los permisos.

funcion_de_usuario
Es el nombre de la función definida por el usuario sobre la que se deniegan los permisos.

CASCADE
Especifica que se deniegan los permisos de cuenta_de_seguridad y los permisos concedidos a otras cuentas de seguridad por cuenta_de_seguridad. Utilice CASCADE cuando deniegue un permiso que se puede conceder. Si no se especifica CASCADE y se ha concedido al usuario especificado el permiso WITH GRANT OPTION, se devuelve un error.

Observaciones

Si se utiliza la instrucción DENY para impedir que un usuario obtenga un permiso y, posteriormente, se agrega el usuario a un grupo o función que tenga el permiso concedido, el usuario no obtiene acceso al permiso.
Utilice la instrucción REVOKE para quitar un permiso denegado de una cuenta de usuario. La cuenta de seguridad no obtiene acceso al permiso a menos que se haya concedido el permiso a un grupo o función del que el usuario es miembro. Utilice la instrucción GRANT para quitar un permiso denegado y para aplicar explícitamente el permiso a la cuenta de seguridad.

Ejemplos

A. Denegar permisos para instrucciones

Este ejemplo deniega varios permisos para instrucciones a varios usuarios. Los usuarios no pueden utilizar las instrucciones CREATE DATABASE o CREATE TABLE a menos que se les haya concedido explícitamente el permiso.

```
DENY CREATE DATABASE, CREATE TABLE
TO Maria, Jose, [Corporate\ GrupoJ]
```

B. Denegar permisos de objeto en la jerarquía de permisos

En este ejemplo se muestra el orden preferente de los permisos. En primer lugar, se conceden permisos SELECT a la función public. A continuación, se deniegan permisos específicos a los usuarios Maria, Jose y Tomas. Estos usuarios no tienen permisos sobre la tabla autores.

```
USE pubs
GO

GRANT SELECT
ON autores
TO public
GO

DENY SELECT, INSERT, UPDATE, DELETE
ON autores
TO Maria, Jose, Tomas
```

C. Denegar permisos a una función de SQL Server

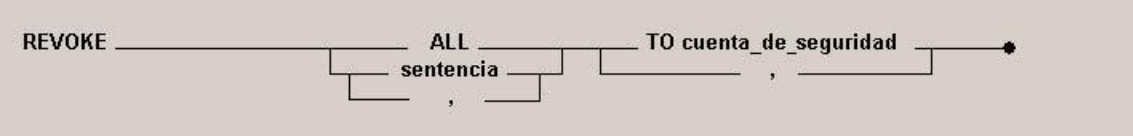
En este ejemplo se deniegan permisos CREATE TABLE a todos los miembros de la función Contar. Aunque se haya concedido explícitamente a los usuarios de Contar el permiso de CREATE TABLE, DENY suplanta ese permiso.

DENY CREATE TABLE TO Contar

REVOKE (Revocar permisos)

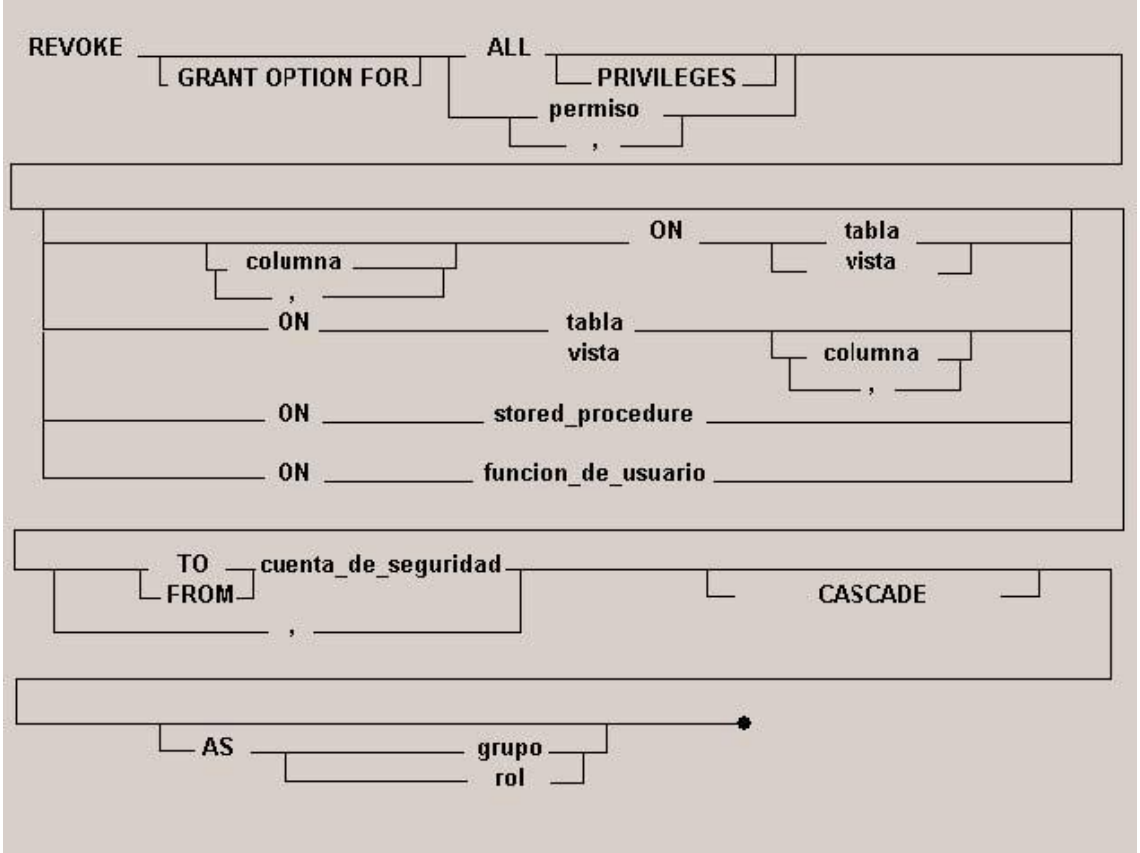
Quita un permiso otorgado o denegado previamente de un usuario de la base datos actual.
Sintaxis

Permisos de la instrucción:



Permisos del objeto:

Argumentos



ALL
Especifica que se quitan todos los permisos aplicables.

sentencia
Es la instrucción para la que se quita el permiso. La lista de instrucciones puede contener:
CREATE DATABASE
CREATE DEFAULT
CREATE FUNCTION
CREATE PROCEDURE
CREATE RULE
CREATE TABLE
CREATE VIEW
BACKUP DATABASE
BACKUP LOG

n
Es un marcador de posición que indica que el elemento se puede repetir en una lista separada por comas.

FROM
Especifica la lista de cuentas de seguridad.

cuenta_de_seguridad
Es la cuenta de seguridad de la base de datos actual cuyos permisos se quitan. La cuenta de seguridad puede ser de:

Un Usuario
Una Función.
Un Usuario del SO.
UnGrupo del SO.

GRANT OPTION FOR
Especifica que se van a quitar los permisos WITH GRANT OPTION. Utilice las palabras clave GRANT OPTION FOR con REVOKE para anular el efecto de la configuración WITH GRANT OPTION especificada en la instrucción GRANT. El usuario todavía tiene los permisos, pero no puede otorgarlos a otros usuarios.
Si los permisos que se van a revocar no se otorgaron originalmente con la opción WITH GRANT OPTION, GRANT OPTION FOR se pasa por alto si se especifica y los permisos se revocan normalmente.
Si los permisos que se van a revocar se otorgaron originalmente con la opción WITH GRANT OPTION, especifique las cláusulas CASCADE y GRANT OPTION FOR; en caso contrario, se devuelve un error.

PRIVILEGES
Es una palabra clave opcional que se puede incluir para cumplir con SQL-92.

permiso
Se trata del permiso de objeto que se revoca. Cuando se revocan permisos en una tabla o una vista, la lista de permisos puede incluir una o más de las siguientes instrucciones: SELECT, INSERT, DELETE o UPDATE.
Los permisos del objeto revocados en una tabla también pueden incluir REFERENCES y los permisos del objeto revocados en un procedimiento almacenado o un procedimiento almacenado extendido pueden ser permisos EXECUTE. Cuando se revocan permisos en columnas, la lista de permisos puede incluir SELECT o UPDATE.

columna
Es el nombre de la columna de la base de datos actual cuyos permisos se quitan.

tabla
Es el nombre de la tabla de la base de datos actual cuyos permisos se quitan.

vista
Es el nombre de la vista de la base de datos actual cuyos permisos se quitan.

stored_procedure
Es el nombre del procedimiento almacenado de la base de datos actual cuyos permisos se quitan.

funcion_de_usuario
Es el nombre de la función definida por el usuario para la que se retiran los permisos.

TO
Especifica la lista de cuentas de seguridad.

CASCADE
Especifica que los permisos se quitan de cuenta_de_seguridad, así como de otras cuentas de seguridad a las que cuenta_de_seguridad otorgó permisos. Utilícelo cuando revoque un

permiso que se puede otorgar.
Si los permisos que se revocan se otorgaron originalmente a cuenta_de_seguridad con la opción WITH GRANT OPTION, especifique las cláusulas CASCADE y GRANT OPTION FOR; en caso contrario, se devuelve un error. Al especificar las cláusulas CASCADE y GRANT OPTION FOR se revocan solamente los permisos otorgados con la opción WITH GRANT OPTION desde cuenta_de_seguridad, así como los de las demás cuentas de seguridad a las que cuenta_de_seguridad otorgó permisos.

AS {grupo | rol}
Especifica el nombre opcional de la cuenta de seguridad de la base de datos actual bajo cuya autoridad se ejecutará la instrucción REVOKE. AS se utiliza cuando los permisos de un objeto se otorgan a un grupo o función, y es necesario revocar los permisos del objeto a otros usuarios. Debido a que sólo un usuario, y no un grupo o función, puede ejecutar una instrucción REVOKE, el miembro específico del grupo o función revoca los permisos del objeto bajo la autoridad del grupo o función.

Observaciones

Utilice REVOKE solamente con los permisos de la base de datos actual.
Un permiso revocado sólo quita los permisos otorgados o denegados en el nivel en el que se revocan (usuario, grupo o función). Por ejemplo, el permiso para ver la tabla autores se otorga explícitamente a la cuenta de usuario Andres, que sólo es miembro de la función empleados. Si a la función empleados se le revoca el permiso de acceso para ver la tabla autores, Andres todavía puede ver la tabla debido a que el permiso se le ha otorgado explícitamente. Andres no podrá ver la tabla autores, sólo en el caso de que también se le revoque el permiso. Si a Andres nunca se le otorgó explícitamente ningún permiso para ver autores, al revocar los permisos a la función empleados, también se impedirá que Andres vea la tabla.

Ejemplos

A. Revocar permisos de instrucción de una cuenta de usuario

En este ejemplo se revocan los permisos CREATE TABLE que se han concedido a los usuarios Jose y Corporate\GrupoJ. Retira los permisos de creación de tablas a Jose y Corporate\GrupoJ. Sin embargo, Jose y Corporate\GrupoJ pueden crear tablas si antes se han concedido permisos CREATE TABLE a alguna función de la que sean miembros.

```
REVOKE CREATE TABLE FROM Jose, [Corporate\GrupoJ]
```

B. Revocar varios permisos de varias cuentas de usuario

En este ejemplo se revocan varios permisos de instrucción de varios usuarios.

```
REVOKE CREATE TABLE, CREATE DEFAULT
FROM Maria, Jose
```

C. Revocar un permiso denegado

La usuaria Maria es miembro de la función Nomina, a la que se han concedido permisos SELECT en la tabla Nomina_Datos. Se ha utilizado la instrucción DENY con Maria para impedir su acceso a la tabla Nomina_Datos a través de los permisos otorgados a la función Nomina.
En este ejemplo se quita el permiso denegado de Maria y, a través de los permisos SELECT aplicados a la función Nomina, se permite que Maria utilice la instrucción SELECT en la tabla.

REVOKE SELECT ON Nomina_Datos TO Maria

EJERCICIOS CLASE 2

1. Crear un índice agrupado para la tabla Reserva sobre el campo codigo_escuela.

2. Eliminar el índice creado en el punto anterior.

3. Crear un índice que optimice la búsqueda y ordenamiento de la tabla reserva_visita, si la consulta que se quisiera optimizar es por Cantidad de Alumnos Reservados.

4. Crear un nuevo índice agrupado sobre la tabla Guia sobre los campos Apellido y Nombre.

   Nota: Puede que no permita borrar la clave primaria.

5. Crear un índice que optimice la búsqueda por Fecha y Hora de la reserva (índice compuesto) teniendo en cuenta que se realizan inserciones con mucha asiduidad sobre esta tabla (FILLFACTOR).

6. Conceder permiso de INSERT y UPDATE sobre la tabla telefono_escuela a los usuarios jPerez, aFernandez.

7. Eliminar los permisos de UPDATE sobre la tabla telefono_escuela para todos los usuarios.

8. Conceder permisos de create table al usuario jPerez

9. Revocar los permisos de select para el usuario public sobre la tabla de Escuela.

CLASE 2 - Adicionales

1. Eliminar el índice sobre nombre y apellido en la tabla guia.

2. Crear un índice que impida ingresar dos guías con igual nombre y apellido.

3. Conceder al grupo CORDOBA permiso de SELECT para la tabla reserva_visita

4. El usuario jPerez (parte del grupo CORDOBA) no debe poder ver el campo Arancel_por_Alumno (pero sí el resto de la tabla). ¿Como se le deniega el permiso?

5. Conceder al grupo ADM todos los permisos de aplicables sobre la Base Jurasik_Park (creacion de tablas, backup, etc)

6. Denegar al usuario aFernandez (miembro del grupo ADM) el permiso para realizar backups sobre la base y sobre el log de transacciones.

CLASE 3: ESTRUCTURAS BÁSICAS DE DML

El DML (Data Manipulation Language), lenguaje de manipulación de datos, nos permite recuperar los datos almacenados en la base de datos y también incluye órdenes para permitir al usuario actualizar la base de datos añadiendo nuevos datos, suprimiendo datos antiguos o modificando datos previamente almacenados

INSERTAR - INSERT INTO

Agrega una fila en una tabla. Se la conoce como una consulta de datos añadidos. Esta consulta puede ser de dos tipos: Insertar un único registro ó Insertar en una tabla los registros contenidos en otra tabla.

PARA INSERTAR UN ÚNICO REGISTRO:

En este caso la sintaxis es la siguiente:

```
INSERT INTO Tabla (campo1, campo2, .., campoN)
VALUES (valor1, valor2, ..., valorN)
```

Esta consulta graba en el campo1 el valor1, en el campo2 y valor2 y así sucesivamente. Hay que prestar especial atención a acotar entre comillas simples (') los valores literales (cadenas de caracteres) y las fechas indicarlas en formato 'aaaammdd' también entre comillas simples y sin separadores.

PARA INSERTAR REGISTROS DE OTRA TABLA:

En este caso la sintaxis es:

```
INSERT INTO Tabla (campo1, campo2, ..., campoN)
SELECT campo1, campo2, ..., campoN
FROM TablaOrigen
```

En este caso se seleccionarán los campos 1,2, ..., n de la tabla origen y se grabarán en los campos 1,2,.., n de la Tabla. La condición SELECT puede incluir la cláusula WHERE para filtrar los registros a copiar. Si Tabla y TablaOrigen poseen la misma estrucutra podemos simplificar la sintaxis a:

```
INSERT INTO Tabla
SELECT TablaOrigen.* FROM TablaOrigen
```

De esta forma los campos de TablaOrigen se grabarán en Tabla, para realizar esta operación es necesario que todos los campos de TablaOrigen estén contenidos con igual nombre en Tabla. Con otras palabras que Tabla posea todos los campos de TablaOrigen (igual nombre e igual tipo).

MODIFICACION - UPDATE

Crea una consulta de actualización que cambia los valores de los campos de una tabla especificada basándose en un criterio específico.

Su sintaxis es:

UPDATE Tabla SET Campo1=Valor1, Campo2=Valor2, ... CampoN=ValorN
WHERE Criterio;

UPDATE es especialmente útil cuando se desea cambiar un gran número de registros. Puede cambiar varios campos a la vez.

El ejemplo siguiente incrementa los valores Cantidad pedidos en un 10 por ciento y los valores Transporte en un 3 por ciento para aquellos que se hayan enviado al Reino Unido:

UPDATE Pedidos SET Pedido = Pedidos * 1.1, Transporte = Transporte * 1.03
WHERE PaisEnvío = 'AR';

UPDATE no genera ningún resultado. Para saber qué registros se van a cambiar, hay que examinar primero el resultado de una consulta de selección que utilice el mismo criterio y después ejecutar la consulta de actualización.

UPDATE Empleados SET Grado = 5 WHERE Grado = 2;

UPDATE Productos SET Precio = Precio * 1.1 WHERE Proveedor = 8 AND Familia = 3;

ATENCIÓN
Si en una consulta de actualización suprimimos la cláusula WHERE todos los registros de la tabla señalada serán actualizados. Por ejemplo:

UPDATE Empleados SET Salario = Salario * 1.1

ELIMINAR - DELETE

Crea una consulta de eliminación que elimina los registros de la tabla especificada en la cláusula FROM que satisfagan la cláusula WHERE. Esta consulta elimina los registros completos, no es posible eliminar el contenido de algún campo en concreto.

Su sintaxis es:

DELETE FROM Tabla WHERE criterio

El siguiente ejemplo elimina las filas de la tabla Empleados de todos aquellos emplados que son vendedores.

DELETE FROM Empleados WHERE Cargo = 'Vendedor';

Si desea eliminar todas las filas de una tabla, eliminar la propia tabla es más eficiente que ejecutar una consulta de borrado.

Se puede utilizar DELETE para eliminar filas de una única tabla o desde varios lados de una relación uno a muchos. Las operaciones de eliminación en cascada en una consulta únicamente eliminan desde varios lados de una relación. Por ejemplo, en la relación entre las tablas Clientes y Pedidos, la tabla Pedidos es la parte de muchos por lo que las operaciones en cascada solo afectaran a la tabla Pedidos. Una consulta de borrado elimina las filas completas, no únicamente los datos en campos específicos.

Si desea eliminar valores en un campo especificado, crear una consulta de actualización que cambie los valores a Null.

CONSULTAS – ESTRUTURA BÁSICA – SELECT

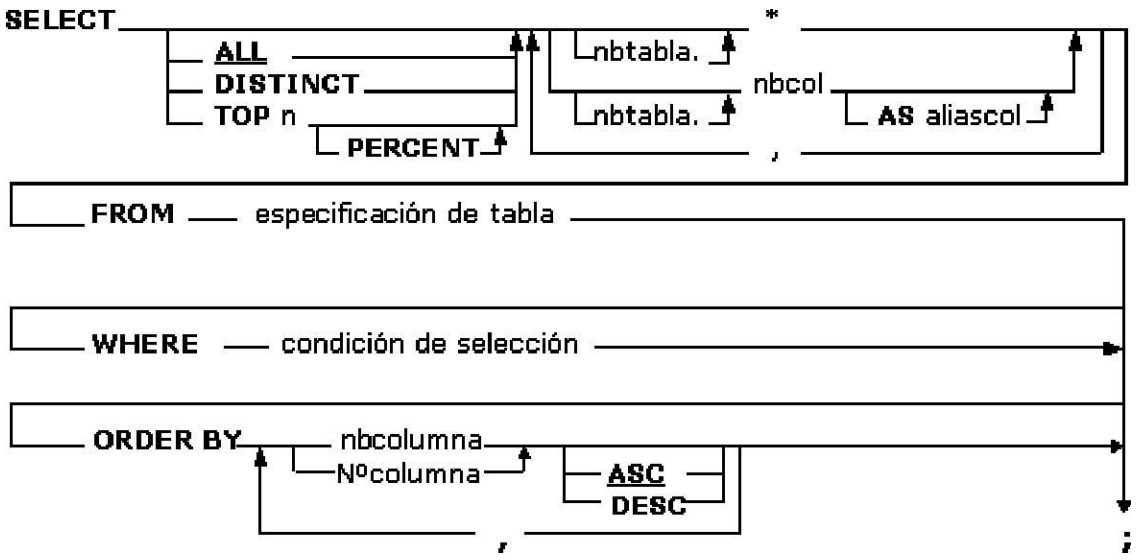
La sentencia SELECT permite recuperar datos de una o varias tablas. La sentencia SELECT es la más compleja y potente de las sentencias SQL.

Esta sentencia permite seleccionar columnas de una tabla, seleccionar filas, obtener las filas ordenadas, etc. El resultado de la consulta es una tabla lógica, porque no se guarda en el disco sino que está en memoria y cada vez que ejecutamos la consulta se vuelve a calcular.

Cuando ejecutamos la consulta se visualiza el resultado en forma de tabla con columnas y filas, pues en la SELECT tenemos que indicar qué columnas queremos que tenga el resultado y qué filas queremos seleccionar de la tabla origen.

Si se desea guarda el resultado de una consulta (en SQL Server) se puede realizar de la siguiente manera:

1 – Ir a Menu Query y seleccionar Results to File
2 – Ejecutar la Consultar
3 – Elegir el lugar a guardar el resultado y escribir el nombre del archivo.

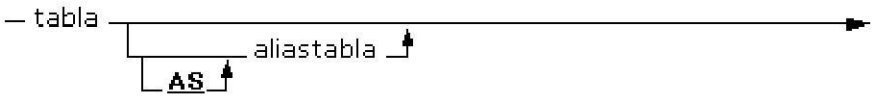


La tabla origen - FROM -

Con la cláusula FROM indicamos en qué tabla tiene que buscar los datos. En esta sección, de consultas simples, el resultado se obtiene de una única tabla. La sintaxis de la cláusula es:

FROM especificación de tabla

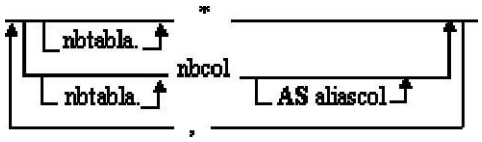
Una especificación de tabla puede ser el nombre de una vista guardada, o el nombre de una tabla que a su vez puede tener el siguiente formato:



Aliastabla es un nombre de alias, es como un segundo nombre que asignamos a la tabla, si en una consulta definimos un alias para la tabla, esta se deberá nombrar utilizando ese nombre y no su nombre real, además ese nombre sólo es válido en la consulta donde se define. El alias se suele emplear en consultas basadas en más de una tabla que veremos en el tema siguiente. La palabra AS que se puede poner delante del nombre de alias es opcional y es el valor por defecto por lo que no tienen ningún efecto.

Selección de columnas

La lista de columnas que queremos que aparezcan en el resultado es lo que llamamos lista de selección y se especifica delante de la cláusula FROM.



Utilización del *

Se utiliza el asterisco * en la lista de selección para indicar 'todas las columnas de la tabla'. Tiene dos ventajas:

a) Evitar nombrar las columnas una a una (es más corto).

b) Si añadimos una columna nueva en la tabla, esta nueva columna saldrá sin tener que modificar la consulta.

Se puede combinar el * con el nombre de una tabla (ej. escuelas.*), pero esto se utiliza más cuando el origen de la consulta son dos tablas.

Ejemplo:

SELECT * FROM escuelas o bien SELECT escuelas.* FROM escuelas

Lista todos los datos de las escuelas

Columnas de la tabla origen

Las columnas se pueden especificar mediante su nombre simple (nbcol) o su nombre cualificado (nbtabla.nbcol, el nombre de la columna precedido del nombre de la tabla que contiene la columna y separados por un punto).

El nombre cualificado se puede emplear siempre que queramos y es obligatorio en algunos casos que veremos más adelante.

Ejemplos :

SELECT CodRes, CodTipoVisita, alumnosReserva, codGuia FROM reservavisita

Lista el Código de la reserva, código del tipo de visita, la cantidad de alumnos reservados por tipo de visita y los códigos de guía de todas las reservas.

SELECT CodEscuela, Nombre, Domicilio FROM escuelas

Lista el código, nombre y domicilio de todas las escuelas

Alias de columna.

Cuando se visualiza el resultado de la consulta, normalmente las columnas toman el nombre que tiene la columna en la tabla, si queremos cambiar ese nombre lo podemos hacer definiendo un alias de columna mediante la cláusula AS será el nombre que aparecerá como título de la columna.

Ejemplo:

SELECT CodTipoVisita AS Codigo_De_Visita, descripcion, arancelalumno FROM TipoVisita

Como título de la primera columna aparecerá Codigo_De_Visita en vez de CodTipoVisita

Columnas calculadas.

Además de las columnas que provienen directamente de la tabla origen, una consulta SQL puede incluir columnas calculadas cuyos valores se calculan a partir de los valores de los datos almacenados.
Para solicitar una columna calculada, se especifica en la lista de selección una expresión en vez de un nombre de columna. La expresión puede contener sumas, restas, multiplicaciones y divisiones, concatenación & , paréntesis y también funciones predefinidas).

Ejemplos:

SELECT ciudad, región, (ventas-objetivo) AS superavit FROM oficinas

Lista la ciudad, región y el superávit de cada oficina.

SELECT idfab, idproducto, descripcion, (existencias * precio) AS valoracion FROM productos

De cada producto obtiene su fabricante, idproducto, su descripción y el valor del inventario.

SELECT nombre, MONTH(contrato), YEAR(contrato) FROM repventas

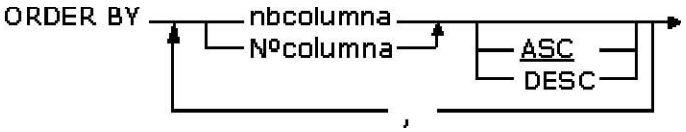
Lista el nombre, mes y año del contrato de cada vendedor. La función MONTH() devuelve el mes de una fecha. La función YEAR() devuelve el año de una fecha

SELECT oficina, 'tiene ventas de ', ventas FROM oficinas

Listar las ventas en cada oficina con el formato: X tiene ventas de 999.

CONSULTAS SIMPLES

Ordenación de las filas - ORDER BY -



Para ordenar las filas del resultado de la consulta, tenemos la cláusula ORDER BY. Con esta cláusula se altera el orden de visualización de las filas de la tabla pero en ningún caso se modifica el orden de las filas dentro de la tabla. La tabla no se modifica. Podemos indicar la columna por la que queremos ordenar utilizando su nombre de columna (nbcolumna) o utilizando su número de orden que ocupa en la lista de selección (Nºcolumna).

Ejemplo:

```
SELECT nombre, oficina, contrato
FROM empleados
ORDER BY oficina
```

es equivalente a

```
SELECT nombre, oficina, contrato
FROM empleados
ORDER BY 2
```

Por defecto el orden será ascendente (ASC) (de menor a mayor si el campo es numérico, por orden alfabético si el campo es de tipo texto, de anterior a posterior si el campo es de tipo fecha/hora, etc...

Ejemplos:

```
SELECT nombre, numemp, oficinarep FROM empleados ORDER BY nombre
```

Obtiene un listado alfabético de los empleados.

```
SELECT nombre, numemp, contrato FROM empleados ORDER BY contrato
```

Obtiene un listado de los empleados por orden de antigüedad en la empresa (los de más antigüedad aparecen primero).

```
SELECT nombre, numemp,ventas FROM empleados ORDER BY ventas
```

Obtiene un listado de los empleados ordenados por volumen de ventas sacando los de menores ventas primero.

Si queremos podemos alterar ese orden utilizando la cláusula DESC (DESCendente), en este caso el orden será el inverso al ASC.

Ejemplo:

```
SELECT nombre, numemp, contrato FROM empleados ORDER BY contrato des
```

Obtiene un listado de los empleados por orden de antigüedad en la empresa empezando por los más recientemente incorporados.

También podemos ordenar por varias columnas, en este caso se indican las columnas separadas por comas. Se ordenan las filas por la primera columna de ordenación, para un mismo valor de la primera columna, se ordenan por la segunda columna, y así sucesivamente. La cláusula DESC o ASC se puede indicar para cada columna y así utilizar una ordenación distinta para cada columna. Por ejemplo ascendente por la primera columna y dentro de la primera columna, descendente por la segunda columna.

Ejemplos:

SELECT region, ciudad, ventas FROM oficinas ORDER BY region, ciudad

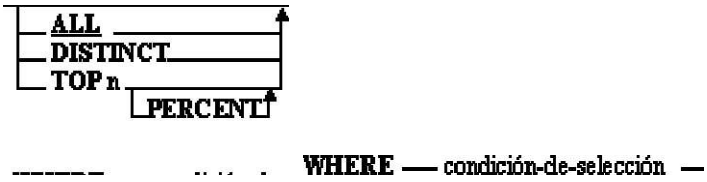
Muestra las ventas de cada oficina, ordenadas por orden alfabético de región y dentro de cada región por ciudad.

SELECT region, ciudad, (ventas - objetivo) AS superavit FROM oficinas ORDER BY region, 3 DESC

Lista las oficinas clasificadas por región y dentro de cada región por superávit de modo que las de mayor superávit aparezcan las primeras.

SELECCIÓN DE FILAS

A continuación veremos las cláusulas que nos permiten indicar qué filas queremos visualizar.



Las cláusulas DISTINCT / ALL

Al incluir la cláusula DISTINCT en la SELECT, se eliminan del resultado las repeticiones de filas. Si por el contrario queremos que aparezcan todas las filas incluidas las duplicadas, podemos incluir la cláusula ALL o nada, ya que ALL es el valor que SQL asume por defecto.

Por ejemplo queremos saber los códigos de los directores de oficina.

SELECT dir FROM oficinas ó SELECT ALL dir FROM oficinas

Lista los códigos de los directores de las oficinas. Si el director 108 aparece en cuatro oficinas, aparecerá cuatro veces en el resultado de la consulta.

SELECT DISTINCT dir FROM oficinas

En este caso el valor 108 aparecerá una sola vez ya que le decimos que liste los distintos valores de directores.

La cláusula TOP

La cláusula TOP permite sacar las n primeras filas de la tabla origen. No elige entre valores

iguales, si pido los 25 primeros valores pero el que hace 26 es el mismo valor que el 25, entonces devolverá 26 registros en vez de 25. Siempre se guía por la columna de ordenación, la que aparece en la cláusula ORDER BY.

Por ejemplo queremos saber los dos empleados más antiguos de la empresa.

SELECT TOP 2 numemp, nombre FROM empleado ORDER BY contrato

Lista el código y nombre de los empleados ordenándolos por fecha de contrato, sacando únicamente los dos primeros (serán los dos más antiguos).

SELECT TOP 3 numemp, nombre FROM empleado ORDER BY contrato

En este caso tiene que sacar los tres primeros, pero si en las fechas de contrato tenemos 20/10/86, 10/12/86, 01/03/87, 01/03/87, la tercera fecha es igual que la cuarta, en este caso sacará estas cuatro filas en vez de tres, y sacaría todas las filas que tuviesen el mismo valor que la tercera fecha de contrato.

El número de filas que queremos visualizar se puede expresar con un número entero o como un porcentaje sobre el número total de filas que se recuperarían sin la cláusula TOP. En este último caso utilizaremos la cláusula TOP n PERCENT.

SELECT TOP 20 PERCENT nombre
FROM empleado
ORDER BY contrato

Lista el nombre de los empleados ordenándolos por fecha de contrato, sacando únicamente un 20% del total de empleados. Como tenemos 10 empleados, sacará los dos primeros, si tuviésemos 100 empleados sacaría los 20 primeros.

LA CLÁUSULA WHERE

WHERE — condición-de-selección —

La cláusula WHERE selecciona únicamente las filas que cumplan la condición de selección especificada.

En la consulta sólo aparecerán las filas para las cuales la condición es verdadera (TRUE), los valores nulos (NULL) no se incluyen por lo tanto en las filas del resultado. La condición de selección puede ser cualquier condición válida o combinación de condiciones utilizando los operadores NOT (no) AND (y) y OR (ó).
Por ejemplo:

SELECT nombre FROM empleados WHERE oficina = 12

Lista el nombre de los empleados de la oficina 12.

SELECT nombre FROM empleados WHERE oficina = 12 AND edad > 30

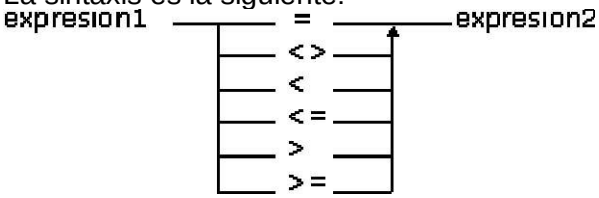
Lista el nombre de los empleados de la oficina 12 que tengan más de 30 años. (oficina igual a 12 y edad mayor que 30)

Condiciones de selección

Las condiciones de selección son las condiciones que pueden aparecer en la cláusula WHERE. En SQL tenemos cinco condiciones básicas:

El test de comparación
El test de rango
El test de pertenencia a un conjunto
El test de valor nulo
El test de correspondencia con patrón

El test de comparación compara el valor de una expresión con el valor de otra.
La sintaxis es la siguiente:



Por ejemplo:
SELECT numemp, nombre
FROM empleados
WHERE ventas > cuota

Lista los empleados cuyas ventas superan su cuota

SELECT numemp, nombre
FROM empleados
WHERE contrato < ’01-01-1988’

Lista los empleados contratados antes del año 88 (cuya fecha de contrato sea anterior al 1 de enero de 1988).

SELECT numemp, nombre
FROM empleados
WHERE YEAR(contrato) < 1988

Este ejemplo obtiene lo mismo que el anterior pero utiliza la función year(). Obtiene los empleados cuyo año de la fecha de contrato sea menor que 1988.

Test de rango (BETWEEN).

Examina si el valor de la expresión está comprendido entre los dos valores definidos por exp1 y exp2.

Tiene la siguiente sintaxis:

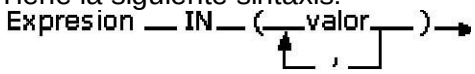
Por ejemplo:

SELECT numemp, nombre
FROM empleados
WHERE ventas BETWEEN 100000 AND 500000

Lista los empleados cuyas ventas estén comprendidas entre 100.000 y 500.00. Se podría obtener lo mismo con la expresión WHERE ventas >= 100000 AND ventas <0 500000.

Test de pertenencia a conjunto (IN)

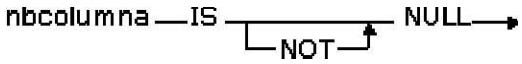
Examina si el valor de la expresión es uno de los valores incluidos en la lista de valores.
Tiene la siguiente sintaxis:



SELECT numemp, nombre, oficina
FROM empleados
WHERE oficina IN (12,14,16)

Lista los empleados de las oficinas 12, 14 y 16. Se podría obtener lo mismo con la expresión WHERE (oficina = 12) OR (oficina = 14) OR (oficina = 16)

Test de valor nulo (IS NULL)
Una condición de selección puede dar como resultado el valor verdadero TRUE, falso FALSE o nulo NULL.
Cuando una columna que interviene en una condición de selección contiene el valor nulo, el resultado de la condición no es verdadero ni falso, sino nulo, sea cual sea el test que se haya utilizado. Por eso si queremos listar las filas que tienen valor en una determinada columna, no podemos utilizar el test de comparación, la condición oficina = null devuelve el valor nulo sea cual sea el valor contenido en oficina. Si queremos preguntar si una columna contiene el valor nulo debemos utilizar un test especial, el test de valor nulo. Tiene la siguiente sintaxis:



Ejemplo:

SELECT oficina, ciudad
FROM oficinas
WHERE dir IS NULL
Lista las oficinas que no tienen director.

Test de correspondencia con patrón (LIKE)
Se utiliza cuando queremos utilizar caracteres comodines para formar el valor con el comparar.
Tiene la siguiente sintaxis:

Los comodines más usados son los siguientes:

_ representa un carácter cualquiera
% representa cero o más caracteres
Ejemplos:

```
SELECT numemp, nombre
FROM empleados
WHERE nombre LIKE 'Luis%'
```

Lista los empleados cuyo nombre empiece por Luis (Luis seguido de cero o más caracteres).

```
SELECT numemp, nombre
FROM empleados
WHERE nombre LIKE '%Luis%'
```

Lista los empleados cuyo nombre contiene Luis, en este caso también saldría los empleados José Luis (cero o más caracteres seguidos de LUIS y seguido de cero o más caracteres).

```
SELECT numemp, nombre
FROM empleados
WHERE nombre LIKE '__a%'
```

Lista los empleados cuyo nombre contenga una a como tercera letra (dos caracteres, la letra a, y cero o más caracteres.

LAS CONSULTAS DE RESUMEN

Es posible definir un tipo de consultas cuyas filas resultantes son un resumen de las filas de la tabla origen, por eso las denominamos consultas de resumen, también se conocen como consultas sumarias.

Es importante entender que las filas del resultado de una consulta de resumen tienen una naturaleza distinta a las filas de las demás tablas resultantes de consultas, ya que corresponden a varias filas de la tabla origen. Para simplificar, veamos el caso de una consulta basada en una sola tabla, una fila de una consulta 'no resumen' corresponde a una fila de la tabla origen, contiene datos que se encuentran en una sola fila del origen, mientras que una fila de una consulta de resumen corresponde a un resumen de varias filas de la tabla origen, esta diferencia es lo que va a originar una serie de restricciones que sufren las consultas de resumen y que veremos a continuación.

En el ejemplo que viene a continuación tienes un ejemplo de consulta normal en la que se visualizan las filas de la tabla oficinas ordenadas por región, en este caso cada fila del resultado se corresponde con una sola fila de la tabla oficinas, mientras que la segunda consulta es una consulta resumen, cada fila del resultado se corresponde con una o varias filas de la tabla oficinas.

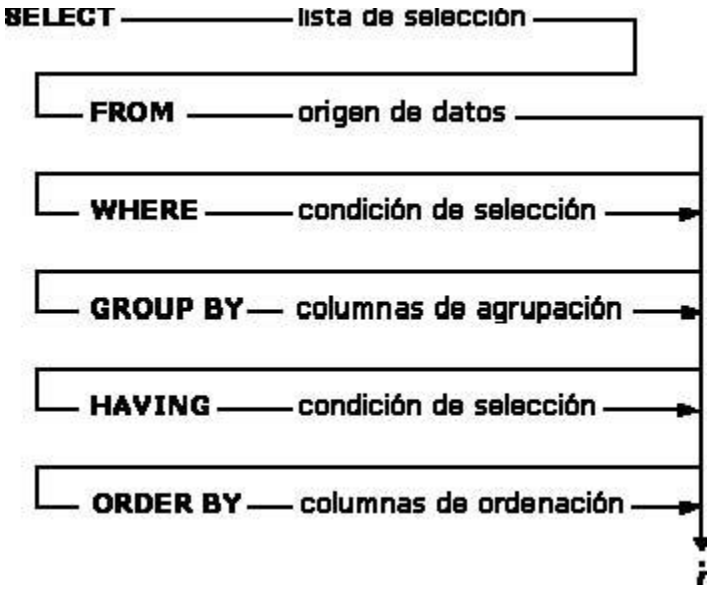
```
SELECT oficina,region,ventas
FROM oficinas
ORDER BY region
```

oficina	region	ventas
24	centro	150.000
23	centro	
28	este	0
13	este	368.000
12	este	735.000
11	este	693.000
26	norte	
22	oeste	186.000
21	oeste	836.000

```
SELECT region,SUM(ventas)
FROM oficinas
GROUP BY region
```

region	SumaDeventas
centro	150000
este	1796000
norte	
oeste	1022000

Las consultas de resumen introducen dos nuevas cláusulas a la sentencia SELECT, la cláusula GROUP BY y la cláusula HAVING, son cláusulas que sólo se pueden utilizar en una consulta de resumen, se tienen que escribir entre la cláusula WHERE y la cláusula ORDER BY y tienen la siguiente sintaxis:



Las detallaremos en la página siguiente del tema, primero vamos a introducir otro concepto relacionado con las consultas de resumen, las funciones de columna.

FUNCIONES DE COLUMNA

En la lista de selección de una consulta de resumen aparecen funciones de columna también denominadas funciones de dominio agregadas. Una función de columna se aplica a una columna y obtiene un valor que resume el contenido de la columna.
Tenemos las siguientes funciones de columna:

SUM (expresión)	MIN (expresión)
AVG (expresión)	MAX (expresión)
STDEV (expresión)	COUNT (nbcolumna)
STDEVP (expresión)	COUNT (*)

El argumento de la función indica con qué valores se tiene que operar, por eso expresión suele ser un nombre de columna, columna que contiene los valores a resumir, pero también puede ser cualquier expresión válida que devuelva una lista de valores.
La función SUM() calcula la suma de los valores indicados en el argumento. Los datos que se suman deben ser de tipo numérico (entero, decimal, coma flotante o monetario...). El resultado será del mismo tipo aunque puede tener una precisión mayor.
Ejemplo:

```
SELECT SUM(ventas)
FROM oficinas
```

Obtiene una sola fila con el resultado de sumar todos los valores de la columna ventas de la tabla oficinas.

La función AVG() calcula el promedio (la media aritmética) de los valores indicados en el argumento, también se aplica a datos numéricos, y en este caso el tipo de dato del resultado puede cambiar según las necesidades del sistema para representar el valor del resultado.

StDev() y StDevP() calculan la desviación estándar de una población o de una muestra de la población representada por los valores contenidos en la columna indicada en el argumento. Si la consulta base (el origen) tiene menos de dos registros, el resultado es nulo. Es interesante destacar que el valor nulo no equivale al valor 0, las funciones de columna no consideran los valores nulos mientras que consideran el valor 0 como un valor, por lo tanto en las funciones AVG(), STDEV(), STDEVP() los resultados no serán los mismos con valores 0 que con valores nulos.
Veámoslo con un ejemplo:

Si tenemos esta tabla:

Tabla1 : Tabla

col1
10
5
0
3
6
0

La consulta SELECT AVG(col1) AS media FROM tabla1

devuelve:

media
4

En este caso los ceros entran en la media por lo que sale igual a 4 (10+5+0+3+6+0)/6 = 4

Con esta otra tabla:

Tabla2 : Tabla

col1
10
5
3
6

SELECT AVG(col1) AS media FROM tabla2

devuelve:

media
6

En este caso los ceros se han sustituido por valores nulos y no entran en el cálculo por lo que la media sale igual a 6 (10+5+3+6)/4 = 4

Las funciones MIN() y MAX() determinan los valores menores y mayores respectivamente. Los valores de la columna pueden ser de tipo numérico, texto o fecha. El resultado de la función tendrá el mismo tipo de dato que la columna. Si la columna es de tipo numérico MIN() devuelve el valor menor contenido en la columna, si la columna es de tipo texto MIN() devuelve el primer valor en orden alfabético, y si la columna es de tipo fecha, MIN() devuelve la fecha más antigua y MAX() la fecha más reciente.

La función COUNT(nb columna) cuenta el número de valores que hay en la columna, los datos de la columna pueden ser de cualquier tipo, y la función siempre devuelve un número entero. Si la columna contiene valores nulos esos valores no se cuentan, si en la columna aparece un valor repetido, lo cuenta varias veces. COUNT(*) permite contar filas en vez de valores. Si la columna no contiene ningún valor nulo, COUNT(nbcolumna) y COUNT(*) devuelven el mismo resultado, mientras que si hay valores nulos en la columna, COUNT(*) cuenta también esos valores mientras que COUNT(nb columna) no los cuenta.

Ejemplo: SELECT COUNT(numemp) FROM empleados
o bien
SELECT COUNT(*) FROM empleados
En este caso las dos sentencias devuelven el mismo resultado ya que la columna numemp no contiene valores nulos (es la clave principal de la tabla empleados).
SELECT COUNT(oficina) FROM empleados
Esta sentencia por el contrario, nos devuelve el número de valores no nulos que se encuentran en la columna oficina de la tabla empleados, por lo tanto nos dice cuántos empleados tienen una oficina asignada.

Se pueden combinar varias funciones de columna en una expresión pero no se pueden anidar funciones de columna, es decir:

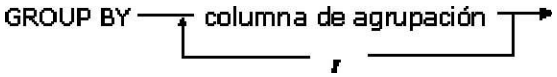
SELECT (AVG(ventas) * 3) + SUM(cuota) FROM ... Es correcto

SELECT AVG(SUM(ventas)) FROM ... NO es correcto, no se puede incluir una función de columna dentro de una función de columna

Si queremos no considerar del origen de datos algunas filas, basta incluir la cláusula WHERE que ya conocemos después de la cláusula FROM. Ejemplo: Queremos saber el acumulado de ventas de los empleados de la oficina 12.

SELECT SUM(ventas) FROM empleados WHERE oficina = 12

LAS FUNCIONES EN LA CLÁUSULA GROUP BY



Hasta ahora las consultas de resumen que hemos visto utilizan todas las filas de la tabla y producen una única fila resultado.
Se pueden obtener subtotales con la cláusula GROUP BY. Una consulta con una cláusula GROUP BY se denomina consulta agrupada ya que agrupa los datos de la tabla origen y produce una única fila resumen por cada grupo formado. Las columnas indicadas en el GROUP BY se llaman columnas de agrupación.
Ejemplo:

SELECT SUM(ventas) FROM repventas

Obtiene la suma de las ventas de todos los empleados.

SELECT SUM(ventas)
FROM repventas
GROUP BY oficina

Se forma un grupo para cada oficina, con las filas de la oficina, y la suma se calcula sobre las filas de cada grupo. El ejemplo obtiene una lista con la suma de las ventas de los empleados de cada oficina.

La consulta quedaría mejor incluyendo en la lista de selección la oficina para saber a qué oficina corresponde la suma de ventas:

SELECT oficina,SUM(ventas)
FROM repventas
GROUP BY oficina

Un columna de agrupación no puede ser de tipo Text o Image.

La columna de agrupación se puede indicar mediante un nombre de columna o cualquier expresión válida basada en una columna pero no se pueden utilizar los alias de campo. Ejemplo:

SELECT importe/cant , SUM(importe)
FROM pedidos
GROUP BY importe/cant

Equivaldría a agrupar las líneas de pedido por precio unitario y sacar de cada precio unitario el importe total vendido.

En la lista de selección sólo pueden aparecer :

valores constantes
funciones de columna
columnas de agrupación (columnas que aparecen en la cláusula GROUP BY)
cualquier expresión basada en las anteriores.

Por lo tanto está permitido una consulta como la que sigue:

```
SELECT SUM(importe), rep*10
FROM pedidos
GROUP BY rep*10
```

Y no está permitido:

```
SELECT SUM(importe), rep
FROM pedidos
GROUP BY rep*10
```

rep es una columna simple que no está encerrada en una función de columna, ni está en la lista de columnas de agrupación.

Se pueden agrupar las filas por varias columnas, en este caso se indican las columnas separadas por una coma y en el orden de mayor a menor agrupación.
Ejemplo: Queremos obtener la suma de las ventas de las oficinas agrupadas por región y ciudad:

```
SELECT SUM(ventas)
FROM oficinas
GROUP BY region, ciudad
```

Todas las filas que tienen valor nulo en el campo de agrupación, pasan a formar un único grupo. Es decir, considera el valor nulo como un valor cualquiera a efectos de agrupación.

LA CLÁUSULA HAVING

HAVING ——— condición de selección ———▶

La cláusula HAVING nos permite seleccionar filas de la tabla resultante de una consulta de resumen.

Para la condición de selección se pueden utilizar los mismos tests de comparación descritos en la cláusula WHERE, también se pueden escribir condiciones compuestas (unidas por los operadores OR, AND, NOT), pero existe una restricción. En la condición de selección sólo pueden aparecer : valores constantes, funciones de columna, columnas de agrupación (columnas que aparecen en la cláusula GROUP BY), o cualquier expresión basada en las anteriores.

Ejemplo: Queremos saber las oficinas con un promedio de ventas de sus empleados mayor que 500.000 pesos.

```
SELECT oficina
FROM empleados
GROUP BY oficina
HAVING AVG(ventas) > 500000
```

NOTA: Para obtener lo que se pide hay que calcular el promedio de ventas de los empleados

de cada oficina, por lo que hay que utilizar la tabla empleados. Tenemos que agrupar los empleados por oficina y calcular el promedio para cada oficina, por último nos queda seleccionar del resultado las filas que tengan un promedio superior a 500.000 pesos.

EJERCICIOS CLASE 3

EJERCICIOS: INSERT

1. Inserte una nueva escuela.

2. Agregue un nuevo guía a la base.

3. Inserte los datos de una escuela existente (nombre y domicilio) pero con un nuevo código.

4. Borre todos los teléfonos que se encuentren en la tabla telefono_Escuela e inserte para todas las escuelas cargadas el teléfono 1111-1111.

   Nota: los valores pueden tener '1111-1111' o 11111111, dependiendo el tipo de dato definido. En este caso, Telefono_Escuela puede ser CHAR o INTERGER.

EJERCICIOS: UPDATE

5. Actualice el teléfono de una de las escuelas por el número 5445-3223.

6. Actualice la fecha de una reserva que usted seleccione por 23/12/2004

7. Debe realizarse un descuento en el arancel por alumno de $2 para todas las reservas de más de 10 alumnos.

8. Actualice el código de guía de las reservas que tengan asignado al guía 1 por el guía 2.

EJERCICIOS: DELETE

9. Borre todas las reservas con menos de 10 Alumnos.

10. Elimine a todos los guías que no tengan cargado su nombre.

EJERCICIOS: SELECT

11. Obtenga un listado de todos los guías de nombre Bernardo.

12. Se desea obtener la cantidad de reservas con fecha mayor a 03/01/2004.

13. Se necesita conocer la cantidad total de alumnos reservados para cada reserva (agrupadas por reservas).

14. Liste todas las reservas que posee una cantidad total de alumnos reservados mayor a 20.

15. Muestre las reservas realizadas en las cuales la inasistencia a las visitas sea mayor a 5.

16. Obtenga la cantidad de escuelas que visitarán el parque después del '3/6/2004'.

CLASE 3 - Adicionales

1. Insertar un nuevo guía con el número inmediato consecutivo al máximo existente.

2. Insertar un nuevo tipo de visita con el número inmediato consecutivo al máximo existente sin utilizar subconsultas.

3. Insertar los datos de la tabla escuela en una nueva tabla, borre los datos de la tabla escuela. En la nueva tabla realice una actualización de los códigos de escuela incrementándolos en uno. Posteriormente reinsértelos en la tabla escuela y vuelva a la normalidad los códigos.

4. Las compañías telefónicas han decidido (por falta de números telefónicos!!), que todas las líneas deben agregar un 9 como primer número. Realice la actualización correspondiente en los teléfonos de las escuelas.

5. Debido a un feriado inesperado, las fechas de las visitas deben posponerse por un día.

6. Obtener los datos de la última reserva existente.

7. Obtener los apellidos de los guias que se encuentren repetidos.

8. Obtener un listado con la cantidad de reservas por fecha.

9. Obtener el promedio de alumnos asistentes, reservados y la diferencia entre estos promedios.

10. Obtener los guias que tengan más de 3 visitas

CLASE 4: CONSULTAS MULTITABLA.

INTRODUCCIÓN

En este tema vamos a estudiar las consultas multitabla llamadas así porque están basadas en más de una tabla.

Existen dos tipos de consultas multitabla:

- la unión de tablas
- la reunión de tablas

LA UNIÓN DE TABLAS

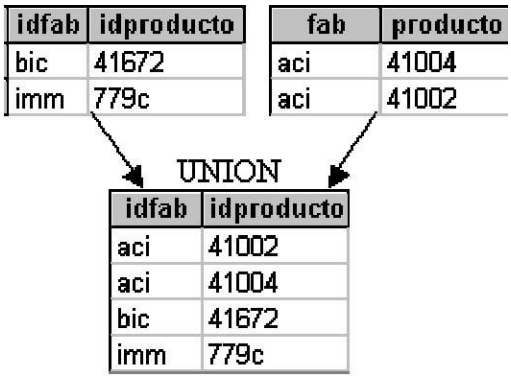
Esta operación se utiliza cuando tenemos dos tablas con las mismas columnas y queremos obtener una nueva tabla con las filas de la primera y las filas de la segunda. En este caso la tabla resultante tiene las mismas columnas que la primera tabla (que son las mismas que las de la segunda tabla).

Por ejemplo tenemos una tabla de libros nuevos y una tabla de libros antiguos y queremos una lista con todos los libros que tenemos. En este caso las dos tablas tienen las mismas columnas, lo único que varía son las filas, además queremos obtener una lista de libros (las columnas de una de las tablas) con las filas que están tanto en libros nuevos como las que están en libros antiguos, en este caso utilizaremos este tipo de operación.

Cuando hablamos de tablas pueden ser tablas reales almacenadas en la base de datos o tablas lógicas (resultados de una consulta), esto nos permite utilizar la operación con más frecuencia ya que pocas veces tenemos en una base de datos tablas idénticas en cuanto a columnas. El resultado es siempre una tabla lógica.

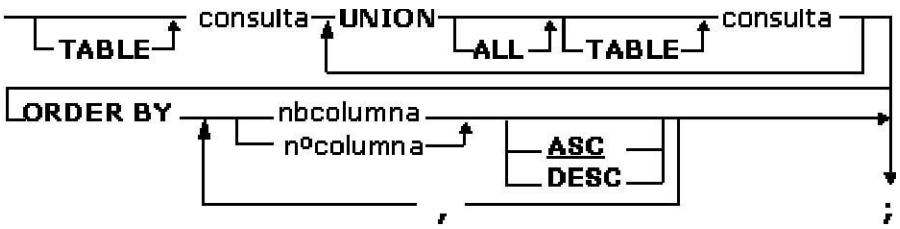
Por ejemplo queremos en un sólo listado los productos cuyas existencias sean iguales a cero y también los productos que aparecen en pedidos del año 90. En este caso tenemos unos productos en la tabla de productos y los otros en la tabla de pedidos, las tablas no tienen las mismas columnas no se puede hacer una union de ellas pero lo que interesa realmente es el identificador del producto (idfab,idproducto), luego por una parte sacamos los códigos de los productos con existencias cero (con una consulta), por otra parte los códigos de los productos que aparecen en pedidos del año 90 (con otra consulta), y luego unimos estas dos tablas lógicas.

El operador que permite realizar esta operación es el operador UNION.



Como ya hemos visto en la página anterior, el operador UNION sirve para obtener a partir de dos tablas con las mismas columnas, una nueva tabla con las filas de la primera y las filas de la segunda.

La sintaxis es la siguiente:



Consulta puede ser un nombre de tabla, un nombre de consulta (en estos dos casos el nombre debe estar precedido de la palabra TABLE), o una sentencia SELECT completa (en este caso no se puede poner TABLE). La sentencia SELECT puede ser cualquier sentencia SELECT con la única restricción de que no puede contener la cláusula ORDER BY.

Después de la primera consulta viene la palabra UNION y a continuación la segunda consulta. La segunda consulta sigue las mismas reglas que la primera consulta.

Las dos consultas deben tener el mismo número de columnas pero las columnas pueden llamarse de diferente forma y ser de tipos de datos distintos.

Las columnas del resultado se llaman como las de la primera consulta.

Por defecto la unión no incluye filas repetidas, si alguna fila está en las dos tablas, sólo aparece una vez en el resultado.

Si queremos que aparezcan todas las filas incluso las repeticiones de filas, incluimos la palabra ALL.

El empleo de ALL tienen una ventaja, la consulta se ejecutará más rapidamente. Puede que la diferencia no se note con tablas pequeñas, pero si tenemos tablas con muchos registros (filas) la diferencia puede ser notable.

Se puede unir más de dos tablas, para ello después de la segunda consulta repetimos la palabra UNION ... y así sucesivamente.

También podemos indicar que queremos el resultado ordenado por algún criterio, en este caso se incluye la cláusula ORDER BY. La cláusula ORDER BY se escribe después de la última consulta, al final de la sentencia; para indicar las columnas de ordenación podemos utilizar su número de orden o el nombre de la columna, en este último caso se deben de utilizar los nombres de columna de la primera consulta ya que son los que se van a utilizar para nombrar las columnas del resultado.

Para ilustrar la operación vamos a realizar el ejercicio visto en la página anterior, vamos a obtener los códigos de los productos que tienen existencias iguales a cero o que aparezcan en pedidos del año 90.

SELECT idfab, idproducto FROM productos WHERE existencias = 0 UNION ALL SELECT fab, producto FROM pedidos WHERE year(fechapedido) = 1990 ORDER BY idproducto

Se ha incluido la cláusula ALL porque no nos importa que salgan filas repetidas.

Se ha incluido ORDER BY para que el resultado salga ordenado por idproducto, observar que hemos utilizado el nombre de la columna de la primera SELECT, también podíamos haber puesto ORDER BY 2 pero no ORDER BY producto (es el nombre de la columna de la segunda tabla).

LA REUNIÓN DE TABLAS

La reunión de tablas consiste en concatenar filas de una tabla con filas de otra. En este caso obtenemos una tabla con las columnas de la primera tabla unidas a las columnas de la segunda tabla, y las filas de la tabla resultante son concatenaciones de filas de la primera tabla con filas de la segunda tabla.

El ejemplo anterior quedaría de la siguiente forma con la reunión:

idfab	idproducto
bic	41672
imm	779c

fab	producto
aci	41004
aci	41002

COMPOSICION

idfab	idproducto	fab	producto
bic	41672	aci	41002
bic	41672	aci	41004
imm	779c	aci	41002
imm	779c	aci	41004

A diferencia de la unión la reunión permite obtener una fila con datos de las dos tablas, esto es muy útil cuando queremos visualizar filas cuyos datos se encuentran en dos tablas.

Por ejemplo queremos listar los pedidos con el nombre del representante que ha hecho el pedido, pues los datos del pedido los tenemos en la tabla de pedidos pero el nombre del representante está en la tabla de empleados y además queremos que aparezcan en la misma

línea; en este caso necesitamos componer las dos tablas (Nota: en el ejemplo expuesto a continuación, hemos seleccionado las filas que nos interesan).

numpedido	fechapedido	rep
112968	11/01/90	101
112992	15/04/90	108

numemp	nombre
101	Antonio Viguer
108	Ana Bustamante

COMPOSICION

numpedido	fechapedido	rep	numemp	nombre
112968	11/01/90	101	101	Antonio Viguer
112992	15/04/90	108	108	Ana Bustamante

Existen distintos tipos de reunión, aprenderemos a utilizarlos todos y a elegir el tipo más apropiado a cada caso.

Los tipos de reunión de tablas son:

- Producto Cartesiano (combinaciones cruzadas CROSS JOIN)
- Reuniones
  - reuniones internas

    INNER JOIN
  - reuniones externas

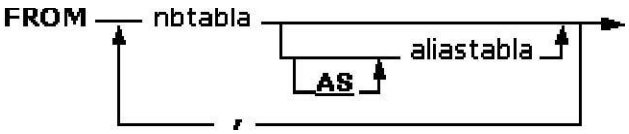
    LEFT OUTER JOIN o LEFT JOIN

    RIGHT OUTER JOIN o RIGHT JOIN

    FULL OUTER JOIN o FULL JOIN

EL PRODUCTO CARTESIANO

El producto cartesiano es un tipo de reunión de tablas, aplicando el producto cartesiano a dos tablas se obtiene una tabla con las columnas de la primera tabla unidas a las columnas de la segunda tabla, y las filas de la tabla resultante son todas las posibles concatenaciones de filas de la primera tabla con filas de la segunda tabla. La sintaxis es la siguiente:



El producto cartesiano se indica poniendo en la FROM las tablas que queremos componer separadas por comas, podemos obtener así el producto cartesiano de dos, tres, o más tablas.

nbtabla puede ser un nombre de tabla o un nombre de consulta.

Hay que tener en cuenta que el producto cartesiano obtiene todas las posibles combinaciones de filas por lo tanto si tenemos dos tablas de 100 registros cada una, el resultado tendrá 100x100 filas, si el producto lo hacemos de estas dos tablas con una tercera de 20 filas, el resultado tendrá 200.000 filas (100x100x20) y estamos hablando de tablas pequeñas. Se ve claramente que el producto cartesiano es una operación costosa sobre todo si operamos con más de dos tablas o con tablas voluminosas.

Se puede reunir una tabla consigo misma, en este caso es obligatorio utilizar un nombre de alias por lo menos para una de las dos.

Por ejemplo:

SELECT * FROM empleados, empleados emp

En este ejemplo obtenemos el producto cartesiano de la tabla de empleados con ella misma. Todas las posibles combinaciones de empleados con empleados.

Esta operación no es de las más utilizadas, normalmente cuando queremos reunir dos tablas es para añadir a las filas de una tabla, una fila de la otra tabla, por ejemplo añadir a los pedidos los datos del cliente correspondiente, o los datos del representante, esto equivaldría a un producto cartesiano con una selección de filas:

SELECT * FROM pedidos,clientes WHERE pedidos.clie=clientes.numclie

Combinamos todos los pedidos con todos los clientes pero luego seleccionamos los que cumplan que el código de cliente de la tabla de pedidos sea igual al código de cliente de la tabla de clientes, por lo tanto nos quedamos con los pedidos combinados con los datos del cliente correspondiente.

Las columnas que aparecen en la cláusula WHERE de nuestra consulta anterior se denominan columnas de emparejamiento ya que permiten emparejar las filas de las dos tablas. Las columnas de emparejamiento no tienen por qué estar incluidas en la lista de selección.

Normalmente emparejamos tablas que están relacionadas entre sí. Para esos casos también es posible usar el tipo de reunión INNER JOIN.

INNER JOIN

La sintaxis es la siguiente:

FROM— tabla1 — **INNER JOIN** — tabla2 — **ON**—tabla1.col1—comp—tabla2.col2

Ejemplo:

SELECT * FROM pedidos INNER JOIN clientes ON pedidos.clie = clientes.numclie

• tabla1 y tabla2 son especificaciones de tabla (nombre de tabla con alias o no, nombre de consulta guardada), de las tablas cuyos registros se van a combinar.

Pueden ser las dos la misma tabla, en este caso es obligatorio definir al menos un alias de tabla.

• col1, col2 son las columnas de emparejamiento.

Observar que dentro de la cláusula ON los nombres de columna deben ser nombres cualificados (llevan delante el nombre de la tabla y un punto).

• Las columnas de emparejamiento deben contener la misma clase de datos, las dos de tipo texto, de tipo fecha etc... los campos numéricos deben ser de tipos similares.

• comp representa cualquier operador de comparación (=, <, >, <=, >=, o <>) y se utiliza para establecer la condición de emparejamiento.

Se pueden definir varias condiciones de emparejamiento unidas por los operadores AND y OR poniendo cada condición entre paréntesis.

Ejemplo:

SELECT * FROM pedidos INNER JOIN productos ON (pedidos.fab = productos.idfab) AND (pedidos.producto = productos.idproducto)

Se pueden combinar más de dos tablas. En este caso hay que sustituir en la sintaxis una tabla por un INNER JOIN completo.

Por ejemplo:

SELECT * FROM (pedidos INNER JOIN clientes ON pedidos.clie = clientes.numclie) INNER JOIN empleados ON pedidos.rep = empleados.numemp

En vez de tabla1 hemos escrito un INNER JOIN completo, también podemos escribir:

SELECT * FROM clientes INNER JOIN (pedidos INNER JOIN empleados ON pedidos.rep = empleados.numemp) ON pedidos.clie = clientes.numclie

En este caso hemos sustituido tabla2 por un INNER JOIN completo.

LEFT JOIN, RIGHT JOIN y FULL JOIN

El LEFT JOIN, RIGHT JOIN y FULL JOIN son otro tipo de reunión de tablas, también denominada reunión externa.

Las composiciones vistas hasta ahora (el producto cartesiano y el INNER JOIN) son reuniones internas ya que todos los valores de las filas del resultado son valores que están en las tablas que se combinan.

Con una reunión interna sólo se obtienen las filas que tienen al menos una fila de la otra tabla que cumpla la condición, veamos un ejemplo:

Queremos combinar los empleados con las oficinas para saber la ciudad de la oficina donde trabaja cada empleado, si utilizamos un producto cartesiano tenemos:

```
SELECT empleados.*,ciudad
FROM empleados, oficinas
WHERE empleados.oficina = oficinas.oficina
```

Observar que hemos cualificado el nombre de columna oficina ya que ese nombre aparece en las dos tablas de la FROM.

Con esta sentencia los empleados que no tienen una oficina asignada (un valor nulo en el campo oficina de la tabla empleados) no aparecen en el resultado ya que la condición empleados.oficina = oficinas.oficina será siempre nula para esos empleados.

Si utilizamos el INNER JOIN:

```
SELECT empleados.*, ciudad FROM empleados INNER JOIN oficinas ON
empleados.oficina = oficinas.oficina
```

Nos pasa lo mismo, el empleado 110 tiene un valor nulo en el campo oficina y no aparecerá en el resultado.

Pues en los casos en que queremos que también aparezcan las filas que no tienen una fila coincidente en la otra tabla, utilizaremos LEFT, RIGHT o FULL JOIN.

La sintaxis del LEFT JOIN es la siguiente:

```
FROM— tabla1 — LEFT JOIN— tabla2— ON—tabla1.col1—comp—tabla2.col2
```

La descripción de la sintaxis es la misma que la del INNER JOIN, lo único que cambia es la palabra INNER por LEFT.

Esta operación consiste en añadir al resultado del INNER JOIN las filas de la tabla de la izquierda que no tienen correspondencia en la otra tabla, y rellenar en esas filas los campos

de la tabla de la derecha con valores nulos.

Ejemplo:

SELECT * FROM empleados LEFT JOIN oficinas ON empleados.oficina = oficinas.oficina

Con el ejemplo anterior obtenemos una lista de los empleados con los datos de su oficina, y el empleado 110 que no tiene oficina aparece con sus datos normales y los datos de su oficina a nulos.

La sintaxis del RIGHT JOIN es la siguiente:

FROM – tabla1 – **RIGHT JOIN** – tabla2 – **ON** – tabla1.col1 – comp – tabla2.col2

La sintaxis es la misma que la del INNER JOIN (ver página anterior), lo único que cambia es la palabra INNER por RIGHT.

Esta operación consiste en añadir al resultado del INNER JOIN las filas de la tabla de la derecha que no tienen correspondencia en la otra tabla, y rellenar en esas filas los campos de la tabla de la izquierda con valores nulos.

Ejemplo:

SELECT *
FROM empleados RIGHT JOIN oficinas ON empleados.oficina = oficinas.oficina

Con el ejemplo anterior obtenemos una lista de los empleados con los datos de su oficina, y además aparece una fila por cada oficina que no está asignada a ningún empleado con los datos del empleado a nulos.

Una operación LEFT JOIN o RIGHT JOIN se puede anidar dentro de una operación INNER JOIN, pero una operación INNER JOIN no se puede anidar dentro de LEFT JOIN o RIGHT JOIN. Los anidamientos de JOIN de distinta naturaleza no funcionan siempre, a veces depende del orden en que colocamos las tablas, en estos casos lo mejor es probar y si no permite el anudamiento, cambiar el orden de las tablas (y por tanto de los JOINs) dentro de la cláusula FROM.

Por ejemplo podemos tener:

SELECT * FROM clientes INNER JOIN (empleados LEFT JOIN oficinas ON
empleados.oficina = oficinas.oficina) ON clientes.repclie = empleados.numclie

Combinamos empleados con oficinas para obtener los datos de la oficina de cada empleado, y luego añadimos los clientes de cada representante, así obtenemos los clientes que tienen un representante asignado y los datos de la oficina del representante asignado.

Si hubiéramos puesto INNER en vez de LEFT no saldrían los clientes que tienen el empleado 110 (porque no tiene oficina y por tanto no aparece en el resultado del LEFT JOIN y por tanto no entrará en el cálculo del INNER JOIN con clientes).

Con FULL JOIN si una fila de la tabla de la izquierda o de la derecha no coincide con los criterios de selección, se especifica que la fila se incluya en el conjunto de resultados y las columnas de resultados que corresponden a la otra tabla se establezcan como NULL. Se trata de una adición a todas las filas que normalmente devuelve la combinación interna (INNER JOIN).

EJERCICIOS CLASE 4

1. Listar las escuelas cuyos domicilios sean de calles que empiecen con S, indicando nombre, domicilio y teléfono. Hacer una versión en la que aparezcan sólo las que tienen teléfono, y hacer otra en la que aparezca solo las escuelas con domicilio en calles que empiecen con S y que no tienen ningún teléfono.

2. Listar las reservas mostrando día, nombre de escuela, cantidad de alumnos de reserva y el nombre del guía.

3. Listar las reservas, la cantidad total real de alumnos y el valor total (cantidad x arancel con iva incluido).

4. Listar las reservas y los nombres de escuelas con valor total mayor a $1.000.

5. Listar las escuelas que fueron atendidas alguna vez por el guía “Eleonora Fernandez”.

6. Listar las escuelas que realizaron más de una reserva para el mismo día.

7. Listar los nombres de los guías y la cantidad de visitas para aquellos con más de 3 visitas de 30 personas.

8. Listar las reservas, el día, la cantidad total de alumnos por tipo de visita, el nombre de la escuela y el nombre del guía.

CLASE 4 - Adicionales

1. Listar los tipos de visita (Codigo y Descripción) que han sido asignadas más de 5 veces.

2. Listar los Guias (Apellido y Nombre) que no han sido nunca asignados a una reserva.

3. Listar los Guias (Apellido y Nombre) con visitas de escuelas cuyo nombre no comience con “E”.

4. Listar las escuelas (Nombre) que tengan visitas en donde la cantidad de alumnos reservada sea igual a la cantidad de alumnos reales asistentes.

5. Realizar la Union de las consultas:
   Nombres de las escuelas con reservas antes de las 9:00 hs

Apellidos de los guías cuyos nombres comiencen con ‘V’
Ordenar el resultado en forma descendente

6. Listar los Nombres y Teléfonos de las Escuelas que concurrieron a la visita: ‘Los Mamuts en Familia’

7. Listar para cada grado (sin importar de cual escuela) cual fue la última vez que hicieron una visita

8. Insertar en una nueva tabla llamada Guia_Performance los datos del Guia y las sumas de las cantidades reservadas y reales del año 2003.

CLASE 5: CONSULTAS ANIDADAS - SUBCONSULTAS

SUBCONSULTAS

Una subconsulta es una sentencia SELECT que aparece dentro de otra sentencia SELECT que llamaremos consulta principal.
Se puede encontrar en la lista de selección SELECT o en las cláusulas FROM, WHERE o HAVING de la consulta principal.

Una subconsulta tiene la misma sintaxis que una sentencia SELECT normal, con las siguientes restricciones:

- se presenta siempre entre paréntesis,
- sólo puede incluir una cláusula ORDER BY cuando se especifica también una cláusula TOP (select top… from… where… order by)
- no puede ser la UNION de varias sentencias SELECT.
- tiene algunas restricciones en cuanto a número de columnas según el lugar donde aparece en la consulta principal.

Cuando se ejecuta una consulta que contiene una subconsulta, la subconsulta se ejecuta por cada fila de la consulta principal. En virtud de ello, se aconseja no utilizar campos calculados en las subconsultas, ya que ralentizan la consulta.

A continuación, a modo de ejemplo, se explica el procesamiento de una consulta que posee una subconsulta:

- Se ejecuta la consulta principal, comparando el campo Codigo_Escuela de cada registro de la tabla Escuela, con el resultado de la subconsulta, que se vuelve a ejecutar para cada registro de la consulta principal.

```
SELECT esc.Nombre_Escuela
FROM escuela esc
WHERE esc.Codigo_Escuela in
```

	Codigo_Escuela	Nombre_Escuela	Domicilio_Escuela
1	1	Escuela Domingo Faustino Sa...	Soler 219 Capital Federal
2	2	Instituto Santa Elena	Las Acacias 23 Pinamar
3	3	Escuela Nacional Nro 4 Nico...	Salvador y Nicaragua s/n C...
4	4	Escuela Técnica El Tornillo	Lomas de Zamora 1 - El Palomar
5	5	Escuela de Modelitos Las Lo...	Paseo Alcorta y Las Cañitas...

	codigo_escuela
	2

```
( select res2.codigo_escuela
from reserva res2 WHERE YEAR(res2.Fecha_Reserva) =
2001 )
```

dando por resultado lo siguiente:

	Nombre_Escuela
1	Instituto Santa Elena

```
SELECT esc.Nombre_Escuela
FROM escuela esc
WHERE esc.Codigo_Escuela in ( select
res2.codigo_escuela
from reserva res2 WHERE YEAR(res2 Fecha Reserva)
```

Referencias externas

A menudo, es necesario, dentro del cuerpo de una subconsulta, hacer referencia al valor de una columna en la fila actual de la consulta principal, ese nombre de columna se denomina referencia externa. Una referencia externa es un nombre de columna que estando en la subconsulta, no se refiere a ninguna columna de las tablas designadas en la cláusula FROM de la subconsulta, sino a las de la consulta principal.

Como la subconsulta se ejecuta por cada fila de la consulta principal, el valor de la referencia externa irá cambiando.

Ejemplo:

SELECT emp.numemp, emp.nombre, (SELECT MIN(ped.fechapedido) FROM pedidos ped WHERE ped.rep = emp.numemp) FROM empleados emp;

En este ejemplo la consulta principal es SELECT... FROM empleados y la subconsulta es (SELECT MIN(fechapedido) FROM pedidos WHERE rep = numemp). En esta subconsulta tenemos una referencia externa (numemp), siendo un campo de la tabla empleados, de la consulta principal).
¿Qué pasa cuando se ejecuta la consulta principal?.

Se toma el primer empleado y se calcula la subconsulta sustituyendo numemp por el valor que tiene en el primer empleado.

La subconsulta obtiene la fecha más antigua en los pedidos del rep = 101. Se toma el segundo empleado y se calcula la subconsulta con numemp = 102 (numemp del segundo empleado)... y así sucesivamente hasta llegar al último empleado.
Al final obtenemos una lista con el número, nombre y fecha del primer pedido de cada empleado. Si quitamos la cláusula WHERE de la subconsulta obtenemos la fecha del primer pedido de todos los pedidos, no del empleado correspondiente.

Anidar subconsultas

Las subconsultas pueden anidarse de forma que una subconsulta aparezca, por ejemplo, en la cláusula WHERE de otra subconsulta, que a su vez forma parte de otra consulta principal.
Pueden existir varios niveles de anidamiento, aunque el límite varía dependiendo de la memoria disponible y de la complejidad del resto de las expresiones de la consulta. En la práctica, una consulta consume mucho más tiempo y memoria cuando se incrementa el número de niveles de anidamiento. La consulta resulta también más difícil de leer, comprender y mantener cuando contiene más de uno o dos niveles de subconsultas.

Ejemplo:

SELECT numemp, nombre FROM empleadosWHERE numemp = (SELECT rep FROM pedidos WHERE clie = (SELECT numclie FROM clientes WHERE nombre = 'Julia Antequera'))

En este ejemplo, por cada linea de pedido se calcula la subconsulta de clientes, y esto se repite por cada empleado, en el caso de tener 10 filas de empleados y 200 filas de pedidos (tablas realmente pequeñas), la subconsulta más interna se ejecutaría 2000 veces (10 x 200).

EMPLEADO	PEDIDO	CLIENTE
1	1	1
2	2	
…	…	
10	200	

10 X 200 X 1

SUBCONSULTA EN LA LISTA DE SELECCIÓN

Cuando la subconsulta aparece en la lista de selección de la consulta principal, en este caso el resultado de la subconsulta, no puede devolver varias filas ni varias columnas, de lo contrario se dará un mensaje de error.

SELECT campo1, (subconsulta), campo2
FROM table [AS] name
WHERE

Por lo tanto, el resultado de la subconsulta que se encuentra en la cláusula SELECT, debe ser

un solo campo (valor).

También se puede obtener lo mismo utilizando como origen de datos las dos tablas. El ejemplo anterior se puede obtener de la siguiente forma:

SELECT numemp, nombre, MIN(fechapedido) FROM empleados LEFT JOIN pedidos ON empleados.numemp = pedidos.rep GROUP BY numemp, nombre

SUBCONSULTA EN LA CLÁUSULA FROM

En la cláusula FROM se puede encontrar una sentencia SELECT encerrada entre paréntesis pero más que subconsulta sería una consulta ya que no se ejecuta para cada fila de la tabla origen sino que se ejecuta una sola vez al principio, su resultado se combina con las filas de la otra tabla para formar las filas origen de la SELECT primera y no admite referencias externas.
En la cláusula FROM vimos que se podía poner un nombre de tabla o un nombre de consulta, pues en vez de poner un nombre de consulta se puede poner directamente la sentencia SELECT correspondiente a esa consulta encerrada entre paréntesis.

SELECT
FROM (subconsulta) [AS] name
WHERE

Por lo tanto, el resultado de la subconsulta en el FROM, puede tener varias filas y varias columnas, razón por la cual sería tratada como una tabla.

SUBCONSULTA EN LAS CLÁUSULAS WHERE Y HAVING

Se suele utilizar subconsultas en las cláusulas WHERE o HAVING cuando los datos que queremos visualizar están en una tabla pero para seleccionar las filas de esa tabla necesitamos un dato que está en otra tabla.

Ejemplo:

SELECT numemp, nombreFROM empleados WHERE contrato = (SELECT MIN(fechapedido) FROM pedidos)

En este ejemplo listamos el número y nombre de los empleados cuya fecha de contrato sea igual a la primera fecha de todos los pedidos de la empresa.

En una cláusula WHERE / HAVING tenemos siempre una condición y la subconsulta actúa de operando dentro de esa condición. En el ejemplo anterior se compara contrato con el resultado de la subconsulta.

Hasta ahora las condiciones estudiadas tenían como operandos valores simples (el valor contenido en una columna de una fila de la tabla, el resultado de una operación aritmética, etc.) ahora la subconsulta puede devolver una columna entera por lo que es necesario definir otro tipo de condiciones especiales para cuando se utilizan con subconsultas.

Podemos decir que hay tres tipos básicos de subconsultas en la clausula WHERE, que son las siguientes:

1. Las que se especifican con un operador de comparación sin modificar y deben devolver un solo valor.

WHERE expression comparison_operator [ANY | ALL] (subconsulta)

Para utilizar este tipo, es necesario que el resultado de la subconsulta sea una sola columna y una fila que permita compararse usando los siguientes operadores: =,<,>,<=,>= ,<>.

2. Las que operan en listas especificadas con IN o modificadas por un operador de comparación mediante ANY o ALL.

WHERE expression [NOT] IN (subconsulta)

Para utilizar este tipo, es necesario que el resultado de la subconsulta sea una sola columna y una o muchas filas que permitan utilizar el operador de comparación de listas IN en la cláusula WHERE.

3. Las que son pruebas de existencia especificadas con EXISTS.

WHERE [NOT] EXISTS (subconsulta)

Para utilizar este tipo, es necesario que el resultado de la subconsulta sea muchas filas y muchas columnas, que permitan comprobar la existencia, usando la palabra EXISTS en la cláusula WHERE.

CONDICIONES DE SELECCIÓN CON SUBCONSULTAS

Las condiciones de selección son las condiciones que pueden aparecer en la cláusula WHERE o HAVING. La mayoría se han visto en el tema 2 pero ahora incluiremos las condiciones que utilizan una subconsulta como operando.

En SQL tenemos cuatro nuevas condiciones:

- el test de comparación con subconsulta (operadores: =,<,>,<=,>= ,<>).
- el test de comparación cuantificada (ANY y ALL).
- el test de pertenencia a un conjunto (IN).
- el test de existencia (EXISTS).

En todos los tests estudiados a continuación la expresion puede ser cualquier nombre de columna de la consulta principal o una expresión válida.

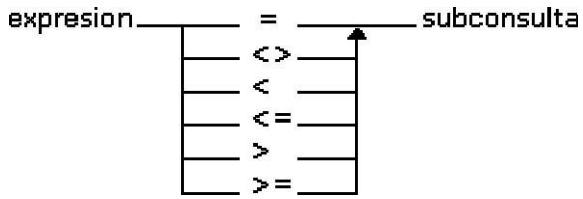
El test de comparación con subconsulta (operadores: =,<,>,<=,>= ,<>).

Es el equivalente al test de comparación simple. Se utiliza para comparar un valor de la fila que se está examinado con un único valor producido por la subconsulta.

La subconsulta debe devolver como resultado, una única columna y una fila, sino se producirá un error. Si la subconsulta no produce ninguna fila o devuelve el valor nulo, el test devuelve el valor nulo, si la subconsulta produce varias filas, SQL devuelve una condición de error.

Suconsulta	Resultado
1 Columna – 1 Fila	Ok
Varias Columnas – 1 Fila	Error
1 Columna – Varias Filas	Error
Varias Columnas – Varias Filas	Error
Ninguna Valor (0 Filas)	Nulo
Nulo	Nulo

La sintaxis es la siguiente:



SELECT oficina, ciudad FROM oficinas WHERE objetivo > (SELECT SUM(ventas) FROM empleados WHERE empleados.oficina = oficinas.oficina);

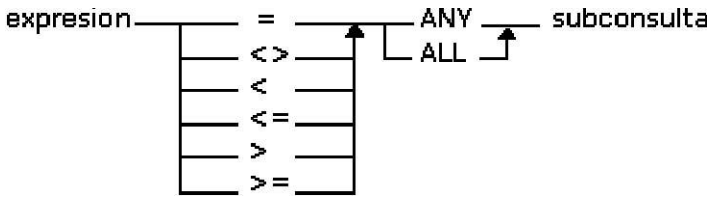
Lista las oficinas cuyo objetivo sea superior a la suma de las ventas de sus empleados. En este caso la subconsulta devuelve una única columna y una única fila (es un consulta de resumen sin GROUP BY)

El test de comparación cuantificada (ANY y ALL)

Este test es una extensión del test de comparación y del test de conjunto. Compara el valor de la expresión con cada uno de los valores producidos por la subconsulta. La subconsulta debe devolver una única columna sino se produce un error. Tenemos el test ANY (algún, alguno en inglés) y el test ALL (todos en inglés).

Suconsulta	Resultado
1 Columna – 1 Fila	Ok
Varias Columnas – 1 Fila	Error
1 Columna – Varias Filas	Ok
Varias Columnas – Varias Filas	Error
Ningún Valor (0 Filas)	Nulo
Nulo	Nulo

La sintaxis es la siguiente:



El test ANY.

Existen las siguientes posibilidades para el test ANY:

- >ANY: El resultado debe cumplir con el requisito de ser mayor de por lo menos de un valor de la lista de los valores devueltos por la subconsulta.
- <ANY: El resultado debe cumplir con el requisito de ser menor de por lo menos de un valor de la lista de los valores devueltos por la subconsulta.
- =ANY: es equivalente a IN. Por ejemplo, para buscar los nombres de todos las escuelas que fueron atendidas por un determinado guía, se puede usar IN o =ANY.
- < >ANY: El resultado debe cumplir con el requisito de ser diferente a por lo menos un valor de la lista de los valores devueltos por la subconsulta.

La subconsulta debe devolver una única columna sino se produce un error.
Se evalúa la comparación con cada valor devuelto por la subconsulta. Si alguna de las comparaciones individuales produce el resultado verdadero, el test ANY devuelve el resultado verdadero. Si la subconsulta no devuelve ningún valor, el test ANY devuelve falso.Si el test de comparación es falso para todos los valores de la columna, ANY devuelve falso. Si el test de comparación no es verdadero para ningún valor de la columna, y es nulo para al menos alguno de los valores, ANY devuelve nulo.

Ejemplo:
SELECT oficina, ciudad FROM oficinas WHERE objetivo > ANY (SELECT SUM(cuota) FROM empleados GROUP BY oficina);

En este caso la subconsulta devuelve una única columna con las sumas de las cuotas de los empleados de cada oficina. Lista las oficinas cuyo objetivo sea superior a alguna de las sumas obtenidas.

El test ALL.

Existen las siguientes posibilidades para el test ANY:

- >ALL: El resultado debe cumplir con el requisito de ser mayor a TODOS los valores de la lista de los valores devueltos por la subconsulta.
- <ALL: El resultado debe cumplir con el requisito de ser menor a TODOS los valores de la lista de los valores devueltos por la subconsulta.
- =ALL: El resultado debe cumplir con el requisito de ser igual a TODOS los valores de la lista de los valores devueltos por la subconsulta.
- < >ALL: es equivalente a NOT IN.

La subconsulta debe devolver una única columna sino se produce un error. Se evalúa la comparación con cada valor devuelto por la subconsulta.

Si todas las comparaciones individuales, producen un resultado verdadero, el test devuelve el valor verdadero. Si la subconsulta no devuelve ningún valor el test ALL devuelve el valor verdadero. (¡Ojo con esto!) Si el test de comparación es falso para algún valor de la columna, el resultado es falso. Si el test de comparación no es falso para ningún valor de la columna, pero es nulo para alguno de esos valores, el test ALL devuelve valor nulo.

Ejemplo:
SELECT oficina, ciudad FROM oficinas WHERE objetivo > ALL (SELECT SUM(cuota) FROM empleados GROUP BY oficina);

En este caso se listan las oficinas cuyo objetivo sea superior a todas las sumas.

Test de pertenencia a conjunto (IN).

El predicado IN se emplea para recuperar únicamente aquellos registros de la consulta principal para los que algunos registros de la subconsulta contienen un valor igual.
Examina si el valor de la expresión es uno de los valores incluidos en la lista de valores producida por la subconsulta. La subconsulta debe generar una única columna y las filas que sean. Si la subconsulta no produce ninguna fila, el test da falso.

Suconsulta	Resultado
1 Columna – 1 Fila	Ok
Varias Columnas – 1 Fila	Error
1 Columna – Varias Filas	Ok
Varias Columnas – Varias Filas	Error
Ningún Valor (0 Filas)	Nulo
Nulo	Nulo

Tiene la siguiente sintaxis:

SELECT numemp, nombre, oficinaFROM empleadosWHERE oficina IN (SELECT oficina FROM oficinas WHERE region = 'este');

Con la subconsulta se obtiene la lista de los números de oficina del este y la consulta principal obtiene los empleados cuyo número de oficina sea uno de los números de oficina del este.Por lo tanto lista los empleados de las oficinas del este.

El test de existencia (EXISTS).

Esta funciona como un test de existencia. La cláusula WHERE de la consulta exterior comprueba por la existencia de las filas retornadas por la subconsulta. La subconsulta en realidad no produce ningún dato, solo retorna un valor de TRUE o FALSE.
Examina si la subconsulta produce alguna fila de resultados. Si la subconsulta contiene filas, el test adopta el valor verdadero, si la subconsulta no contiene ninguna fila, el test toma el valor falso, nunca puede tomar el valor nulo.Con este test la subconsulta puede tener varias columnas, no importa ya que el test se fija no en los valores devueltos sino en si hay o no fila en la tabla resultado de la subconsulta.Cuando se utiliza el test de existencia en la mayoría de los casos habrá que utilizar una referencia externa. Si no se utiliza una referencia externa la subconsulta devuelta siempre será la misma para todas las filas de la consulta principal y en

este caso se seleccionan todas las filas de la consulta principal (si la subconsulta genera filas) o ninguna (si la subconsulta no devuelve ninguna fila).

Suconsulta	Resultado
1 Columna – 1 Fila	Ok
Varias Columnas – 1 Fila	Ok
1 Columna – Varias Filas	Ok
Varias Columnas – Varias Filas	Ok
Ningún Valor (0 Filas)	Falso
Nulo	N/A

La sintaxis es la siguiente:

SELECT numemp, nombre, oficinaFROM empleadosWHERE EXISTS (SELECT * FROM oficinas WHERE region = 'este' AND empleados.oficina = oficinas.oficina);

Este ejemplo obtiene lo mismo que el ejemplo del test IN.Observa que delante de EXISTS no va ningún nombre de columna.En la subconsulta se pueden poner las columnas que queramos en la lista de selección (hemos utilizado el *).Hemos añadido una condición adicional al WHERE, la de la referencia externa para que la oficina que se compare sea la oficina del empleado.
NOTA. Cuando se trabaja con tablas muy voluminosas el test EXISTS suele dar mejor rendimiento que el test IN.

SUBCONSULTAS EN LAS INSTRUCCIONES UPDATE, DELETE, INSERT

En las instrucciones de manipulación de datos (DML) UPDATE, DELETE, INSERT y SELECT se pueden anidar subconsultas.

Para entender mejor esta parte, explicaremos la sintaxis mediante los siguientes ejemplos:

Subconsultas en INSERT

```
--inserta los datos en la TABLA1 con la condicion CAMPO2=OTRO_CAMPO,
--donde OTRO_CAMPO será la columna devuelta de la tabla OTRA_TABLA.
INSERT INTO TABLA1(CAMPO1,CAMPO2,CAMPO3...) VALUES(VALOR1,VALOR2,VALOR3...)
WHERE CAMPO2 IN (SELECT OTRO_CAMPO FROM OTRA_TABLA WHERE condicion)
```

Subconsultas en UPDATE

```
--actualiza la tabla PRODUCTOS, aumentando en 20% el precio unitario del producto
--donde el nombre de la  categoria empieza con la letra P y termina en S.
UPDATE PRODUCTOS SET PRECIO_UNIDAD=PRECIO_UNIDAD*1.2
WHERE ID_CATEGORIA IN (
                        SELECT ID_CATEGORIA FROM CATEGORIAS
                        WHERE NOMBRE_CATEGORIA LIKE 'P%S'
                       )
```

Subconsultas en DELETE

```
--elimina los clientes cuyo cantidad de pedido sea menor a 67.
DELETE  FROM  CLIENTES
WHERE ID_CLIENTE IN (
                      SELECT ID_CLIENTE FROM PEDIDOS WHERE CANTIDAD<67
                     )
```

EJERCICIOS CLASE 5

Los ejercicios que te proponemos a continuación se pueden resolver de varias maneras, intenta resolverlos utilizando subconsultas ya que de eso trata el tema, además un mismo ejercicio lo puedes intentar resolver de diferentes maneras utilizando distintos tipos de condiciones, así un ejercicio se puede convertir en dos o tres ejercicios.

1. Listar los tipos de visita que fueron guiadas alguna vez por Cristina Zaluzi.

2. Listar los nombres de escuela que visitaron en 2002 pero no lo hicieron en el 2001.

3. Listar los guías (codigo, nombre y apellido) que fueron asignados a más de 2 tipos de visita distintos y con una cantidad total real de alumnos que guiaron, mayor a 200.

4. Listar las escuelas que poseen más de 1 reservas con más de 2 tipos de visitas para cada reserva.

5. Listar el nombre, apellido y código de aquellos guías que, en alguna visita de una reserva en particular, hayan atendido por lo menos al 40% de los alumnos totales guiados en todas sus visitas.

6. Listar el nombre y el código de aquellas escuelas que hayan asistido el día en que se registró la mayor cantidad de alumnos reales.

CLASE 5 - Adicionales

1. Listar el código y nombre de las escuelas cuya fecha de reserva sea igual a la primera fecha de reserva realizada.

2. Listar las escuelas que visitaron entre los años 2001 y en el 2002.

3. Listar los guías que tuvieron más de 3 escuelas diferentes y una cantidad total real de alumnos mayor a 200.

4. Listar los nombres y códigos de escuelas con gasto total de todas las visitas mayor a $1700.

5. Listar los guías que hayan tenido en solo un tipo de visita de una reserva en particular por lo menos el 45% del total de alumnos totales que esa persona atendió.

CLASE 6: CREACIÓN DE PROCEDIMIENTOS ALMACENADOS Y TRIGGERS.

PROCEDIMIENTOS ALMACENADOS

CREATE PROCEDURE (Crear Procedimientos almacenados)

Sintaxis (SQL 2000)

Un Procedimiento Almacenado es un programa escrito en lenguaje del DBMS, almacenado como parte de la Base de Datos.

Los procedimientos almacenados son similares a los procedimientos de otros lenguajes de programación en el sentido de que pueden:

Aceptar parámetros de entrada y devolver varios valores en forma de parámetros de salida al lote o al procedimiento que realiza la llamada.

Contener instrucciones de programación que realicen operaciones en la base de datos, incluidas las llamadas a otros procedimientos.

Devolver un valor de estado a un lote o a un procedimiento que realiza una llamada para indicar si la operación se ha realizado correctamente o ha habido un error (y el motivo del mismo)

Los procedimientos almacenados difieren de las funciones en que no devuelven valores en lugar de sus nombres ni pueden utilizarse directamente en una expresión.

El utilizar procedimientos almacenados en vez de programas almacenados localmente en equipos clientes presenta las siguientes ventajas:

Permiten una programación modular.

Se puede crear el procedimiento una vez, almacenarlo en la base de datos y llamarlo desde el programa tantas veces como desee. Un especialista en programación de bases de datos puede crear procedimientos almacenados, que luego será posible modificar independientemente del código fuente del programa.

Permiten una ejecución más rápida: Los procedimientos son analizados y optimizados después de haberlos ejecutado una primera vez, y es posible utilizar una versión compilada del procedimiento que se encuentra en la memoria. Esto significa que no es necesario volver a analizar y optimizar el procedimiento almacenado para cada uso, lo que permite una ejecución mucho más rápida.

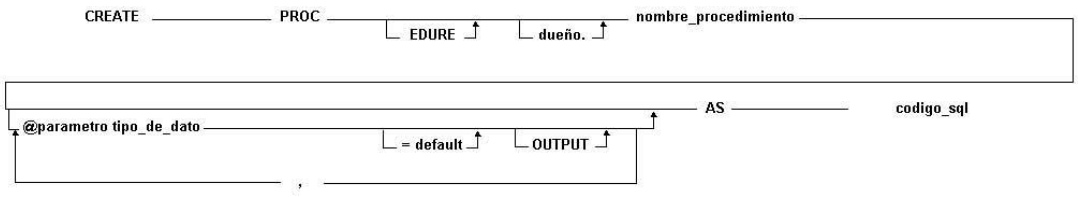
Pueden reducir el tráfico de red: Una operación que necesite centenares de líneas de código SQL puede realizarse mediante una sola instrucción que ejecute el código en un procedimiento, en vez de enviar cientos de líneas de código por la red.

Pueden utilizarse como mecanismo de seguridad: Es posible conceder permisos a los usuarios para ejecutar un procedimiento almacenado, incluso si no cuentan con permiso para ejecutar directamente las instrucciones del procedimiento.

Para crear un procedimiento almacenado se utiliza la instrucción CREATE PROCEDURE de SQL; asimismo, puede utilizarse la instrucción ALTER PROCEDURE para modificarlo. La definición del procedimiento almacenado contiene dos componentes principales:

la especificación del nombre del procedimiento y sus parámetros

y el cuerpo del procedimiento que contiene las instrucciones de SQL que pueden realizar las operaciones del procedimiento.



dueño
Es el nombre del id. de usuario propietario del procedimiento almacenado.

nombre_procedimiento
Es el nombre del nuevo procedimiento almacenado. Los nombres de procedimiento deben seguir las reglas de los identificadores y deben ser únicos en la base de datos y para su propietario.

@parametro
Es un parámetro del procedimiento. En una instrucción CREATE PROCEDURE se pueden declarar uno o más parámetros. El usuario debe proporcionar el valor de cada parámetro declarado cuando se ejecuta el procedimiento, a menos que se haya definido un valor predeterminado para el parámetro o que el valor se establezca para igualar a otro parámetro. Se especifica el nombre de parámetro con un signo (@) como el primer carácter.

tipo_de_dato
Es el tipo de datos del parámetro.

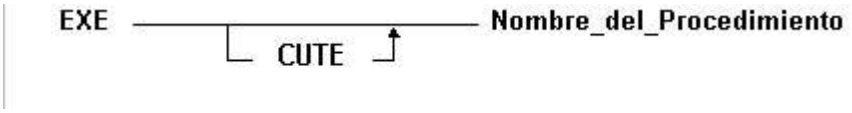
default
Es un valor predeterminado para el parámetro. Si se define un valor predeterminado, el procedimiento se puede ejecutar sin especificar un valor para ese parámetro.

OUTPUT
Indica que se trata de un parámetro de retorno. El valor de esta opción puede devolverse a EXEC[UTE].

AS
Son las acciones que va a llevar a cabo el procedimiento.

codigo_sql
Es cualquier número y tipo de instrucciones Transact-SQL que se incluirán en el procedimiento. Tiene algunas limitaciones. Se pueden incluir varias instrucciones Transact-SQL en el procedimiento.

EXECUTE (Ejecutar Procedimientos almacenados)



Puede usarse la versión abreviada: PROC por PROCEDURE y EXEC por EXECUTE.

Ejemplo:

```
CREATE PROCEDURE MiStoredProcedure @Provincia AS
DECLARE @cuenta_autores int
SELECT @cuenta_autores = COUNT(*) FROM Autor where
Provincia = @Provincia
RETURN @cuenta_autores
```

Este procedimiento almacenado acepta un parámetro del tipo string (cadena de caracteres) y devuelve un integer (entero).

Para encontrar los autores que viven en Misiones Puede usarse la siguiente sintaxis:

```
DECLARE @resultado int
EXEC @resultado = MiStoredProcedure ('MI')
```

Un procedimiento almacenado puede aceptar parámetros de entrada y retornar valores con parámetros de salida. En el siguiente ejemplo se muestra la diferencia:

```
CREATE PROCEDURE MiStoredProcedure
@Provincia varchar(2),@cuenta_autores int OUTPUT AS
SELECT @cuenta_autores =COUNT(*) FROM Autor where Provincia = @Provincia
RETURN 0
```

La sintaxis del llamado de la función ha cambiado. Ahora es necesario enviar una variable al procedimiento almacenado como parámetro de salida:

```
DECLARE @resultado int
EXECUTE MiStoredProcedure 'MI', @resultado OUTPUT
```

TRIGGERS

CREATE TRIGGER (Creación de Triggers)

Un Triggers o disparador es una rutina autónoma asociada con una tabla o vista que automáticamente realiza una acción cuando una fila en la tabla o la vista se inserta (INSERT), se actualiza (UPDATE), o borra (DELETE).

Un Trigger nunca se llama directamente. En cambio, cuando una aplicación o usuario intenta insertar, actualizar, o anular una fila en una tabla, la acción definida en el disparador se ejecuta automáticamente (se dispara).

Los triggers pueden tener varias utilidades. Por ejemplo:

Evitar el almacenamiento de información errónea en la base de datos. Aumentar la integridad referencial o conseguir una seguridad adicional

Controlar restricciones de usuario: por ejemplo, una regla de negocio de una empresa puede especificar que un empleado no puede ganar más que su jefe. Mediante el uso de triggers, si se intenta incluir en la base de datos una información que incumpla esa regla, es la propia base de datos la que no lo permite.

Mantenimiento de valores deducidos: Si tenemos una tabla con valores parciales de ventas y otra con totales, al insertar, borrar o modificar un valor parcial, automáticamente se actualizará el valor total.

Mantener coherentes las tablas replicadas en una BD distribuida.

Registrar en tablas de auditoria las actualizaciones de la base de datos.

El uso de triggers nos proporciona ventajas, como son una mayor facilidad en la codificación de las aplicaciones que se ejecutan contra la base de datos, así como una mayor consistencia de los datos al no tener que realizar las mismas comprobaciones cada vez que se intentan modificar los datos.

Los triggers se denominan también reglas ECA (Evento-Condición-Acción), dado que estos son los tres elementos básicos de un trigger:

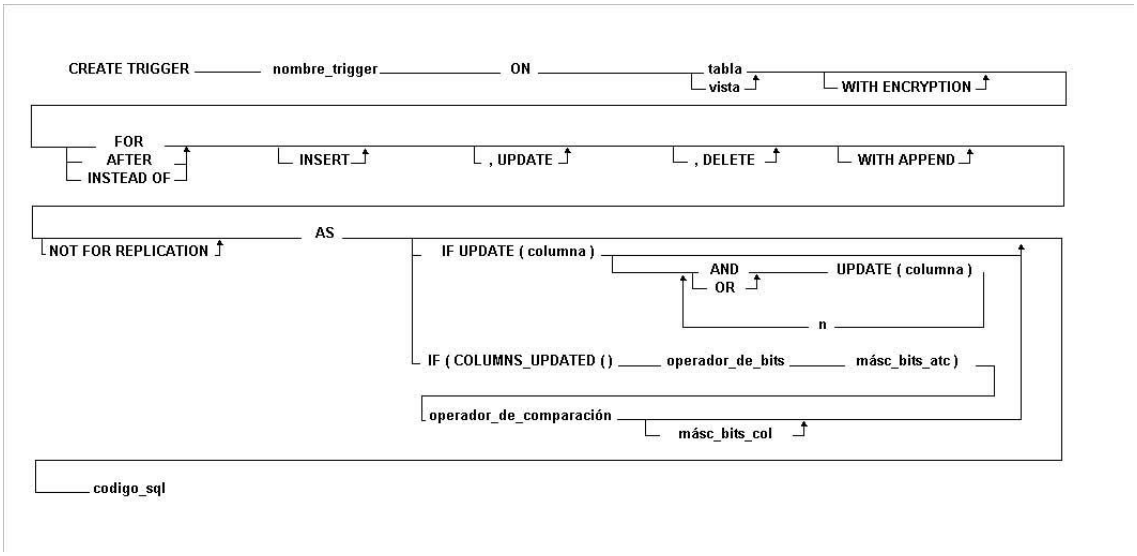
Evento: Es la modificación de datos que se controla con el trigger, y hará que este se dispare. Esta modificación puede ser una inserción (INSERT), un borrado (DELETE) o una actualización (UPDATE).

Condición: La condición que debe verificarse para que el trigger se dispare.

Acción: La acción (normalmente una modificación de datos o evitar que la operación en curso se realice) que se produce como reacción de la BD ante el cambio (el evento, cuando se da la condición).

Sintaxis SQL Server 2000

Se presenta a continuación la sintaxis para crear Triggers en SQL Server 2000. Esta sintaxis no sigue el Estándar ANSI.



nombre_trigger
Es el nombre del trigger. Un nombre de trigger debe cumplir las reglas de los identificadores y debe ser único en la base de datos. Especificar el propietario del trigger es opcional.

tabla | vista
Es la tabla o vista en que se ejecuta el trigger; algunas veces se denomina tabla del trigger o vista del trigger. Especificar el nombre del propietario de la tabla o vista es opcional. Sólo se puede hacer referencia a una vista mediante un trigger INSTEAD OF.

WITH ENCRYPTION
Codifica las entradas syscomments que contienen el texto de CREATE TRIGGER. El uso de WITH ENCRYPTION impide que el trigger se publique como parte de la duplicación de SQL Server.

AFTER
Especifica que el trigger sólo se activa cuando todas las operaciones especificadas en la instrucción SQL desencadenadora se han ejecutado correctamente. Además, todas las acciones referenciales en cascada y las comprobaciones de restricciones deben ser correctas para que este desencadenador se ejecute.
AFTER es el valor predeterminado, si sólo se especifica la palabra clave FOR.
Los triggers AFTER no se pueden definir en las vistas.

INSTEAD OF
Especifica que se ejecuta el trigger en vez de la instrucción SQL desencadenadora, por lo que se suplantan las acciones de las instrucciones desencadenadoras.
Como máximo, se puede definir un trigger INSTEAD OF por cada instrucción INSERT, UPDATE o DELETE en cada tabla o vista. No obstante, en las vistas es posible definir otras vistas que tengan su propio trigger INSTEAD OF.
Los triggers INSTEAD OF no se permiten en las vistas actualizables WITH CHECK OPTION. SQL Server emitirá un error si se agrega un trigger INSTEAD OF a una vista actualizable donde se ha especificado WITH CHECK OPTION. El usuario debe quitar esta opción mediante ALTER VIEW antes de definir el trigger INSTEAD OF.

{ [DELETE] [,] [INSERT] [,] [UPDATE] }
Son palabras clave que especifican qué instrucciones de modificación de datos activan el trigger cuando se intentan en esta tabla o vista. Se debe especificar al menos una opción. En la definición del trigger se permite cualquier combinación de éstas, en cualquier orden. Si especifica más de una opción, sepárelas con comas.
Para los triggers INSTEAD OF, no se permite la opción DELETE en tablas que tengan una relación de integridad referencial que especifica una acción ON DELETE en cascada. Igualmente, no se permite la opción UPDATE en tablas que tengan una relación de integridad referencial que especifica una acción ON UPDATE en cascada.

AS
Son las acciones que va a llevar a cabo el trigger.

codigo_sql
Son las condiciones y acciones del trigger. Las condiciones del trigger especifican los criterios adicionales que determinan si los intentos de las instrucciones DELETE, INSERT o UPDATE hacen que se lleven a cabo las acciones del trigger.
Las acciones del trigger especificadas en las instrucciones Transact-SQL entran en efecto cuando se intenta la operación DELETE, INSERT o UPDATE.
Un trigger está diseñado para comprobar o cambiar los datos en base a una instrucción de modificación de datos; no debe devolver datos al usuario. Las instrucciones Transact-SQL de un trigger incluyen a menudo lenguaje de control de flujo. En las instrucciones CREATE TRIGGER se utilizan unas cuantas tablas especiales:
deleted e inserted son tablas lógicas (conceptuales). Son de estructura similar a la tabla en que se define el trigger, es decir, la tabla en que se intenta la acción del usuario, y guarda los valores antiguos o nuevos de las filas que la acción del usuario puede cambiar.

Por ejemplo, para recuperar todos los valores de la tabla deleted, utilice:

```
SELECT *
FROM deleted
```

n
Se trata de un marcador de posición que indica que se pueden incluir varias instrucciones Transact-SQL en el trigger. Para la instrucción IF UPDATE (columna), se pueden incluir varias columnas repitiendo la cláusula UPDATE (columna).

IF UPDATE (columna)
Prueba una acción INSERT o UPDATE en una columna especificada y no se utiliza con operaciones DELETE. Se puede especificar más de una columna. Como el nombre de la tabla se especifica en la cláusula ON, no lo incluya antes del nombre de la columna en una cláusula IF UPDATE. Para probar una acción INSERT o UPDATE para más de una columna, especifique una cláusula UPDATE(columna) separada a continuación de la primera. IF UPDATE devolverá el valor TRUE en las acciones INSERT porque en las columnas se insertaron valores explícitos o implícitos (NULL).

Nota La cláusula IF UPDATE (columna) funciona de forma idéntica a una instrucción IF, IF...ELSE o WHILE, y puede utilizar el bloque BEGIN...END. UPDATE(columna) puede utilizarse en cualquier parte dentro del cuerpo del trigger.

columna

Es el nombre de la columna que se va a probar para una acción INSERT o UPDATE. Esta columna puede ser de cualquier tipo de datos admitido por SQL Server. No obstante, no se pueden utilizar columnas calculadas en este contexto.

IF (COLUMNS_UPDATED())
Prueba, sólo en un trigger INSERT o UPDATE, si la columna o columnas mencionadas se insertan o se actualizan. COLUMNS_UPDATED devuelve un patrón de bits varbinary que indica qué columnas de la tabla se insertaron o se actualizaron.
La función COLUMNS_UPDATED devuelve los bits en orden de izquierda a derecha, siendo el bit menos significativo el primero de la izquierda. El primer bit de la izquierda representa la primera columna de la tabla, el siguiente representa la segunda columna, etc. COLUMNS_UPDATED devuelve varios bytes si la tabla en que se ha creado el trigger contiene más de 8 columnas, siendo el menos significativo el primero de la izquierda. COLUMNS_UPDATED devolverá el valor TRUE en todas las columnas de las acciones INSERT porque en las columnas se insertaron valores explícitos o implícitos (NULL).

Ejemplos:

Utilizar una regla de empresa trigger entre las tablas empleado y trabajo

```
CREATE TRIGGER empleado_trabajo
ON empleado
FOR INSERT, UPDATE
AS
/* Obtiene el rango del nivel para este tipo de trabajo desde la tabla trabajo */
DECLARE @min_nivel tinyint,
  @max_nivel tinyint,
  @emp_nivel tinyint,
  @trab_id smallint
SELECT @min_nivel = min_nivel,
  @max_nivel = max_nivel,
  @emp_nivel = i.trab_nivel,
  @trab_id = i.trab_id
FROM empleado e INNER JOIN inserted i ON e.emp_id = i.emp_id
  JOIN trabajo j ON j.trab_id = i.trab_id
IF (@trab_id = 1) and (@emp_nivel <> 10)
   BEGIN
     ROLLBACK TRANSACTION
   END
ELSE
   IF NOT (@emp_nivel BETWEEN @min_nivel AND @max_nivel)
     BEGIN
       ROLLBACK TRANSACTION
     END
```

Debido a que las restricciones CHECK sólo pueden hacer referencia a las columnas en que se han definido las restricciones de columna o de tabla, cualquier restricción de referencias cruzadas, en este caso, reglas de empresa, deben definirse como triggers.
Este ejemplo crea un trigger que, cuando se inserta o se cambia un nivel de trabajo de empleado, comprueba que el nivel especificado del trabajo del empleado (nivel_trabajo) en el que se basan los salarios se encuentra en el intervalo definido para el trabajo. Para obtener el intervalo adecuado, debe hacerse referencia a la tabla trabajo.

Como forma de auditar actualizaciones.

```
CREATE TRIGGER actDatosEmpleado
ON DatosEmpleado
FOR update AS
/*Chequea si las columnas 2, 3, o 4 han sido actualizadas. Si alguna de ellas han cambiado, crea un registro de auditoria. La máscara de bits es: power(2,(2-1))+ power(2,(3-1)) + power(2,(4-1)) = 14.

Para chequear si todas las columnas (2, 3, y 4) han cambiado usar =14 en vez de >0*/

IF (COLUMNS_UPDATED() & 14) > 0

/* usar IF (COLUMNS_UPDATED() & 14) = 14 para chequear si todas las columnas (2, 3, y 4) han cambiado.*/

  BEGIN
   INSERT INTO auditaDatosEmpleado
       (audit_log_tipo,
       audit_emp_id,
       audit_emp_NumeroCuentaBancaria,
       audit_emp_salario,
       audit_emp_NumeroObraSocial)
       SELECT 'ANTERIOR',
         del.emp_id,
         del.emp_NumeroCuentaBancaria,
         del.emp_salario,
         del.emp_NumeroObraSocial
       FROM deleted del

    INSERT INTO auditaDatosEmpleado
       (audit_log_tipo,
       audit_emp_id,
       audit_emp_NumeroCuentaBancaria,
       audit_emp_salario,
       audit_emp_NumeroObraSocial)
       SELECT 'NUEVO',
         ins.emp_id,
         ins.emp_NumeroCuentaBancaria,
         ins.emp_salario,
         ins.emp_NumeroObraSocial
       FROM inserted ins
   END
```

En este ejemplo se crean dos tablas: una tabla DatosEmpleado y una tabla AuditaDatosEmpleado. La tabla DatosEmpleado, que contiene información confidencial de los sueldos de los empleados, puede ser modificada por los miembros del departamento de recursos humanos. Si se cambia el número de seguridad social del empleado, el sueldo anual o el número de cuenta bancaria, se genera un registro de auditoría y se inserta en la tabla de auditoría auditaDatosEmpleado.
Con la función COLUMNS_UPDATED(), es posible comprobar rápidamente cualquier cambio en estas columnas que contienen información confidencial de los empleados. COLUMNS_UPDATED() sólo se puede utilizar de esta manera para intentar detectar

modificaciones en las primeras 8 columnas de la tabla.

EJERCICIOS CLASE 6

EJERCICIOS: PROCEDIMIENTOS ALMACENADOS

1. Crear un procedimiento almacenado que tenga como parámetro de entrada un número entero (Codigo_Guia) y devuelva la cantidad total de Alumnos Reales asignados a ese guía.

2. Crear un procedimiento almacenado para ingresar nuevas escuelas, que tenga como parámetros de entrada los valores de Código_Escuela, Nombre_Escuela, y Domicilio_Escuela.

3. Crear un procedimiento almacenado que liste los nombres de los guías con apellidos que comiencen con una cadena de texto determinada por un parámetro de entrada (con valor predeterminado es: comienza con A).

EJERCICIOS: TRIGGERS

4. Crear un trigger sobre la tabla escuela que en caso de que se borre una fila de ésta elimine las respectivas filas de la tabla telefono_escuela.

5. Crear un trigger que en caso de UPDATE del codigo_guia de la tabla Guia, haga el correspondiente update en la tabla reserva_visita.

6. Crear un trigger que en caso de inserción sobre la tabla telefono_escuela verifique la existencia del código de escuela correspondiente y en caso de no encontrarla no realice el proceso.

CLASE 6 - Adicionales

PROCEDIMIENTOS ALMACENADOS Y TRIGGERS

1. Crear un stored procedure que realice una inserción de 26 escuelas en forma secuencial de la forma:

2. Crear un stored procedure que inserte valores en la tabla de Guía (Codigo_Guia, Apellido y Nombre) pero que además de estos parámetros requiera UserID (char) y Password (char). Estos dos parámetros deben ser validados (deben existir) en una tabla llamada USERID.

3. Crear un trigger en donde, si se insertan valores en la tabla Reserva_Visita, si la cantidad de alumnos asistentes es mayor a 50, inserte en una tabla llamada AUDITORIA los valores insertados.

ANEXO I TIPOS DE DATOS DE SQL

Microsoft Access 2000

Los tipos de datos del SQL del motor de base de datos Microsoft Jet son 13 tipos de datos principales definidos por el motor de base de datos Microsoft Jet y varios sinónimos válidos reconocidos para estos tipos de datos.

La tabla siguiente muestra los tipos de datos principales.

Tipo de datos	Tamaño de almacenamiento	Descripción
BINARY	1 byte por carácter	Se puede almacenar cualquier tipo de datos en un campo de este tipo. Los datos no se traducen (por ejemplo, a texto). La forma en que se introducen los datos en un campo binario indica cómo aparecerán al mostrarlos.
BIT	1 byte	Valores Síy No, y campos que contienen solamente uno de dos valores.
TINYINT	1 byte	Un número entero entre 0 y 255.
MONEY	8 bytes	Un número entero comprendido entre –922.337.203.685.477,5808 y 922.337.203.685.477,5807.
DATETIME (Vea DOUBLE)	8 bytes	Una valor de fecha u hora entre los años 100 y 9999
UNIQUEIDENTIFIER	128 bits	Un número de identificación único utilizado con llamadas a procedimientos remotos.
REAL	4 bytes	Un valor de coma flotante de precisión simple con un intervalo comprendido entre –3,402823E38 y – 1,401298E-45 para valores negativos, y desde 1,401298E-45 a 3,402823E38 para valores positivos, y 0.

FLOAT	8 bytes	Un valor de coma flotante de precisión doble con un intervalo comprendido entre –1,79769313486232E308 y –4,94065645841247E-324 para valores negativos, y desde 4,94065645841247E-324 a 1,79769313486232E308 para valores positivos, y 0.
SMALLINT	2 bytes	Un entero de tipo Short (corto) entre – 32.768 y 32.767. (Consulte las notas).
INTEGER	4 bytes	Un entero de tipo Long (largo) entre –2.147.483.648 y 2.147.483.647. (Consulte las notas).

DECIMAL	17 bytes	Un tipo de datos numérico exacto con valores comprendidos entre 1028 - 1 y -1028 - 1. Puede definir la precisión (1 -28) y la escala (0 - precisión definida). La precisión y la escala predeterminadas son 18 y 0, respectivamente.
TEXT (*)	2 bytes por carácter.(Consulte las notas).	Desde cero hasta un máximo de 2.14 gigabytes.
IMAGE	Lo que se requiera	Desde cero hasta un máximo de 2.14 gigabytes. Se utiliza para objetos OLE.
CHARACTER	2 bytes por carácter.(Consulte las notas).	Desde cero a 255 caracteres.

(*) Los caracteres de los campos definidos como TEXT (también conocidos como MEMO) o CHAR (también conocidos como TEXT(n) con una longitud específica) se almacenan en el formato de representación Unicode. Los caracteres Unicode requieren siempre dos bytes para el almacenamiento de cada carácter. Para las bases de datos de Microsoft Jet ya existentes que contengan principalmente datos de tipo carácter, esto puede significar que el tamaño del archivo de base de datos sea casi el doble cuando se convierta al formato Microsoft Jet 4.0.

Microsoft SQL Server 2000

En Microsoft® SQL Server™, cada columna, variable local, expresión y parámetro dispone de un tipo de datos relacionado, que es un atributo que especifica el tipo de datos (integer, character, money, etc) que el objeto puede contener. SQL Server suministra un conjunto de tipos de datos del sistema que define todos los tipos de datos que pueden utilizarse con SQL Server. El conjunto de tipos de datos suministrados por el sistema se muestra debajo.

Tipo De Datos De Sql Server 2000	Descripción
BIGINT	Datos enteros (números enteros) comprendidos entre -2^63 (-9223372036854775808) y 2^63 -1 (9223372036854775807).
BINARY(N)	Datos binarios de longitud fija con una longitud máxima de 8.000 bytes.
BIT	Datos enteros con valor 1 ó 0.
CHAR(N)	Datos de caracteres no Unicode de longitud fija con una longitud máxima de 8.000 caracteres.
CURSOR	Una referencia a un cursor.
DATETIME	Datos de fecha y hora comprendidos entre el 1 de enero de 1753 y el 31 de diciembre de 9999, con una precisión de 3,33 milisegundos.
DECIMAL	Datos de precisión y escala numérica fijas comprendidos entre -1038 +1 y 1038 - 1.

FLOAT	Números con precisión de coma flotante comprendidos entre -1,79E + 308 y 1,79E + 308.
IMAGE	Datos binarios de longitud variable con una longitud máxima de 231 - 1 (2.147.483.647) bytes.
INT	Datos enteros (números enteros) comprendidos entre -2^31 (-2.147.483.648) y 2^31 - 1 (2.147.483.647).
MONEY	Valores de moneda comprendidos entre -263 (-922.337.203.685.477,5808) y 263 - 1 (+922.337.203.685.477,5807), con una precisión de una diezmilésima de la unidad monetaria.
NCHAR(N)	Datos Unicode de longitud variable con una longitud máxima de 4.000 caracteres.
NTEXT	Datos Unicode de longitud variable con una longitud máxima de 230 - 1 (1.073.741.823) caracteres.
NUMERIC	Funcionalmente equivalente a decimal.
NVARCHAR(N)	Datos Unicode de longitud variable con una longitud máxima de 4.000 caracteres.
REAL	Números con precisión de coma flotante comprendidos entre -3,40E + 38 y 3,40E + 38.
SMALLDATETIME	Datos de fecha y hora comprendidos entre el 1 de enero de 1900 y el 6 de junio de 2079, con una precisión de un minuto.
SMALLINT	Datos enteros comprendidos entre 215 (-32.768) y 215 -1 (32.767).
SMALLMONEY	Valores de moneda comprendidos entre -214.748,3648 y +214.748,3647, con una precisión de una diezmilésima de la unidad monetaria.
SQL_VARIANT	Un tipo de datos que almacena valores de varios tipos de datos aceptados en SQL Server, excepto text, ntext, timestamp y sql_variant.
TABLE	Un tipo de datos especial que se utiliza para almacenar un conjunto de resultados para un proceso posterior.
TEXT	Datos no Unicode de longitud variable con una longitud máxima de 231 - 1 (2.147.483.647) caracteres.
TIMESTAMP	Un número único para toda la base de datos que se actualiza cada vez que se actualiza una fila.
TINYINT	Datos enteros comprendidos 0 y 255.
UNIQUEIDENTIFIER	Un identificador exclusivo global (GUID).
VARBINARY(N)	Datos binarios de longitud variable con una longitud máxima de 8.000 bytes.
VARCHAR(N)	Datos no Unicode delongitud variable con un máximo de 8.000 caracteres.

Notas:

• SQL Server 2000 también admite un tipo de datos base table, que se puede utilizar para almacenar el conjunto de resultados de una instrucción SQL. No se puede utilizar el tipo de datos table en columnas de una tabla. Sólo es posible utilizarlo en variables Transact-SQL y en los valores de retorno de funciones definidas por el usuario.

• El tipo de datos sql_variant de SQL Server 2000 es un tipo de datos especial que le permite almacenar valores de múltiples tipos de datos base en la misma columna. Por ejemplo, en la misma columna puede almacenar valores nchar, valores int y valores decimal.

• En SQL Server 2005 aparecen nuevos tipos de datos para manejar las fechas y las horas, Date para el manejo de fechas desde el 01/01/0001 al 31/12/9999; Time para el manejo del tiempo de hasta 24 horas y UtcDateTime con control de la zona horaria. Cabe mencionar que las versiones anteriores de SQL Server, los tipos de datos datetime y smalldatetime incluían tanto la fecha como la hora, con lo cual no había tipos de datos separados para manejar cada una de las partes.

Tipos de datos equivalentes del SQL de ANSI

La tabla siguiente enumera los tipos de datos del SQL de ANSI, los tipos de datos equivalentes del SQL del motor de base de datos Microsoft Jet y sus sinónimos válidos. También enumera los tipos de datos equivalentes de Microsoft® SQL Server™.

Tipo de datos del SQL de ANSI	Tipo de datos delSQL de Microsoft Jet	Sinónimo	Tipo de datos de Microsoft SQL Server
BIT, BIT VARYING	BINARY (consulte las notas)	VARBINARY, BINARY VARYINGBIT VARYING	BINARY, VARBINARY
No admitido	BIT (consulte las notas)	BOOLEAN, LOGICAL, LOGICAL1, YESNO	BIT
No admitido	TINYINT	INTEGER1, BYTE	TINYINT
No admitido	COUNTER (consulte las notas)	AUTOINCREMENT	(Consulte las notas)
No admitido	MONEY	CURRENCY	MONEY
DATE, TIME, TIMESTAMP	DATETIME	DATE, TIME (consulte las notas)	DATETIME
No admitido	UNIQUEIDENTIFIER	GUID	UNIQUEIDENTIFIER
DECIMAL	DECIMAL	NUMERIC, DEC	DECIMAL
REAL	REAL	SINGLE, FLOAT4, IEEESINGLE	REAL

DOUBLE PRECISION, FLOAT	FLOAT	DOUBLE, FLOAT8, IEEEDOUBLE, NUMBER (consulte las notas)	FLOAT
SMALLINT	SMALLINT	SHORT, INTEGER2	SMALLINT
INTEGER	INTEGER	LONG, INT, INTEGER4	INTEGER
INTERVAL	No admitido		No admitido
No admitido	IMAGE	LONGBINARY, GENERAL, OLEOBJECT	IMAGE
No admitido	TEXT (consulte las notas)	LONGTEXT, LONGCHAR, MEMO, NOTE, NTEXT (consulte las notas)	TEXT

CHARACTER, CHARACTER VARYING, NATIONAL CHARACTER, NATIONAL CHARACTER VARYING	CHAR (consulte las notas)	TEXT(n), ALPHANUMERIC, CHARACTER, STRING, VARCHAR, CHARACTER VARYING, NCHAR, NATIONAL CHARACTER, NATIONAL CHAR, NATIONAL CHARACTER VARYING, NATIONAL CHAR VARYING (consulte las notas)	CHAR, VARCHAR, NCHAR, NVARCHAR

Notas:

El tipo de datos BIT del SQL de ANSI no se corresponde con el tipo de datos BIT del

SQL de Microsoft Jet. Corresponde al tipo de datos BINARY. No hay equivalente en el SQL de ANSI a ningún tipo de datos BIT del SQL de Microsoft Jet.
Ya no se puede utilizar TIMESTAMP como sinónimo de DATETIME.
Tampoco se puede utilizar NUMERIC como sinónimo de FLOAT ni de DOUBLE. NUMERIC se utiliza ahora como sinónimo de DECIMAL.
Un campo LONGTEXT se almacena siempre en el formato de representación Unicode.
Si se utiliza el nombre del tipo de datos TEXT sin especificar la longitud opcional (TEXT(25), por ejemplo), se crea un campo LONGTEXT. Esto permite escribir instrucciones CREATE TABLE que producirán tipos de datos coherentes con Microsoft SQL Server.
Un campo CHAR se almacena siempre en el formato de representación Unicode, que es el equivalente del tipo de datos NATIONAL CHAR del SQL de ANSI.
Si se utiliza el nombre del tipo de datos TEXT y se especifica la longitud opcional (TEXT(25), por ejemplo), el tipo de datos del campo es equivalente al tipo de datos CHAR. De ese modo, se mantiene la compatibilidad con versiones anteriores para la mayoría de las aplicaciones de Microsoft Jet, a la vez que se habilita el tipo de datos TEXT (sin especificación de longitud) para la alineación con Microsoft SQL Server.

Valores NULL

Las columnas pueden aceptar o rechazar valores NULL. NULL es un valor especial en las bases de datos que representa el concepto de un valor desconocido. NULL no es igual a un carácter de espacio en blanco ó 0. En realidad, el espacio en blanco es un carácter válido y 0 es un número válido. NULL simplemente representa la idea de que este valor no se conoce. NULL tampoco es lo mismo que una cadena de longitud cero.
Si la definición de una columna contiene la cláusula NOT NULL, no se pueden insertar filas que tengan el valor NULL en dicha columna. Si la definición de una columna sólo tiene la palabra clave NULL, acepta valores NULL.
La aceptación de valores NULL en una columna puede aumentar la complejidad de las comparaciones lógicas que utilicen la columna. El estándar SQL-92 indica que el resultado de las comparaciones con valores NULL no es TRUE (verdadero) o FALSE (falso), sino UNKNOWN. Esto introduce una lógica de tres valores en los operadores de comparación, que puede ser difícil de controlar correctamente.

ANEXO II - EJERCICIOS RESUELTOS

CLASE 1

9. Crear una base de datos de nombre Jurassic_Park sin especificar los valores de la base.

```
CREATE DATABASE Jurassic_Park ;
```

10. Crear la tabla Escuela y definir su clave principal en la misma instrucción de creación. Continuar con tablas Guia, Reserva y Tipo_Visita.

```
CREATE TABLE Escuela (
codigo_escuela  CHAR(10) PRIMARY KEY,
nombre_escuela VARCHAR(30) NOT NULL,
domicilio_escuela TEXT(30) NOT NULL);

CREATE TABLE Guia (
codigo_guia CHAR(10) PRIMARY KEY,
nombre_guia VARCHAR(20) NOT NULL,
apellido_guia VARCHAR(20) NOT NULL);

CREATE TABLE Reserva (
numero_reserva CHAR (10) PRIMARY KEY,
fecha_visita_reservada DATETIME NOT NULL,
hora_visita_reservada DATETIME NOT NULL);

CREATE TABLE Tipo_Visita (
codigo_tipo_visita CHAR(10) PRIMARY KEY,
descripción_visita VARCHAR(30),
arancel_por_alumno MONEY);
```

11. Crear la tabla Telefono_Escuela con su clave principal. (hacer restricción en caso de ser una CP compuesta).

```
CREATE TABLE Telefono_Escuela(
telefono_escuela INT,
codigo_escuela CHAR(10) CONSTRAINT codigo_escuela REFERENCES
Escuela,
CONSTRAINT cp PRIMARY KEY (telefono_escuela, codigo_escuela));
```

12. Crear la tabla Reserva_Por_Grado con su clave principal. Hacer las correspondientes restricciones.

```
CREATE TABLE Reserva_Por_Grado (
numero_reserva CHAR(10) CONSTRAINT numero_reserva REFERENCES
Reserva,
codigo_tipo_visita CHAR(10) CONSTRAINT Tipo_Visita REFERENCES
```

```
Tipo_Visita,
numero_grado INT,
CONSTRAINT cp PRIMARY KEY (
numero_reserva, codigo_tipo_visita, numero_grado));
```

13. Crear la tabla Reserva_Tipo_Visita con sus campos propios y los referenciados. Sin generar claves.

```
CREATE TABLE Reserva_Tipo_Visita (
numero_reserva  CHAR(10) CONSTRAINT numero_reserva2 REFERENCES
Reserva,
codigo_tipo_visita CHAR(10) CONSTRAINT codigo_tipo_visita2
REFERENCES Tipo_Visita,
cantidad_alumnos_reservados INT,
cantidad_alumno_reales  INT);
```

14. Completar el ejercicio anterior, con la creación de las claves correspondientes.

```
CREATE TABLE Reserva_Tipo_Visita (
numero_reserva  CHAR(10) CONSTRAINT numero_reserva2 REFERENCES
Reserva, codigo_tipo_visita CHAR(10) CONSTRAINT
codigo_tipo_visita2 REFERENCES Tipo_Visita,
cantidad_alumnos_reservados INT,
cantidad_alumno_reales  INT,
codigo_guia CHAR(10),
CONSTRAINT cf_codigo_guia FOREIGN KEY (codigo_guia) REFERENCES
Guia, CONSTRAINT cp PRIMARY KEY (numero_reserva,
codigo_tipo_visita ));
```

15. Añadir a la tabla de Guía la columna sueldo_hora.

```
ALTER TABLE guia
ADD sueldo_hora MONEY ;
```

Nota: En SQL Server no es necesario agregar COLUMN.

16. Hacer que no puedan haber dos escuelas con el mismo nombre.

Solución 1

```
ALTER TABLE escuela
ADD CONSTRAINT u_nombre UNIQUE (nombre_escuela);
```

Solución 2

```
CREATE UNIQUE INDEX u_nombre ON escuela (nombre);
```

CLASE 1 - Adicionales

11. Crear la tabla Distrito_Escolar con su correspondiente CP.

```
CREATE TABLE Distrito_Escolar
(codigo_distrito CHAR(10) PRIMARY KEY,
descripcion_distrito CHAR(50));
```

12. Agregar clave foránea codigo_distrito_escolar a la tabla Escuela

```
ALTER TABLE Escuela
ADD codigo_distrito CHAR(10) CONSTRAINT cfE FOREIGN KEY
(codigo_distrito) REFERENCES Distrito_Escolar;
```

13. Eliminar la columna de domicilios de la tabla Escuela.

```
ALTER TABLE Escuela
DROP COLUMN domicilio_escuela;
```

14. Agregar columnas calle_escuela y altura_escuela a la tabla Escuela.

```
ALTER TABLE Escuela
DROP COLUMN domicilio_escuela;

ALTER TABLE Escuela
ADD calle_escuela CHAR(30),
altura_escuela INT;
```

15. Agregar domicilio_guia en tabla Guia.

```
ALTER TABLE Guia
ADD domicilio_guia CHAR(50);
```

16. Eliminar tabla Telefono_Escuela

```
DROP TABLE Telefono_Escuela;
```

17. Agregar tabla Email_Escuela (sin Clave primaria)

```
CREATE TABLE Email_Escuela
(email_escuela CHAR(30) NOT NULL,
codigo_escuela CHAR(10) NOT NULL CONSTRAINT cfEE FOREIGN KEY
(codigo_escuela) REFERENCES escuela);
```

18. Establecer clave primaria para Email_Escuela

```
ALTER TABLE Email_Escuela
ADD CONSTRAINT cpEE PRIMARY KEY (codigo_escuela,
telefono_escuela);
```

19. Establecer que los nombres y apellidos de los guias no tengan valores nulos. nulos.

```
ALTER TABLE Guia
```

```
ALTER COLUMN nombre_guia CHAR(10) NOT NULL;
ALTER COLUMN apellido_guia CHAR(10) NOT NULL;
```

20. Establecer que no se repita la calle y la altura de las escuelas.

```
ALTER TABLE Escuela
ADD constraint Unico_E UNIQUE (calle_escuela, altura_escuela);
```

CLASE 2

1. Crear un índice agrupado para la tabla Reserva sobre el campo codigo_escuela.

```
CREATE INDEX [idx_cod_escuela] ON [dbo].[reserva]
([Codigo_Escuela]) ;
```

2. Eliminar el índice creado en el punto anterior.

```
DROP INDEX reserva.idx_cod_escuela ;
```

3. Crear un índice que optimice la búsqueda y ordenamiento de la tabla reserva_visita, si la consulta que se quisiera optimizar es por Cantidad de Alumnos Reservados.

```
CREATE INDEX [idx_cant_reservas] ON [dbo].[reserva_tipo_visita]
([Cantidad_Alumnos_Reservados]) ;
```

4. Crear un nuevo índice agrupado sobre la tabla Guia sobre los campos Apellido y Nombre.

```
ALTER TABLE dbo.guia DROP CONSTRAINT PK__guia__35BCFE0A ;

CREATE CLUSTERED INDEX [idx_Apellido] ON [dbo].[guia]
([Apellido_Guia], [Nombre_Guia]) ;
```

Nota: Puede que no permita borrar la clave primaria.

5. Crear un índice que optimice la búsqueda por Fecha y Hora de la reserva (índice compuesto) teniendo en cuenta que se realizan inserciones con mucha asiduidad sobre esta tabla (FILLFACTOR).

```
CREATE INDEX [idx_fecha_hora] ON [dbo].[reserva]
([Fecha_visita_Reservada], [Hora_visita_Reservada]) WITH
FILLFACTOR = 40 ;
```

6. Conceder permiso de INSERT y UPDATE sobre la tabla telefono_escuela a los usuarios jPerez, aFernandez.

```
GRANT INSERT, UPDATE ON telefono_escuela TO jPerez, aFernandez ;
```

7. Eliminar los permisos de UPDATE sobre la tabla telefono_escuela para todos los usuarios.

```
DENY UPDATE ON telefono_escuela TO EVERYONE ;
```

8. Conceder permisos de create table al usuario jPerez

```
GRANT CREATE TABLE TO jPerez ;
```

9. Revocar los permisos de select para el usuario public sobre la tabla de Escuela.

```
DENY SELECT ON escuela TO PUBLIC ;
```

CLASE 2 - Adicionales

7. Eliminar el índice sobre nombre y apellido en la tabla guia.

```
DROP INDEX guia.idx_cod_guia ;
```

8. Crear un índice que impida ingresar dos guías con igual nombre y apellido.

```
CREATE unique INDEX idx_nom_guia ON guia (
apellido_guia, nombre_guia) ;
```

9. Conceder al grupo CORDOBA permiso de SELECT para la tabla reserva_visita

```
GRANT SELECT ON reserva_tipo_visita TO CORDOBA;
```

10. El usuario jPerez (parte del grupo CORDOBA) no debe poder ver el campo Arancel_por_Alumno (pero sí el resto de la tabla). ¿Como se le deniega el permiso?

```
DENY SELECT Arancel_por_Alumno ON reserva_visita TO jPerez ;
```

11. Conceder al grupo ADM todos los permisos de aplicables sobre la Base Jurasik_Park (creacion de tablas, backup, etc)

```
GRANT ALL TO ADM ;
```

12. Denegar al usuario aFernandez (miembro del grupo ADM) el permiso para realizar backups sobre la base y sobre el log de transacciones.

```
DENY BACKUP DATABASE, BACKUP LOG TO aFernandez ;
```

CLASE 3

EJERCICIOS: INSERT

1. Inserte una nueva escuela.

```
INSERT INTO escuela
 VALUES (6, 'N° 10', 'TEST 4444') ;
INSERT INTO telefono_escuela (codigo_escuela, telefono_escuela)
 VALUES (6, 44445444) ;
```

2. Agregue un nuevo guía a la base.

```
INSERT INTO guia (codigo_guia, nombre_guia, apellido_guia)
VALUES(11, 'FERNANDA', 'GARCIA') ;
```

3. Inserte los datos de una escuela existente (nombre y domicilio) pero con un nuevo código.

```
INSERT INTO escuela
SELECT 10, nombre_escuela, domicilio_escuela
  FROM escuela
 WHERE codigo_escuela = 6 ;
```

4. Borre todos los teléfonos que se encuentren en la tabla telefono_Escuela e inserte para todas las escuelas cargadas el teléfono 1111-1111.

```
DELETE FROM telefono_escuela ;

INSERT INTO telefono_escuela(codigo_escuela, telefono_escuela)
SELECT codigo_escuela, '1111-1111'
  FROM escuela ;
```

Nota: los valores pueden tener '1111-1111' o 11111111, dependiendo el tipo de dato definido. En este caso, Telefono_Escuela puede ser CHAR o INTERGER.

EJERCICIOS: UPDATE

5. Actualice el teléfono de una de las escuelas por el número 5445-3223.

```
UPDATE telefono_escuela SET telefono_escuela = '5445-3223'
 WHERE codigo_escuela = 5 ;
```

6. Actualice la fecha de una reserva que usted seleccione por 23/12/2004

```
UPDATE reserva SET fecha_visita_reservada = '20041223'
 WHERE numero_reserva = 2 ;
```

7. Debe realizarse un descuento en el arancel por alumno de $2 para todas las reservas de más de 10 alumnos.

Si arancel_por_alumno está definido en la tabla Reserva_Tipo_Visita es:

```
UPDATE reserva_tipo_visita
   SET arancel_por_alumno = arancel_por_alumno - 2
 WHERE cantidad_alumnos_reservados > 10 ;
```

Si arancel_por_alumno está definido en la tabla Tipo_Visita es:

```
UPDATE tipo_visita
   SET arancel_por_alumno = arancel_por_alumno - 2 ;
-- sin utilizar el criterio de "mas de 10 alumnos".
```

8. Actualice el código de guía de las reservas que tengan asignado al guía 1 por el guía 2.

```
UPDATE reserva_tipo_visita
   SET codigo_guia = 2
 WHERE codigo_guia = 1 ;
```

EJERCICIOS: DELETE

9. Borre todas las reservas con menos de 10 Alumnos.

```
DELETE FROM reserva_tipo_visita
 WHERE cantidad_alumnos_reservados  < 10 ;
```

10. Elimine a todos los guías que no tengan cargado su nombre.

```
DELETE FROM guia
 WHERE nombre_guia = '' OR nombre_guia IS NULL ;
```

EJERCICIOS: SELECT

11. Obtenga un listado de todos los guías de nombre Bernardo.

```
SELECT *
  FROM guia
 WHERE nombre_guia LIKE '%Bernardo%' ;
```

Resultado:
1 Lopez Aguado Bernardo

12. Se desea obtener la cantidad de reservas con fecha mayor a 03/01/2004.

```
SELECT COUNT(* )
  FROM reserva
 WHERE fecha_visita_reservada > '20040103' ;
```

Resultado:
1

13. Se necesita conocer la cantidad total de alumnos reservados para cada reserva (agrupadas por reservas).

```
SELECT numero_reserva,
       SUM(cantidad_alumnos_reservados) AS total_reserva
    FROM reserva_tipo_visita
GROUP BY numero_reserva ;
```

Resultado:

1	165
2	246
3	227
4	250
5	209
6	65
7	210
8	57
9	70

14. Liste todas las reservas que posee una cantidad total de alumnos reservados mayor a 20.

```
  SELECT numero_reserva,
         SUM(cantidad_alumnos_reservados) AS total_reserva
    FROM reserva_tipo_visita
GROUP BY numero_reserva
  HAVING SUM(cantidad_alumnos_reservados) > 100 ;
```

Resultado:

1	165
2	246
3	227
4	250
5	209
7	210

15. Muestre las reservas realizadas en las cuales la inasistencia a las visitas sea mayor a 5.

```
SELECT numero_reserva,
       cantidad_alumnos_reservados ,
       cantidad_alumnos_reales
  FROM reserva_tipo_visita
 WHERE (cantidad_alumnos_reservados - cantidad_alumnos_reales) > 5
;
```

Resultado:

1	110	89
2	56	50
2	100	53
2	90	78
3	42	22
3	54	46
3	21	15

3	110	98
4	120	103
6	65	52
7	210	190
8	32	26

16. Obtenga la cantidad de escuelas que visitarán el parque después del '3/6/2004'.

```
SELECT count(distinct codigo_escuela)
  FROM reserva
 WHERE fecha_visita_reservada > '20040603'  ;
```

Resultado:
1

CLASE 3 - Adicionales

11. Insertar un nuevo guía con el número inmediato consecutivo al máximo existente.

```
INSERT INTO guia
SELECT MAX(codigo_guia) + 1, 'Gonzalez', 'Roberto'
  FROM guia;
```

12. Insertar un nuevo tipo de visita con el número inmediato consecutivo al máximo existente sin utilizar subconsultas.

```
  INSERT INTO tipo_visita
  SELECT TOP 1 codigo_tipo_visita + 1, 'NUEVA VISITA'
    FROM tipo_visita
ORDER BY codigo_tipo_visita DESC ;
```

13. Insertar los datos de la tabla escuela en una nueva tabla, borre los datos de la tabla escuela. En la nueva tabla realice una actualización de los códigos de escuela incrementándolos en uno. Posteriormente reinsértelos en la tabla escuela y vuelva a la normalidad los códigos.

```
SELECT * INTO escuela2 FROM ESCUELA ;

DELETE FROM ESCUELA ;

UPDATE escuela2 SET codigo_escuela = codigo_escuela + 1 ;

INSERT INTO escuela
SELECT * FROM escuela2 ;

UPDATE ESCUELA SET codigo_escuela = codigo_escuela - 1 ;

DROP TABLE escuela2 ;
```

14. Las compañías telefónicas han decidido (por falta de números telefónicos!!), que todas

las líneas deben agregar un 9 como primer número. Realice la actualización correspondiente en los teléfonos de las escuelas.

```
UPDATE telefono_escuela SET telefono_escuela = '9' +
telefono_escuela ;
```

15. Debido a un feriado inesperado, las fechas de las visitas deben posponerse por un día.

```
UPDATE reserva SET fecha_visita_reservada =
fecha_visita_reservada + 1 ;
```

16. Obtener los datos de la última reserva existente.

```
  SELECT TOP 1 *
    FROM reserva
ORDER BY fecha_visita_reservada DESC, hora_visita_reservada ASC ;
```

Resultado:
2 2004-12-23 00:00:00 1900-01-01 09:00:00 2

17. Obtener los apellidos de los guias que se encuentren repetidos.

```
  SELECT COUNT(apellido_guia) AS repetidos, apellido_guia
    FROM guia
GROUP BY apellido_guia
  HAVING COUNT(apellido_guia)> 1 ;
```

Resultado:
3 Fernandez

18. Obtener un listado con la cantidad de reservas por fecha.

```
  SELECT fecha_visita_reservada, COUNT(*) AS cantidad_de_reservas
    FROM reserva
GROUP BY fecha_visita_reservada ;
```

Resultado:
2000-04-10 00:00:00 1
2002-06-14 00:00:00 1
2003-05-10 00:00:00 5
2003-05-11 00:00:00 1
2004-12-23 00:00:00 1

19. Obtener el promedio de alumnos asistentes, reservados y la diferencia entre estos promedios.

```
SELECT AVG(cantidad_alumnos_reales),
       AVG(cantidad_alumnos_reservados),
       AVG(cantidad_alumnos_reservados) - AVG(cantidad_alumnos_reales)
  FROM reserva_tipo_visita ;
```

Resultado:
67 78 11

20. Obtener los guias que tengan más de 3 visitas

```
  SELECT codigo_guia, COUNT(codigo_tipo_visita)
    FROM reserva_tipo_visita
GROUP BY codigo_guia
  HAVING COUNT(codigo_tipo_visita) > 3 ;
```

Resultado:
2 7
3 4
5 5

CLASE 4

1. Listar las escuelas cuyos domicilios sean de calles que empiecen con S, indicando nombre, domicilio y teléfono. Hacer una versión en la que aparezcan sólo las que tienen teléfono, y hacer otra en la que aparezca solo las escuelas con domicilio en calles que empiecen con S y que no tienen ningún teléfono.

```
SELECT ESC.Nombre_Escuela, ESC.Domicilio_Escuela, TEL.Telefono_Escuela
  FROM escuela ESC INNER JOIN telefono_escuela TEL ON
       ESC.Codigo_Escuela = TEL.Codigo_Escuela
 WHERE ESC.Domicilio_Escuela Like 'S%';
```

Resultado:
Escuela Domingo Faustino Sarmiento Soler 219 Capital Federal 4591-1221
Escuela Domingo Faustino Sarmiento Soler 219 Capital Federal 4591-2411
Escuela Domingo Faustino Sarmiento Soler 219 Capital Federal 4591-2413
Escuela Nacional Nro 4 Nicolas Avellaneda Salvador y Nicaragua s/n Córdoba 5151-5151
Escuela Nacional Nro 4 Nicolas Avellaneda Salvador y Nicaragua s/n Córdoba 5451-5151

```
SELECT ESC.Nombre_Escuela, ESC.Domicilio_Escuela, TEL.Telefono_Escuela
  FROM escuela ESC LEFT JOIN telefono_escuela TEL ON
       ESC.Codigo_Escuela = TEL.Codigo_Escuela
 WHERE ESC.Nombre_Escuela Like 'S%' AND TEL.Codigo_Escuela Is Null;
```

Resultado:
Vacio (no hay escuelas sin teléfono)

2. Listar las reservas mostrando día, nombre de escuela, cantidad de alumnos de reserva y el nombre del guía.

```
SELECT RES.Numero_Reserva,
       RES.Fecha_Visita_Reservada, ESC.Nombre_Escuela,
```

```
        REV.Cantidad_Alumnos_Reservados, GUI.Nombre_Guia,
        GUI.Apellido_Guia
  FROM guia GUI INNER JOIN ((escuela ESC INNER JOIN reserva RES ON
        ESC.Codigo_Escuela = RES.Codigo_Escuela) INNER JOIN
        Reserva_Tipo_Visita REV ON RES.Numero_Reserva =
        REV.Numero_Reserva) ON GUI.Codigo_Guia = REV.Codigo_Guia;
```

Resultado:

1	2000-04-10 00:00:00	Escuela Domingo Faustino Sarmiento	30	Victor Manuel	Grondona Echague
1	2000-04-10 00:00:00	Escuela Domingo Faustino Sarmiento	25	Victor Manuel	Grondona Echague
1	2000-04-10 00:00:00	Escuela Domingo Faustino Sarmiento	110	Victor Hugo	Morales
2	2004-12-23 00:00:00	Instituto Santa Elena	56	Victor Hugo	Morales
2	2004-12-23 00:00:00	Instituto Santa Elena	100	Cristina	Zaluzi
2	2004-12-23 00:00:00	Instituto Santa Elena	90	Eleonora	Fernandez
3	2002-06-14 00:00:00	Escuela Domingo Faustino Sarmiento	42	Eleonora	Fernandez
3	2002-06-14 00:00:00	Escuela Domingo Faustino Sarmiento	54	Victor Manuel	Grondona Echague
3	2002-06-14 00:00:00	Escuela Domingo Faustino Sarmiento	21	Victor Manuel	Grondona Echague
3	2002-06-14 00:00:00	Escuela Domingo Faustino Sarmiento	110	Cristina	Zaluzi
4	2003-05-10 00:00:00	Escuela Nacional Nro 4 Nicolas Avellaneda	120	Victor Manuel	Grondona Echague
4	2003-05-10 00:00:00	Escuela Nacional Nro 4 Nicolas Avellaneda	130	Victor Hugo	Morales
5	2003-05-11 00:00:00	Escuela Técnica El Tornillo	95	Eleonora	Fernandez
5	2003-05-11 00:00:00	Escuela Técnica El Tornillo	114	Cristina	Zaluzi
6	2003-05-10 00:00:00	Instituto Santa Elena	65	Cristina	Zaluzi
7	2003-05-10 00:00:00	Escuela Técnica El Tornillo	210	Victor Manuel	Grondona Echague
8	2003-05-10 00:00:00	Escuela de Modelitos Las Lolitas	25	Victor Manuel	Grondona Echague
8	2003-05-10 00:00:00	Escuela de Modelitos Las Lolitas	32	Victor Hugo	Morales
9	2003-05-10 00:00:00	Escuela Domingo Faustino Sarmiento	70	Cristina	Zaluzi

3. Listar las reservas, la cantidad total real de alumnos y el valor total (cantidad x arancel con iva incluido).

```
SELECT REV.Numero_Reserva,
       Sum(REV.Cantidad_Alumnos_Reales ) AS TOTAL_Alumn_Asistentes,
       Sum(REV.Cantidad_Alumnos_Reales * TVI.Arancel_por_Alumno * 1.21)
       AS VALOR_TOTAL
  FROM tipo_visita TVI INNER JOIN Reserva_Tipo_Visita REV ON
       TVI.Codigo_Tipo_Visita = REV.Codigo_Tipo_Visita
GROUP BY REV.Numero_Reserva;
```

Resultado:

1	135	597.1350
2	181	989.7800
3	181	1029.7100
4	228	1828.9150

5	202	869.9900
6	52	377.5200
7	190	1264.4500
8	46	366.6300
9	68	493.6800

4. Listar las reservas y los nombres de escuelas con valor total mayor a $1.000.

```
  SELECT ESC.Nombre_Escuela, REV.Numero_Reserva,
         Sum(REV.Cantidad_Alumnos_Reales) AS TOTAL_CantAlumn_Asistentes,
         Sum(REV.Cantidad_Alumnos_Reales * TVI.Arancel_por_Alumno * 1.21)
         AS VALOR_TOTAL
    FROM (escuela ESC INNER JOIN (Reserva_Tipo_Visita AS REV INNER JOIN
          reserva ON REV.Numero_Reserva = reserva.Numero_Reserva) ON
          ESC.Codigo_Escuela = reserva.Codigo_Escuela ) INNER JOIN
          tipo_visita TVI ON (TVI.Codigo_Tipo_Visita =
          REV.Codigo_Tipo_Visita)
GROUP BY ESC.Nombre_Escuela, REV.Numero_Reserva
  HAVING Sum(REV.Cantidad_Alumnos_Reales*TVI.Arancel_por_Alumno*1.21)>1000;
```

Resultado:

Escuela Domingo Faustino Sarmiento	3	181	1029.7100
Escuela Nacional Nro 4 Nicolas Avellaneda	4	228	1828.9150
Escuela Técnica El Tornillo	7	190	1264.4500

5. Listar las escuelas que fueron atendidas alguna vez por el guía “Eleonora Fernandez”.

```
  SELECT ESC.Nombre_Escuela
    FROM (escuela ESC INNER JOIN reserva RES ON ESC.Codigo_Escuela =
          RES.Codigo_Escuela) INNER JOIN (guia GUI INNER JOIN
          Reserva_Tipo_Visita REV ON GUI.Codigo_Guia = REV.Codigo_Guia) ON
          RES.Numero_Reserva = REV.Numero_Reserva
   WHERE GUI.Nombre_Guia='Eleonora' AND GUI.Apellido_Guia='Fernandez'
GROUP BY ESC.Nombre_Escuela;
```

Resultado:
Escuela Domingo Faustino Sarmiento
Escuela Técnica El Tornillo
Instituto Santa Elena

6. Listar las escuelas que realizaron más de una reserva para el mismo día.

```
  SELECT ESC.Nombre_Escuela, RES.Numero_Reserva,
         RES.Fecha_Visita_Reservada
    FROM (escuela ESC INNER JOIN reserva RES ON ESC.Codigo_Escuela =
          RES.Codigo_Escuela) INNER JOIN Reserva_Tipo_Visita REV ON
          RES.Numero_Reserva = REV.Numero_Reserva
GROUP BY ESC.Nombre_Escuela, RES.Numero_Reserva,
         RES.Fecha_Visita_Reservada
  HAVING Count(RES.Fecha_Visita_Reservada)>1;
```

Resultado:

Escuela Domingo Faustino Sarmiento	1	2000-04-10 00:00:00

Instituto Santa Elena 2 2004-12-23 00:00:00
Escuela Domingo Faustino Sarmiento 3 2002-06-14 00:00:00
Escuela Nacional Nro 4 Nicolas Avellaneda 4 2003-05-10 00:00:00
Escuela Técnica El Tornillo 5 2003-05-11 00:00:00
Escuela de Modelitos Las Lolitas 8 2003-05-10 00:00:00

7. Listar los nombres de los guías y la cantidad de visitas para aquellos con más de 3 visitas de 30 personas.

```
SELECT GUI.Nombre_Guia, GUI.Apellido_Guia,
       Count(REV.Numero_Reserva) AS CANT_TOTAL_RESERVAS ,
       Sum(REV.Cantidad_Alumnos_Reales) AS
       Cantidad_TOTAL_Alumnos_Asistentes
  FROM ((escuela ESC INNER JOIN reserva RES ON ESC.Codigo_Escuela =
       RES.Codigo_Escuela) INNER JOIN Reserva_Tipo_Visita REV ON
       RES.Numero_Reserva = REV.Numero_Reserva) INNER JOIN guia GUI ON
       REV.Codigo_Guia = GUI.Codigo_Guia
 WHERE REV.Cantidad_Alumnos_Reales > 30
GROUP BY GUI.Nombre_Guia, GUI.Apellido_Guia
HAVING Count(REV.Numero_Reserva) > 3  ;
```

Resultado:
Cristina Zaluzi 5 383

8. Listar las reservas, el día, la cantidad total de alumnos por tipo de visita, el nombre de la escuela y el nombre del guía.

```
SELECT ESC.Nombre_Escuela, GUI.Apellido_Guia, GUI.Nombre_Guia,
       RES.Numero_Reserva, TV.Descripcion_Tipo_Visita,
       Sum(REV.Cantidad_Alumnos_Reales) AS
       TOTAL_Cantidad_Alumnos_Reales
  FROM tipo_visita TV INNER JOIN (escuela ESC INNER JOIN (guia GUI
       INNER JOIN (reserva RES INNER JOIN Reserva_Tipo_Visita REV ON
       RES.Numero_Reserva = REV.Numero_Reserva) ON GUI.Codigo_Guia =
       REV.Codigo_Guia) ON ESC.Codigo_Escuela = RES.Codigo_Escuela) ON
       TV.Codigo_Tipo_Visita = REV.Codigo_Tipo_Visita
GROUP BY ESC.Nombre_Escuela, GUI.Apellido_Guia, GUI.Nombre_Guia,
       RES.Numero_Reserva, TV.Descripcion_Tipo_Visita;
```

Resultado:
Escuela de Modelitos Las Lolitas Grondona Echague Victor Manuel 8 Parque de Brontosaurios 20
Escuela de Modelitos Las Lolitas Morales Victor Hugo 8 Los Mamuts en Familia 26
Escuela Domingo Faustino Sarmiento Fernandez Eleonora 3 Parque de Brontosaurios 22
Escuela Domingo Faustino Sarmiento Grondona Echague Victor Manuel 1 Los Mamuts en Familia 21
Escuela Domingo Faustino Sarmiento Grondona Echague Victor Manuel 1 Parque de Brontosaurios 25
Escuela Domingo Faustino Sarmiento Grondona Echague Victor Manuel 3 Las Tortugas Prehistoricas 15
Escuela Domingo Faustino Sarmiento Grondona Echague Victor Manuel 3 Los Mamuts en Familia 46
Escuela Domingo Faustino Sarmiento Morales Victor Hugo 1 Las Tortugas Prehistoricas 89
Escuela Domingo Faustino Sarmiento Zaluzi Cristina 3 Laguna de Nahuelito 98
Escuela Domingo Faustino Sarmiento Zaluzi Cristina 9 Las Cenizas del Meteorito Mortal 68
Escuela Nacional Nro 4 Nicolas Avellaneda Grondona Echague Victor Manuel 4 Parque de Brontosaurios 103
Escuela Nacional Nro 4 Nicolas Avellaneda Morales Victor Hugo 4 Los Mamuts en Familia 125
Escuela Técnica El Tornillo Fernandez Eleonora 5 Los Mamuts en Familia 90
Escuela Técnica El Tornillo Grondona Echague Victor Manuel 7 Los Mamuts en Familia 190
Escuela Técnica El Tornillo Zaluzi Cristina 5 Las Tortugas Prehistoricas 112
Instituto Santa Elena Fernandez Eleonora 2 Laguna de Nahuelito 78

Instituto Santa Elena	Morales	Victor Hugo	2	Parque de Brontosaurios	50	
Instituto Santa Elena	Zaluzi	Cristina	2	Las Tortugas Prehistoricas	53	
Instituto Santa Elena	Zaluzi	Cristina	6	Las Cenizas del Meteorito Mortal	52	

CLASE 4 - Adicionales

9. Listar los tipos de visita (Codigo y Descripción) que han sido asignadas más de 5 veces.

```
  SELECT TV.Codigo_Tipo_Visita, TV.Descripcion_Tipo_Visita
    FROM Reserva_Tipo_Visita REV INNER JOIN tipo_visita TV ON
         REV.Codigo_Tipo_Visita = TV.Codigo_Tipo_Visita
GROUP BY TV.Codigo_Tipo_Visita, TV.Descripcion_Tipo_Visita
  HAVING Count(REV.Codigo_Tipo_Visita)> 5;
```

Resultado:
2 Los Mamuts en Familia

10. Listar los Guias (Apellido y Nombre) que no han sido nunca asignados a una reserva.

```
  SELECT GUI.Apellido_Guia, GUI.Nombre_Guia
    FROM guia GUI LEFT JOIN Reserva_Tipo_Visita REV ON GUI.Codigo_Guia =
         REV.Codigo_Guia
   WHERE REV.Codigo_Guia Is Null
GROUP BY GUI.Apellido_Guia, GUI.Nombre_Guia;
```

Resultado:
Fernandez Juan
Fernandez Maria
Gonzalez Roberto
Lopez Aguado Bernardo

11. Listar los Guias (Apellido y Nombre) con visitas de escuelas cuyo nombre no comience con “E”.

```
  SELECT GUI.Apellido_Guia, GUI.Nombre_Guia
    FROM (escuela ESC INNER JOIN reserva RES ON ESC.Codigo_Escuela =
         RES.Codigo_Escuela) INNER JOIN (guia GUI INNER JOIN
         Reserva_Tipo_Visita REV ON GUI.Codigo_Guia = REV.Codigo_Guia) ON
         RES.Numero_Reserva = REV.Numero_Reserva
   WHERE ESC.Nombre_Escuela Not Like 'E%'
GROUP BY GUI.Apellido_Guia, GUI.Nombre_Guia;
```

Resultado:
Fernandez Eleonora
MoralesVictor Hugo
Zaluzi Cristina

12. Listar las escuelas (Nombre) que tengan visitas en donde la cantidad de alumnos reservada sea igual a la cantidad de alumnos reales asistentes.

```
  SELECT ESC.Nombre_Escuela
```

```
      FROM (escuela ESC INNER JOIN reserva RES ON ESC.Codigo_Escuela =
            RES.Codigo_Escuela) INNER JOIN Reserva_Tipo_Visita REV ON
            RES.Numero_Reserva = REV.Numero_Reserva
     WHERE REV.Cantidad_Alumnos_Reservados=REV.Cantidad_Alumnos_Reales
GROUP BY ESC.Nombre_Escuela;
```

Resultado:
Vacio (ninguna reserva cumple esa condición)

13. Realizar la Union de las consultas:
    Nombres de las escuelas con reservas antes de las 9:00 hs
    Apellidos de los guías cuyos nombres comiencen con 'V'
    Ordenar el resultado en forma descendente

```
(
SELECT ESC.Nombre_Escuela
  FROM escuela AS ESC INNER JOIN reserva AS RES ON ESC.Codigo_Escuela =
       RES.Codigo_Escuela
 WHERE RES.Hora_Visita_Reservada< '9:00:00 AM'
)
UNION
(
SELECT GUI.Apellido_Guia
  FROM guia GUI
 WHERE GUI.Nombre_Guia Like 'V%'
)
ORDER BY 1 DESC;
```

Resultado:
Morales
Grondona Echague
Escuela Domingo Faustino Sarmiento

14. Listar los Nombres y Teléfonos de las Escuelas que concurrieron a la visita: 'Los Mamuts en Familia'

```
SELECT TV.Descripcion_Tipo_Visita, ESC.Nombre_Escuela,
       TEL.Telefono_Escuela
  FROM ((escuela ESC INNER JOIN telefono_escuela TEL ON
       ESC.Codigo_Escuela = TEL.Codigo_Escuela) INNER JOIN reserva RES
       ON ESC.Codigo_Escuela = RES.Codigo_Escuela) INNER JOIN
       (tipo_visita TV INNER JOIN Reserva_Tipo_Visita REV ON
       TV.Codigo_Tipo_Visita = REV.Codigo_Tipo_Visita) ON
       RES.Numero_Reserva = REV.Numero_Reserva
 WHERE TV.Descripcion_Tipo_Visita='Los Mamuts en Familia'
GROUP BY TV.Descripcion_Tipo_Visita, ESC.Nombre_Escuela,
       TEL.Telefono_Escuela;
```

Resultado:

Los Mamuts en Familia	Escuela de Modelitos Las Lolitas	5445-3224
Los Mamuts en Familia	Escuela Domingo Faustino Sarmiento	4591-1221
Los Mamuts en Familia	Escuela Domingo Faustino Sarmiento	4591-2411
Los Mamuts en Familia	Escuela Domingo Faustino Sarmiento	4591-2413
Los Mamuts en Familia	Escuela Nacional Nro 4 Nicolas Avellaneda	5151-5151
Los Mamuts en Familia	Escuela Nacional Nro 4 Nicolas Avellaneda	5451-5151

15. Listar para cada grado (sin importar de cual escuela) cual fue la última vez que hicieron una visita

```
 SELECT RVG.Numero_Grado, Max(RES.Fecha_Visita_Reservada) AS
        Ultima_Reserva
   FROM reserva RES INNER JOIN Reserva_Por_Grado RVG ON
        RES.Numero_Reserva = RVG.Numero_Reserva
GROUP BY RVG.Numero_Grado;
```

Resultado:

1	2002-06-14 00:00:00
2	2003-05-10 00:00:00
3	2003-05-10 00:00:00
4	2004-12-23 00:00:00
5	2004-12-23 00:00:00
6	2003-05-11 00:00:00
7	2004-12-23 00:00:00

16. Insertar en una nueva tabla llamada Guia_Performance los datos del Guia y las sumas de las cantidades reservadas y reales del año 2003.

```
 SELECT GUI.Codigo_Guia, GUI.Apellido_Guia, GUI.Nombre_Guia,
        Sum(REV.Cantidad_Alumnos_Reservados) AS
        Cantidad_Alumnos_Reservados, Sum(REV.Cantidad_Alumnos_Reales) AS
        Cantidad_Alumnos_Reales INTO Guia_Performance
   FROM (guia AS GUI INNER JOIN Reserva_Tipo_Visita AS REV ON
        GUI.Codigo_Guia = REV.Codigo_Guia) INNER JOIN reserva ON
        REV.Numero_Reserva = reserva.Numero_Reserva
  WHERE Year(Fecha_Visita_Reservada)=2003
GROUP BY GUI.Codigo_Guia, GUI.Apellido_Guia, GUI.Nombre_Guia;
```

Resultado (en la nueva tabla):

2	Grondona Echague	Victor Manuel	355	313
3	MoralesVictor Hugo	162	151	
4	Fernandez	Eleonora	95	90
5	Zaluzi	Cristina 249	232	

CLASE 5

1. Listar los tipos de visita que fueron guiadas alguna vez por Cristina Zaluzi.

```
SELECT distinct tv.descripcion_tipo_visita
  FROM Reserva_Tipo_Visita rev INNER JOIN tipo_visita tv
       ON rev.codigo_tipo_visita = tv.codigo_tipo_visita
WHERE rev.codigo_guia IN
     (
      SELECT gui.Codigo_Guia
```

```
          FROM guia gui
         WHERE gui.Apellido_Guia = 'Zaluzi'
               AND gui.Nombre_Guia = 'Cristina'
        );
```

Resultado (en la nueva tabla):
Laguna de Nahuelito
Las Cenizas del Meteorito Mortal
Las Tortugas Prehistoricas

2Listar los nombres de escuela que visitaron en 2002 pero no lo hicieron en el 2001.

```
SELECT Nombre_Escuela
  FROM escuela esc
 WHERE Codigo_Escuela not in (
       SELECT res1.codigo_escuela
         FROM reserva res1 WHERE YEAR(res1.Fecha_Visita_Reservada) =
               2001) AND esc.Codigo_Escuela IN (
                              SELECT res2.codigo_escuela
                                FROM reserva res2
                               WHERE YEAR(res2.Fecha_Visita_Reservada)
                                          = 2002)
```

Resultado:
Escuela Domingo Faustino Sarmiento

3. Listar los guías (codigo, nombre y apellido) que fueron asignados a más de 2 tipos de visita distintos y con una cantidad total real de alumnos que guiaron, mayor a 200.

```
SELECT gui.codigo_guia, gui.nombre_guia, gui.apellido_guia
  FROM guia gui
 WHERE gui.codigo_guia IN
      (SELECT rev.codigo_guia
         FROM Reserva_Tipo_Visita rev
     GROUP BY rev.codigo_guia
       HAVING COUNT (DISTINCT (rev.codigo_tipo_visita)) > 2 AND
              SUM(rev.Cantidad_Alumnos_Reales) > 200) ;
```

Resultado:

2	Victor Manuel	Grondona Echague
3	Victor Hugo	Morales
5	Cristina Zaluzi	

4. Listar las escuelas que poseen más de 1 reservas con más de 2 tipos de visitas para cada reserva.

```
SELECT esc.codigo_escuela, nombre_escuela
  FROM escuela esc
 WHERE esc.codigo_escuela IN
       (
       SELECT res.codigo_escuela
         FROM reserva res
        WHERE res.Numero_Reserva IN
                        (
                  SELECT rev.Numero_Reserva
                    FROM Reserva_Tipo_Visita rev
                GROUP BY rev.Numero_Reserva
                  HAVING COUNT (rev.codigo_tipo_visita)> 2
                        )
     GROUP BY res.codigo_escuela
       HAVING COUNT(res.Numero_Reserva) > 1
        );
```

Resultado:

1 Escuela Domingo Faustino Sarmiento

5. Listar el nombre, apellido y código de aquellos guías que, en alguna visita de una reserva en particular, hayan atendido por lo menos al 40% de los alumnos totales guiados en todas sus visitas.

```
SELECT gui.codigo_guia, gui.apellido_guia, gui.nombre_guia
  FROM guia gui
 WHERE gui.codigo_guia in
      (
      SELECT rev1.codigo_guia
        FROM  Reserva_Tipo_Visita rev1
       WHERE rev1.Cantidad_Alumnos_Reales >
            (
            SELECT SUM(Cantidad_Alumnos_Reales)*0.40
              FROM Reserva_Tipo_Visita rev2
             WHERE rev2.codigo_guia = rev1.codigo_guia
            )
      );
```

Resultado:

2 Grondona Echague Victor Manuel
3 MoralesVictor Hugo
4 Fernandez Eleonora

6. Listar el nombre y el código de aquellas escuelas que hayan asistido el día en que se registró la mayor cantidad de alumnos reales.

```
SELECT esc.codigo_escuela, esc.nombre_escuela
  FROM escuela esc
 WHERE esc.codigo_escuela IN
      (
      SELECT res.codigo_escuela
```

```
  FROM reserva res
 WHERE res.fecha_visita_reservada =
       (
         SELECT TOP 1 res2.fecha_visita_reservada
           FROM reserva res2 INNER JOIN Reserva_Tipo_Visita rev ON
                res2.Numero_Reserva = rev.Numero_Reserva
       GROUP BY res2.fecha_visita_reservada
       ORDER BY SUM(rev.Cantidad_Alumnos_Reales) DESC
       )
) ;
```

Resultado:
1 Escuela Domingo Faustino Sarmiento
2 Instituto Santa Elena
3 Escuela Nacional Nro 4 Nicolas Avellaneda
4 Escuela Técnica El Tornillo
5 Escuela de Modelitos Las Lolitas

CLASE 5 - Adicionales

6. Listar el código y nombre de las escuelas cuya fecha de reserva sea igual a la primera fecha de reserva realizada.

```
SELECT esc.codigo_escuela, esc.nombre_escuela
  FROM escuela esc INNER JOIN reserva res ON
       esc.codigo_escuela=res.codigo_escuela
 WHERE res.Fecha_Visita_Reservada =
       (
       SELECT MIN(Fecha_Visita_Reservada)
         FROM reserva
       );
```

Resultado:
1 Escuela Domingo Faustino Sarmiento

7. Listar las escuelas que visitaron entre los años 2001 y en el 2002.

```
SELECT esc.Nombre_Escuela
  FROM escuela esc
 WHERE esc.Codigo_Escuela IN
      (
      SELECT res1.codigo_escuela
        FROM reserva res1
        WHERE YEAR(res1.Fecha_Visita_Reservada) BETWEEN 2001 AND 2002
      );
```

Resultado:
1 Escuela Domingo Faustino Sarmiento

8. Listar los guías que tuvieron más de 3 escuelas diferentes y una cantidad total real de alumnos mayor a 200.

```
SELECT gui.codigo_guia, gui.nombre_guia, gui.apellido_guia
  FROM guia gui
 WHERE gui.codigo_guia IN
```

```
        (
          SELECT rev.codigo_guia
            FROM Reserva_Tipo_Visita rev INNER JOIN reserva res ON
                 rev.numero_reserva=res.numero_reserva
        GROUP BY rev.codigo_guia
          HAVING COUNT (DISTINCT (res.codigo_escuela)) > 3 AND
                 SUM(rev.cantidad_alumnos_reales) > 100
           );
```

Resultado:

2	Victor Manuel	Grondona Echague
3	Victor Hugo	Morales

9. Listar los nombres y códigos de escuelas con gasto total de todas las visitas mayor a $1700.

```
SELECT esc.Nombre_Escuela
  FROM escuela esc
 WHERE esc.Codigo_Escuela IN
      (
      SELECT res.codigo_escuela
        FROM reserva res INNER JOIN reserva_tipo_visita rev ON
             res.numero_reserva = rev.numero_reserva  INNER JOIN
             tipo_visita tv ON tv.codigo_tipo_visita =
             rev.codigo_tipo_visita
    GROUP BY res.codigo_escuela
      HAVING ( SUM( (tv.Arancel_por_alumno) * rev.cantidad_alumnos_reales)
               > 1700)
         );

Nota: En caso que el arancel se encuentre en la tabla Reserva_Tipo_Visita,
no es necesario el Inner Join con la tabla Tipo_Visita.
```

Resultado:
Escuela Domingo Faustino Sarmiento
Escuela Técnica El Tornillo

10. Listar los guías que hayan tenido en solo un tipo de visita de una reserva en particular por lo menos el 45% del total de alumnos totales que esa persona atendió.

```
SELECT gui.codigo_guia, gui.apellido_guia, gui.nombre_guia
  FROM guia gui WHERE gui.codigo_guia IN
      (
      SELECT rev1.codigo_guia
        FROM  Reserva_Tipo_Visita rev1
       WHERE rev1.cantidad_alumnos_reales >
            (
            SELECT SUM(cantidad_alumnos_reales)*0.45
              FROM Reserva_Tipo_Visita rev2
             WHERE rev2.codigo_guia = rev1.codigo_guia
            )
    );
```

Resultado:

2	Grondona Echague	Victor Manuel

CLASE 6

EJERCICIOS: PROCEDIMIENTOS ALMACENADOS

1. Crear un procedimiento almacenado que tenga como parámetro de entrada un número entero (Codigo_Guia) y devuelva la cantidad total de Alumnos Reales asignados a ese guía.

```
CREATE PROCEDURE Alum_Reales_Por_Guia @Cod_Guia int AS
DECLARE @Sum_Alumnos_Reales int

SELECT @Sum_Alumnos_Reales = SUM(Cantidad_Alumnos_Reales)
  FROM RESERVA_TIPO_VISITA
 WHERE Codigo_Guia= @Cod_Guia

RETURN @Sum_Alumnos_Reales

Ejecucion:
DECLARE @resultado int
EXEC @resultado = Alum_Reales_Por_Guia (1250)
```

2. Crear un procedimiento almacenado para ingresar nuevas escuelas, que tenga como parámetros de entrada los valores de Código_Escuela, Nombre_Escuela, y Domicilio_Escuela.

```
CREATE PROCEDURE Ingresar_Escuela @Cod_Escuela smallint,
                                  @Nom_Escuela varchar(100),
                                  @Dom_Escuela varchar(200) AS

INSERT INTO ESCUELA (Codigo_Escuela, Nombre_Escuela, Domicilio_Escuela)
VALUES ( @Cod_Escuela, @Nom_Escuela, @Dom_Escuela )

RETURN 0

Ejecucion:
EXEC Ingresar_Escuela '562', 'Esc. No. 10 DE 11', 'Lafuente 451 Capital
Federal'
```

3. Crear un procedimiento almacenado que liste los nombres de los guías con apellidos que comiencen con una cadena de texto determinada por un parámetro de entrada (con valor predeterminado es: comienza con A).

```
CREATE PROCEDURE ListaGuias @Apell char(20) = 'A%' AS

SELECT Nombre_Guia, Apellido_Guia
```

```
 FROM GUIA
WHERE Apellido_Guia LIKE @Apell

Ejecucion:

EXEC ListaGuias 'A%'
-- resultado: lista guías cuyo apellido empieza con A

EXEC ListaGuias 'G%'
-- resultado: lista guías cuyo apellido empieza con G
```

EJERCICIOS: TRIGGERS

4. Crear un trigger sobre la tabla escuela que en caso de que se borre una fila de ésta elimine las respectivas filas de la tabla telefono_escuela.

```
CREATE TRIGGER trig_delete ON escuela FOR DELETE AS

DELETE FROM telefono_escuela
      WHERE codigo_escuela IN
                              (
                              SELECT codigo_Escuela
                                FROM deleted
                              ) ;
```

5. Crear un trigger que en caso de UPDATE del codigo_guia de la tabla Guia, haga el correspondiente update en la tabla reserva_visita.

```
CREATE TRIGGER trig_update ON guia FOR UPDATE AS
   IF (columns_updated() & 1)  > 0
     BEGIN
       DECLARE @cod_guia INT

      SELECT @cod_guia =
            (SELECT codigo_guia FROM inserted)

          UPDATE reserva_visita SET codigo_guia =   @cod_guia
           WHERE codigo_guia = (SELECT codigo_guia FROM deleted)

      END
```

6. Crear un trigger que en caso de inserción sobre la tabla telefono_escuela verifique la existencia del código de escuela correspondiente y en caso de no encontrarla no realice el proceso.

```
CREATE TRIGGER trig_insert ON telefono_escuela
            INSTEAD OF INSERT
            AS
            IF EXISTS( SELECT codigo_escuela
```

```
                    FROM escuela
                   WHERE codigo_escuela = (
                         SELECT codigo_escuela FROM inserted))
BEGIN
   INSERT INTO telefono_escuela SELECT * FROM inserted
END
ELSE ROLLBACK TRAN
```

CLASE 6 - Adicionales

PROCEDIMIENTOS ALMACENADOS Y TRIGGERS

1. Crear un stored procedure que realice una inserción de 26 escuelas en forma secuencial de la forma:

1 Nom_Escuela1 Dir_Escuela1
2 Nom_Escuela2 Dir_Escuela2
3 Nom_Escuela3 Dir_Escuela3

```
CREATE PROC insterta_valores_secuenciales AS
SET NOCOUNT ON
DECLARE @contador INT
SET @contador = 0
WHILE (@contador < 26)
BEGIN
   INSERT INTO escuela (codigo_escuela,
nombre_escuela, domicilio_escuela)
VALUES (@contador
, 'NOM_ESC' +  CAST(@contador AS
CHAR(7)), 'DOM_ESC' +  CAST(@contador AS
CHAR(7)) )
   SET @contador = @contador + 1
END
SET NOCOUNT OFF
RETURN 0
```

2. Crear un stored procedure que inserte valores en la tabla de Guía (Codigo_Guia, Apellido y Nombre) pero que además de estos parámetros requiera UserID (char) y Password (char). Estos dos parámetros deben ser validados (deben existir) en una tabla llamada USERID.

```
CREATE PROC inserta_guia @codigo_guia INT, @apellido CHAR(20),
                         @nombre CHAR(20),
   @userid CHAR(20), @password CHAR(20) AS
IF ( SELECT COUNT(*)
 FROM userid
 WHERE userid=@userid AND pass=@password)=0
 PRINT 'EL USUARIO NO TIENE ACCESO A LA BASE DE DATOS'
 ELSE
 BEGIN
 INSERT INTO guia(codigo_guia, apellido_guia, nombre_guia)
```

```
 VALUES(@codigo_guia, @apellido, @nombre)
END
```

3. Crear un trigger en donde, si se insertan valores en la tabla Reserva_Visita, si la cantidad de alumnos asistentes es mayor a 50, inserte en una tabla llamada AUDITORIA los valores insertados.

```
CREATE TRIGGER TR_RV ON RESERVA_TIPO_VISITA FOR INSERT AS
IF (SELECT SUM(cantidad_alumnos_reales) FROM inserted) >50
BEGIN

INSERT INTO auditoria (codigo_reserva, codigo_tipo_visita, codigo_guia
,cantidad_alumnos_reservados, cantidad_alumnos_reales,
arancel_por_alumno)
      SELECT *
        FROM inserted

END
```